

Namespace MDBX

Classes

[MdbxCursor](#)

Высокоуровневая обертка курсора libmdbx.

[MdbxDatabase](#)

Высокоуровневая обертка таблицы (DBI) MDBX.

[MdbxEntry](#)

Пара ключ-значение, прочитанная из таблицы через курсор. Ключ и значение представлены в виде "сырых" байтов и копируются из нативной памяти.

[MdbxEnvironment](#)

Высокоуровневая обертка окружения libmdbx.

[MdbxErrorCodes](#)

Коды ошибок libmdbx.

[MdbxException](#)

Исключение, возникающее при ошибках libmdbx.

[MdbxQuery](#)

Высокоуровневая отложенная (lazy) абстракция над курсором MDBX. Представляет собой [IEnumerable<T>](#) , перебирающий все записи таблицы поверх нативного курсора. Каждый вызов [GetEnumerator\(\)](#) открывает новый курсор, а перебор выполняется лениво, что делает тип полностью совместимым с LINQ (Where, Select, First, ToList и т.д.).

[MdbxQueryExtensions](#)

Методы расширения для [MdbxQuery](#), упрощающие проекцию записей и построение LINQ-запросов, где параметр предиката — это значение или ключ.

[MdbxTransaction](#)

Высокоуровневая обертка транзакции MDBX.

Class MdbxCursor

Namespace: [MDBX](#)

Assembly: mdbx-dotnet.dll

Высокоуровневая обертка курсора libmdbx.

```
public sealed class MdbxCursor : IDisposable
```

Inheritance

[object](#)  ← MdbxCursor

Implements

[IDisposable](#) 

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Properties

NativeCursor

Получает нативный указатель на курсор.

```
public MDBX_cursor* NativeCursor { get; }
```

Property Value

[MDBX_cursor](#) 

Methods

Close()

Закрывает курсор.

```
public void Close()
```

CopyTo(MdbxCursor)

Копирует состояние курсора в другой курсор.

```
public void CopyTo(MdbxCursor dest)
```

Parameters

dest [MdbxCursor](#)

Курсор назначения.

Count()

Получает количество элементов, доступных с текущей позиции курсора.

```
public nuint Count()
```

Returns

[nuint](#)

Количество элементов.

Create(MdbxTransaction, MdbxDatabase)

Создает новый курсор для указанной транзакции и таблицы.

```
public static MdbxCursor Create(MdbxTransaction txn, MdbxDatabase db)
```

Parameters

txn [MdbxTransaction](#)

Транзакция.

`db` [MdbxDatabase](#)

Таблица (DBI).

Returns

[MdbxCursor](#)

Новый курсор.

Delete(MdbxPutFlags)

Удаляет текущий элемент через курсор.

```
public bool Delete(MdbxPutFlags flags = MdbxPutFlags.MDBX_UPSERT)
```

Parameters

`flags` [MdbxPutFlags](#)

Флаги удаления.

Returns

[bool](#)

True если элемент был удален, false если элемент не найден.

Dispose()

Освобождает ресурсы курсора.

```
public void Dispose()
```

~MdbxCursor()

Финализатор.

`protected ~MdbxCursor()`

GetCurrent(out byte[]?)

Получает текущий элемент (ключ и значение).

```
public byte[]? GetCurrent(out byte[]? key)
```

Parameters

key [byte\[\]](#)

Ключ текущего элемента.

Returns

[byte\[\]](#)

Значение текущего элемента или null если курсор не позиционирован.

GetFirst(out byte[]?)

Перемещает курсор на первый элемент.

```
public byte[]? GetFirst(out byte[]? key)
```

Parameters

key [byte\[\]](#)

Ключ первого элемента.

Returns

[byte\[\]](#)

Значение первого элемента или null если таблица пуста.

GetFirstDup(out byte[]?)

Перемещает курсор на первый дубликат текущего ключа.

```
public byte[]? GetFirstDup(out byte[]? key)
```

Parameters

key [byte\[\]](#)

Ключ элемента.

Returns

[byte\[\]](#)

Значение первого дубликата или null если дубликатов нет.

GetLast(out byte[]?)

Перемещает курсор на последний элемент.

```
public byte[]? GetLast(out byte[]? key)
```

Parameters

key [byte\[\]](#)

Ключ последнего элемента.

Returns

[byte\[\]](#)

Значение последнего элемента или null если таблица пуста.

GetLastDup(out byte[]?)

Перемещает курсор на последний дубликат текущего ключа.

```
public byte[]? GetLastDup(out byte[]? key)
```

Parameters

key [byte\[\]](#)

Ключ элемента.

Returns

[byte\[\]](#)

Значение последнего дубликата или null если дубликатов нет.

GetNext(out byte[]?)

Перемещает курсор на следующий элемент.

```
public byte[]? GetNext(out byte[]? key)
```

Parameters

key [byte\[\]](#)

Ключ следующего элемента.

Returns

[byte\[\]](#)

Значение следующего элемента или null если достигнут конец таблицы.

GetNextDup(out byte[]?)

Перемещает курсор на следующий дубликат.

```
public byte[]? GetNextDup(out byte[]? key)
```

Parameters

key [byte](#)[]

Ключ элемента.

Returns

[byte](#)[]

Значение следующего дубликата или null если достигнут конец дубликатов.

GetNextNoDup(out byte[]?)

Перемещает курсор на следующий элемент с другим ключом.

```
public byte[]? GetNextNoDup(out byte[]? key)
```

Parameters

key [byte](#)[]

Ключ следующего элемента.

Returns

[byte](#)[]

Значение следующего элемента с другим ключом или null если достигнут конец таблицы.

GetPrev(out byte[]?)

Перемещает курсор на предыдущий элемент.

```
public byte[]? GetPrev(out byte[]? key)
```

Parameters

key [byte](#)[]

Ключ предыдущего элемента.

Returns

[byte](#)[]

Значение предыдущего элемента или null если достигнуто начало таблицы.

GetPrevDup(out byte[]?)

Перемещает курсор на предыдущий дубликат.

```
public byte[]? GetPrevDup(out byte[]? key)
```

Parameters

key [byte](#)[]

Ключ элемента.

Returns

[byte](#)[]

Значение предыдущего дубликата или null если достигнуто начало дубликатов.

GetPrevNoDup(out byte[]?)

Перемещает курсор на предыдущий элемент с другим ключом.

```
public byte[]? GetPrevNoDup(out byte[]? key)
```

Parameters

key [byte](#)[]

Ключ предыдущего элемента.

Returns

[byte](#)[]

Значение предыдущего элемента с другим ключом или null если достигнуто начало таблицы.

GetSet(byte[], out byte[]?)

Позиционирует курсор на указанный ключ.

```
public byte[]? GetSet(byte[] key, out byte[]? foundKey)
```

Parameters

key [byte](#)[]

Ключ для позиционирования.

foundKey [byte](#)[]

Фактический найденный ключ.

Returns

[byte](#)[]

Значение найденного элемента или null если ключ не существует.

IsEof()

Проверяет, достигнут ли конец таблицы.

```
public bool IsEof()
```

Returns

[bool](#)

True если курсор за пределами таблицы.

IsOnFirst()

Проверяет, находится ли курсор на первом элементе.

```
public bool IsOnFirst()
```

Returns

[bool](#)

True если курсор на первом элементе.

IsOnFirstDup()

Проверяет, находится ли курсор на первом дубликate.

```
public bool IsOnFirstDup()
```

Returns

[bool](#)

True если курсор на первом дубликate.

IsOnLast()

Проверяет, находится ли курсор на последнем элементе.

```
public bool IsOnLast()
```

Returns

[bool](#)

True если курсор на последнем элементе.

IsOnLastDup()

Проверяет, находится ли курсор на последнем дубликате.

```
public bool IsOnLastDup()
```

Returns

[bool](#)

True если курсор на последнем дубликате.

Put(byte[], byte[], MdbxPutFlags)

Записывает пару ключ-значение через курсор.

```
public void Put(byte[] key, byte[] value, MdbxPutFlags flags = MdbxPutFlags.MDBX_UPSERT)
```

Parameters

key [byte](#)[]

Ключ.

value [byte](#)[]

Значение.

flags [MdbxPutFlags](#)

Флаги операции записи.

Put(string, string, MdbxPutFlags)

Записывает строковую пару ключ-значение через курсор (UTF-8).

```
public void Put(string key, string value, MdbxPutFlags flags = MdbxPutFlags.MDBX_UPSERT)
```

Parameters

key [string](#)

Строковый ключ.

value [string](#)[↗]

Строковое значение.

flags [MdbxPutFlags](#)

Флаги операции записи.

Renew(MdbxTransaction)

Обновляет курсор для использования в новой транзакции.

```
public void Renew(MdbxTransaction txn)
```

Parameters

txn [MdbxTransaction](#)

Новая транзакция.

Reset()

Сбрасывает курсор (отвязывает от таблицы).

```
public void Reset()
```

Class MdbxDatabase

Namespace: [MDBX](#)

Assembly: mdbx-dotnet.dll

Высокоуровневая обертка таблицы (DBI) MDBX.

```
public sealed class MdbxDatabase : IDisposable
```

Inheritance

[object](#)  ← MdbxDatabase

Implements

[IDisposable](#) 

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Properties

Dbi

Получает идентификатор таблицы (DBI).

```
public uint Dbi { get; }
```

Property Value

[uint](#) 

Methods

Delete(MdbxTransaction, byte[])

Удаляет значение по ключу.

```
public bool Delete(MdbxTransaction txn, byte[] key)
```

Parameters

txn [MdbxTransaction](#)

Транзакция.

key [byte](#)[]

Ключ.

Returns

[bool](#)

True если элемент был удален, false если не найден.

Delete(MdbxTransaction, string)

Удаляет значение по строковому ключу.

```
public bool Delete(MdbxTransaction txn, string key)
```

Parameters

txn [MdbxTransaction](#)

Транзакция.

key [string](#)

Строковый ключ (UTF-8).

Returns

[bool](#)

True если элемент был удален, false если не найден.

Dispose()

Освобождает ресурсы таблицы.

```
public void Dispose()
```

~MdbxDatabase()

Финализатор.

```
protected ~MdbxDatabase()
```

Find(MdbxTransaction)

Возвращает отложенный (lazy) запрос поверх курсора, перебирающий все записи таблицы. Каждый вызов [GetEnumerator\(\)](#) открывает новый курсор, а перебор выполняется лениво и совместим с LINQ (Where, Select, First, ToList и т.д.).

```
public MdbxQuery Find(MdbxTransaction txn)
```

Parameters

txn [MdbxTransaction](#)

Транзакция.

Returns

[MdbxQuery](#)

Запрос на основе курсора.

Find(MdbxTransaction, Encoding?)

Возвращает отложенный перечислитель строковых значений таблицы (декодируются как UTF-8).

```
public IEnumerable<string> Find(MdbxTransaction txn, Encoding? encoding = null)
```

Parameters

txn [MdbxTransaction](#)

Транзакция.

encoding [Encoding](#) 

Кодировка для декодирования значений (по умолчанию UTF-8).

Returns

[IEnumerable](#)  [string](#)  >

Последовательность строковых значений.

FindValues(MdbxTransaction)

Возвращает отложенный перечислитель "сырых" значений таблицы (без ключей).

```
public IEnumerable<byte[]> FindValues(MdbxTransaction txn)
```

Parameters

txn [MdbxTransaction](#)

Транзакция.

Returns

[IEnumerable](#)  [byte](#)  [] >

Последовательность значений.

Find<TValue>(MdbxTransaction, Func<byte[], TValue>)

Возвращает отложенный перечислитель значений таблицы, преобразованных в **TValue**. Позволяет писать LINQ-запросы, где параметр предиката — это значение: `db.Find(txn, Encoding.UTF8.GetString).Where(x => x == "Some Value")`.

```
public IEnumerable<TValue> Find<TValue>(MdbxTransaction txn, Func<byte[],  
TValue> valueSelector)
```

Parameters

txn [MdbxTransaction](#)

Транзакция.

valueSelector [Func](#) [<byte](#) [\[\], TValue>](#)

Преобразование "сырых" байтов значения в TValue.

Returns

[IEnumerable](#) [<TValue>](#)

Последовательность преобразованных значений.

Type Parameters

TValue

Целевой тип значения.

Find<TKey, TValue>(MdbxTransaction, Func<byte[], TKey>, Func<byte[], TValue>)

Возвращает отложенный перечислитель пар ключ-значение, преобразованных в заданные типы.

```
public IEnumerable<KeyValuePair<TKey, TValue>> Find<TKey, TValue>(MdbxTransaction txn,  
Func<byte[], TKey> keySelector, Func<byte[], TValue> valueSelector)
```

Parameters

txn [MdbxTransaction](#)

Транзакция.

keySelector [Func](#) <[byte](#)[], TKey>

Преобразование "сырых" байтов ключа в TKey.

valueSelector [Func](#) <[byte](#)[], TValue>

Преобразование "сырых" байтов значения в TValue.

Returns

[IEnumerable](#) <[KeyValuePair](#) <TKey, TValue> >

Последовательность пар ключ-значение.

Type Parameters

TKey

Целевой тип ключа.

TValue

Целевой тип значения.

Get(MdbxTransaction, byte[])

Получает значение по ключу.

```
public byte[]? Get(MdbxTransaction txn, byte[] key)
```

Parameters

txn [MdbxTransaction](#)

Транзакция.

key [byte](#)[]

Ключ.

Returns

[byte](#)[]

Значение или null если ключ не найден.

Get(MdbxTransaction, string)

Получает значение по строковому ключу.

```
public byte[]? Get(MdbxTransaction txn, string key)
```

Parameters

txn [MdbxTransaction](#)

Транзакция.

key [string](#)

Строковый ключ (UTF-8).

Returns

[byte](#)[]

Значение или null если ключ не найден.

Put(MdbxTransaction, byte[], byte[], MdbxPutFlags)

Записывает значение по ключу.

```
public void Put(MdbxTransaction txn, byte[] key, byte[] value, MdbxPutFlags flags  
= MdbxPutFlags.MDBX_UPSERT)
```

Parameters

txn [MdbxTransaction](#)

Транзакция.

key [byte](#)[]

Ключ.

value [byte](#)[]

Значение.

flags [MdbxPutFlags](#)

Флаги записи.

Put(MdbxTransaction, string, byte[], MdbxPutFlags)

Записывает значение по строковому ключу.

```
public void Put(MdbxTransaction txn, string key, byte[] value, MdbxPutFlags flags  
= MdbxPutFlags.MDBX_UPSERT)
```

Parameters

txn [MdbxTransaction](#)

Транзакция.

key [string](#)

Строковый ключ (UTF-8).

value [byte](#)[]

Значение.

flags [MdbxPutFlags](#)

Флаги записи.

Put(MdbxTransaction, string, string, MdbxPutFlags)

Записывает строковое значение по строковому ключу.

```
public void Put(MdbxTransaction txn, string key, string value, MdbxPutFlags flags  
= MdbxPutFlags.MDBX_UPSERT)
```

Parameters

txn [MdbxTransaction](#)

Транзакция.

key [string](#) 

Строковый ключ (UTF-8).

value [string](#) 

Строковое значение (UTF-8).

flags [MdbxPutFlags](#)

Флаги записи.

Class MdbxEntry

Namespace: [MDBX](#)

Assembly: mdbx-dotnet.dll

Пара ключ-значение, прочитанная из таблицы через курсор. Ключ и значение представлены в виде "сырых" байтов и копируются из нативной памяти.

```
public sealed class MdbxEntry
```

Inheritance

[object](#)  ← MdbxEntry

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Constructors

MdbxEntry(byte[], byte[])

Создает запись из ключа и значения.

```
public MdbxEntry(byte[] key, byte[] value)
```

Parameters

key [byte](#)  []

Ключ.

value [byte](#)  []

Значение.

Properties

Key

Ключ записи.

```
public byte[] Key { get; }
```

Property Value

[byte](#)[]

Value

Значение записи.

```
public byte[] Value { get; }
```

Property Value

[byte](#)[]

Class MdbxEnvironment

Namespace: [MDBX](#)

Assembly: mdbx-dotnet.dll

Высокоуровневая обертка окружения libmdbx.

```
public sealed class MdbxEnvironment : IDisposable
```

Inheritance

[object](#)  ← MdbxEnvironment

Implements

[IDisposable](#) 

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Properties

DefaultPageSize

Получает размер страницы по умолчанию.

```
public static nuint DefaultPageSize { get; }
```

Property Value

[nuint](#) 

FileDescriptor

Получает дескриптор файла окружения.

```
public nint FileDescriptor { get; }
```

Property Value

[nint](#)

NativeEnv

Получает нативный указатель на окружение.

```
public MDBX_env* NativeEnv { get; }
```

Property Value

[MDBX_env*](#)

Path

Получает путь к окружению.

```
public string Path { get; }
```

Property Value

[string](#)

Methods

BeginTransaction(MdbxTxnFlags)

Начинает новую транзакцию.

```
public MdbxTransaction BeginTransaction(MdbxTxnFlags flags = MdbxTxnFlags.MDBX_TXN_DEFAULTS)
```

Parameters

flags [MdbxTxnFlags](#)

Флаги транзакции.

Returns

[MdbxTransaction](#)

Новая транзакция.

CheckReaders(out int)

Проверяет читателей и возвращает количество мертвых читателей.

```
public void CheckReaders(out int deadReaders)
```

Parameters

deadReaders [int](#)

Количество мертвых читателей.

Close()

Закрывает окружение MDBX.

```
public void Close()
```

Close(bool)

Закрывает окружение MDBX с опциональной отменой синхронизации.

```
public void Close(bool dontSync)
```

Parameters

dontSync [bool](#)

Не синхронизировать при закрытии.

CloseDatabase(uint)

Закрывает таблицу (DBI).

```
public void CloseDatabase(uint dbi)
```

Parameters

dbi [uint](#)

Идентификатор таблицы.

Copy(string, MdbxCopyFlags)

Копирует окружение MDBX в указанный путь.

```
public void Copy(string destinationPath, MdbxCopyFlags flags  
= MdbxCopyFlags.MDBX_CP_DEFAULT)
```

Parameters

destinationPath [string](#)

Путь назначения.

flags [MdbxCopyFlags](#)

Флаги копирования.

CopyToFd(nint, MdbxCopyFlags)

Копирует окружение MDBX в файловый дескриптор.

```
public void CopyToFd(nint fd, MdbxCopyFlags flags = MdbxCopyFlags.MDBX_CP_DEFAULT)
```

Parameters

fd [nint](#)

Файловый дескриптор.

flags [MdbxCopyFlags](#)

Флаги копирования.

Create()

Создает новое окружение MDBX.

```
public static MdbxEnvironment Create()
```

Returns

[MdbxEnvironment](#)

Новое окружение.

Defrag(nuint, nuint, nuint, nuint, nint, nint, MdbxDefragProgress Func?)

Фрагментирует окружение (defrag).

```
public void Defrag(nuint defragAtleast = 0, nuint timeAtleastDot16 = 0, nuint defragEnough = 0, nuint timeLimitDot16 = 0, nint acceptableBacklash = 0, nint preferredBatch = 0, MdbxDefragProgressFunc? progressCallback = null)
```

Parameters

defragAtleast [nuint](#)

Минимальное количество страниц для дефрагментации (0 = по умолчанию).

timeAtleastDot16 [nuint](#)

Минимальное время в 1/65536 сек (0 = по умолчанию).

defragEnough [nuint](#)

Достаточное количество страниц (0 = по умолчанию).

`timeLimitDot16` [nuint](#)

Лимит времени в 1/65536 сек (0 = без лимита).

`acceptableBacklash` [nint](#)

Допустимый откат.

`preferredBatch` [nint](#)

Предпочтительный размер пакета.

`progressCallback` [MdbxDefragProgressFunc](#)

Функция обратного вызова для уведомления о прогрессе.

Delete(string, MdbxEnvDeleteMode)

Удаляет окружение MDBX по пути.

```
public static void Delete(string path, MdbxEnvDeleteMode mode =  
MdbxEnvDeleteMode.MDBX_ENV_DELETE_ALL)
```

Parameters

`path` [string](#)

Путь к окружению.

`mode` [MdbxEnvDeleteMode](#)

Режим удаления.

Dispose()

Освобождает ресурсы окружения.

```
public void Dispose()
```

DropDatabase(uint, bool)

Удаляет таблицу (содержимое и/или саму таблицу).

```
public void DropDatabase(uint dbi, bool deleteData = true)
```

Parameters

dbi [uint](#)

Идентификатор таблицы.

deleteData [bool](#)

Удалить данные.

EnumerateReaders(Action<int, int, uint, uint, ulong, ulong, string?>)

Перечисляет читателей окружения.

```
public void EnumerateReaders(Action<int, int, uint, uint, ulong, ulong, string?> action)
```

Parameters

action [Action](#) <[int](#), [int](#), [uint](#), [uint](#), [ulong](#), [ulong](#), [string](#)>

Действие для каждого читателя.

EnumerateTables(Action<string, MdbxDbFlags, uint>)

Перечисляет все таблицы в окружении.

```
public void EnumerateTables(Action<string, MdbxDbFlags, uint> action)
```

Parameters

action [Action](#) <[string](#), [MdbxDbFlags](#), [uint](#)>

Действие для каждой таблицы (имя, флаги, ID).

~MdbxEnvironment()

Финализатор.

```
protected ~MdbxEnvironment()
```

GetErrorMessage(int)

Получает строковое описание кода ошибки.

```
public static string GetErrorMessage(int errorCode)
```

Parameters

errorCode [int](#)[↗]

Код ошибки.

Returns

[string](#)[↗]

GetFlags()

Получает флаги окружения.

```
public MdbxEnvFlags GetFlags()
```

Returns

[MdbxEnvFlags](#)

Флаги окружения.

GetHsr()

Получает функцию обработки медленных читателей (HSR).

```
public MdbxHsrFunc? GetHsr()
```

Returns

[MdbxHsrFunc](#)

Функция обратного вызова или null.

GetInfo()

Получает информацию об окружении.

```
public MDBX_envinfo GetInfo()
```

Returns

[MDBX_envinfo](#)

GetMaxKeySize(MdbxDBFlags)

Получает максимальный размер ключа.

```
public int GetMaxKeySize(MdbxDBFlags flags = MdbxDBFlags.MDBX_DB_DEFAULTS)
```

Parameters

flags [MdbxDBFlags](#)

Флаги таблицы.

Returns

[int](#)

GetMaxPairSizeForPage(MdbxDbFlags)

Получает максимальный размер пары ключ-значение для страницы.

```
public int GetMaxPairSizeForPage(MdbxDbFlags flags = MdbxDbFlags.MDBX_DB_DEFAULTS)
```

Parameters

flags [MdbxDbFlags](#)

Флаги таблицы.

Returns

[int](#)

GetMaxValueSize(MdbxDbFlags)

Получает максимальный размер значения.

```
public int GetMaxValueSize(MdbxDbFlags flags = MdbxDbFlags.MDBX_DB_DEFAULTS)
```

Parameters

flags [MdbxDbFlags](#)

Флаги таблицы.

Returns

[int](#)

GetMaxValueSizeForPage(MdbxDbFlags)

Получает максимальный размер значения для страницы.

```
public int GetMaxValueSizeForPage(MdbxDbFlags flags = MdbxDbFlags.MDBX_DB_DEFAULTS)
```

Parameters

flags [MdbxDbFlags](#)

Флаги таблицы.

Returns

[int](#)[↗]

GetOption(MdbxOption)

Получает параметр окружения.

```
public ulong GetOption(MdbxOption option)
```

Parameters

option [MdbxOption](#)

Параметр.

Returns

[ulong](#)[↗]

Значение параметра.

GetStat()

Получает статистику окружения.

```
public MdbxStat GetStat()
```

Returns

[MdbxStat](#)

GetSystemRamInfo(out nint, out nint, out nint)

Получает информацию о системной RAM.

```
public static void GetSystemRamInfo(out nint pageSize, out nint totalPages, out nint availPages)
```

Parameters

pageSize [nint](#)

Размер страницы.

totalPages [nint](#)

Общее количество страниц.

availPages [nint](#)

Доступное количество страниц.

GetUserContext()

Получает пользовательский контекст окружения.

```
public void* GetUserContext()
```

Returns

[void](#)*

Указатель на пользовательский контекст.

IsReadaheadReasonable(nuint, nint)

Проверяет, разумно ли использовать readahead для заданного объема.

```
public static bool IsReadaheadReasonable(nuint volume, nint redundancy)
```


Parameters

volume [nuint](#)

Объем данных.

redundancy [nint](#)

Избыточность.

Returns

[bool](#)

LimitsDatabaseSizeMax(nint)

Получает максимальный размер базы данных.

```
public static nint LimitsDatabaseSizeMax(nint pageSize)
```

Parameters

pageSize [nint](#)

Размер страницы.

Returns

[nint](#)

LimitsDatabaseSizeMin(nint)

Получает минимальный размер базы данных.

```
public static nint LimitsDatabaseSizeMin(nint pageSize)
```

Parameters

pageSize [nint](#)

Размер страницы.

Returns

[nint](#)

LimitsKeySizeMax(nint, MdbxDbFlags)

Получает максимальный размер ключа.

```
public static nint LimitsKeySizeMax(nint pageSize, MdbxDbFlags flags)
```

Parameters

pageSize [nint](#)

Размер страницы.

flags [MdbxDbFlags](#)

Флаги таблицы.

Returns

[nint](#)

LimitsKeySizeMin(MdbxDbFlags)

Получает минимальный размер ключа.

```
public static nint LimitsKeySizeMin(MdbxDbFlags flags)
```

Parameters

flags [MdbxDbFlags](#)

Флаги таблицы.

Returns

[nint](#)

LimitsPairSizeForPageMax(nint, MdbxDbFlags)

Получает максимальный размер пары ключ-значение для страницы.

```
public static nint LimitsPairSizeForPageMax(nint pageSize, MdbxDbFlags flags)
```

Parameters

pageSize [nint](#)

Размер страницы.

flags [MdbxDbFlags](#)

Флаги таблицы.

Returns

[nint](#)

LimitsTransactionSizeMax(nint)

Получает максимальный размер транзакции.

```
public static nint LimitsTransactionSizeMax(nint pageSize)
```

Parameters

pageSize [nint](#)

Размер страницы.

Returns

[nint](#)

LimitsValueSizeForPageMax(nint, MdbxDbFlags)

Получает максимальный размер значения для страницы.

```
public static nint LimitsValueSizeForPageMax(nint pageSize, MdbxDbFlags flags)
```

Parameters

pageSize [nint](#)

Размер страницы.

flags [MdbxDbFlags](#)

Флаги таблицы.

Returns

[nint](#)

LimitsValueSizeMax(nint, MdbxDbFlags)

Получает максимальный размер значения.

```
public static nint LimitsValueSizeMax(nint pageSize, MdbxDbFlags flags)
```

Parameters

pageSize [nint](#)

Размер страницы.

flags [MdbxDbFlags](#)

Флаги таблицы.

Returns

[nint](#)

LimitsValueSizeMin(MdbxDBFlags)

Получает минимальный размер значения.

```
public static nint LimitsValueSizeMin(MdbxDBFlags flags)
```

Parameters

flags [MdbxDBFlags](#)

Флаги таблицы.

Returns

[nint](#)

LockTransaction(bool)

Блокирует окружение для создания транзакции.

```
public void LockTransaction(bool dontWait = false)
```

Parameters

dontWait [bool](#)

Не ждать, если окружение заблокировано.

Open(string, MdbxEnvFlags, ushort, int)

Создает и открывает окружение MDBX.

```
public static MdbxEnvironment Open(string path, MdbxEnvFlags flags =  
MdbxEnvFlags.MDBX_ENV_DEFAULTS, ushort mode = 1792, int maxDatabases = 0)
```

Parameters

path [string](#)

Путь к базе данных.

flags [MdbxEnvFlags](#)

Флаги окружения.

mode [ushort](#)

Режим доступа (права доступа для создания файлов).

maxDatabases [int](#)

Максимальное количество таблиц (0 = по умолчанию).

Returns

[MdbxEnvironment](#)

Открытое окружение.

OpenDatabase(string?, MdbxDbFlags)

Открывает именованную таблицу (DBI).

```
public uint OpenDatabase(string? name = null, MdbxDbFlags flags = MdbxDbFlags.MDBX_CREATE)
```

Parameters

name [string](#)

Имя таблицы (null для дефолтной таблицы).

flags [MdbxDbFlags](#)

Флаги таблицы.

Returns

[uint](#)

Идентификатор таблицы (DBI).

OpenDatabaseEx(string?, MdbxDbFlags)

Открывает именованную таблицу (DBI) и возвращает высокоуровневую обертку.

```
public MdbxDatabase OpenDatabaseEx(string? name = null, MdbxDbFlags flags  
= MdbxDbFlags.MDBX_CREATE)
```

Parameters

name [string](#) 

Имя таблицы (null для дефолтной таблицы).

flags [MdbxDbFlags](#)

Флаги таблицы.

Returns

[MdbxDatabase](#)

Обертка таблицы (DBI).

OpenForRecovery(string, uint, bool)

Открывает окружение для восстановления.

```
public void OpenForRecovery(string path, uint targetMeta, bool writable)
```

Parameters

path [string](#) 

Путь к базе данных.

targetMeta [uint](#) 

Целевая мета-страница.

writable [bool](#) 

Доступно для записи.

OpenInternal(string, MdbxEnvFlags, ushort)

Открывает окружение MDBX.

```
public void OpenInternal(string path, MdbxEnvFlags flags = MdbxEnvFlags.MDBX_ENV_DEFAULTS,
ushort mode = 1792)
```

Parameters

path [string](#)

Путь к базе данных.

flags [MdbxEnvFlags](#)

Флаги окружения.

mode [ushort](#)

Режим доступа (права доступа для создания файлов).

RegisterThread()

Регистрирует текущий поток для работы с окружением.

```
public void RegisterThread()
```

RenameDatabase(uint, string)

Переименовывает таблицу.

```
public void RenameDatabase(uint dbi, string newName)
```

Parameters

dbi [uint](#)

Идентификатор таблицы.

newName [string](#)

Новое имя таблицы.

ResurrectAfterFork()

Восстанавливает окружение после fork.

```
public void ResurrectAfterFork()
```

SetFlags(MdbxEnvFlags, bool)

Устанавливает флаги окружения.

```
public void SetFlags(MdbxEnvFlags flags, bool on)
```

Parameters

flags [MdbxEnvFlags](#)

Флаги.

on [bool](#)

Включить или выключить.

SetGeometry(long, long, long, long, long, long)

Устанавливает геометрию окружения (размеры файлов).

```
public void SetGeometry(long sizeLower = -1, long sizeNow = -1, long sizeUpper = -1, long  
growthStep = -1, long shrinkThreshold = -1, long pageSize = 0)
```

Parameters

sizeLower [long](#)

Нижний предел размера (-1 = оставить без изменений).

`sizeNow` [long](#)

Текущий размер (-1 = оставить без изменений).

`sizeUpper` [long](#)

Верхний предел размера (-1 = оставить без изменений).

`growthStep` [long](#)

Шаг роста (-1 = по умолчанию).

`shrinkThreshold` [long](#)

Порог сжатия (-1 = по умолчанию).

`pageSize` [long](#)

Размер страницы (0 = по умолчанию).

SetHsr(MdbxHsrFunc?)

Устанавливает функцию обработки медленных читателей (HSR).

```
public void SetHsr(MdbxHsrFunc? callback)
```

Parameters

`callback` [MdbxHsrFunc](#)

Функция обратного вызова (null для сброса).

SetOption(MdbxOption, ulong)

Устанавливает параметр окружения.

```
public void SetOption(MdbxOption option, ulong value)
```

Parameters

`option` [MdbxOption](#)

Параметр.

value [ulong](#)

Значение.

SetUserContext(void*)

Устанавливает пользовательский контекст окружения.

```
public void SetUserContext(void* context)
```

Parameters

context [void](#)*

Пользовательский контекст.

SetupDebug(MdbxLogLevel, MdbxDebugFlags, MdbxDebugFunc?)

Настраивает отладочное логирование libmdbx.

```
public static void SetupDebug(MdbxLogLevel logLevel, MdbxDebugFlags debugFlags,  
MdbxDebugFunc? logger)
```

Parameters

logLevel [MdbxLogLevel](#)

Уровень логирования.

debugFlags [MdbxDebugFlags](#)

Флаги отладки.

logger [MdbxDebugFunc](#)

Функция логирования (null для отключения).

SetupDebugNoFormat(MdbxLogLevel, MdbxDebugFlags, MdbxDebugFuncNofmt?, byte*, nuint)

Настраивает отладочное логирование без форматирования.

```
public static void SetupDebugNoFormat(MdbxLogLevel logLevel, MdbxDebugFlags debugFlags,  
MdbxDebugFuncNofmt? logger, byte* loggerBuffer = null, nuint loggerBufferSize = 0)
```

Parameters

logLevel [MdbxLogLevel](#)

Уровень логирования.

debugFlags [MdbxDebugFlags](#)

Флаги отладки.

logger [MdbxDebugFuncNofmt](#)

Функция логирования без форматирования (null для отключения).

loggerBuffer [byte](#)[↗]*

Буфер для форматирования сообщений.

loggerBufferSize [nuint](#)[↗]

Размер буфера.

Sync(bool, bool)

Синхронизирует окружение на диск.

```
public void Sync(bool force = false, bool nonblock = false)
```

Parameters

force [bool](#)[↗]

Принудительная синхронизация.

`nonblock` [bool](#)

Неблокирующая синхронизация.

TurnForRecovery(uint)

Переключает окружение для восстановления на указанную мета-страницу.

```
public void TurnForRecovery(uint targetMeta)
```

Parameters

`targetMeta` [uint](#)

Целевая мета-страница.

UnlockTransaction()

Разблокирует окружение.

```
public void UnlockTransaction()
```

UnregisterThread()

Отменяет регистрацию текущего потока.

```
public void UnregisterThread()
```

Warmup(MDBX_txn*, MdbxWarmupFlags, uint)

Прогревает окружение (предзагружает страницы в память).

```
public void Warmup(MDBX_txn* txn = null, MdbxWarmupFlags flags =  
MdbxWarmupFlags.MDBX_WARMUP_DEFAULT, uint timeoutSeconds16Dot16 = 0)
```

Parameters

`txn` [MDBX_txn](#)*

Транзакция (может быть null).

`flags` [MdbxWarmupFlags](#)

Флаги прогрева.

`timeoutSeconds16Dot16` [uint](#)

Таймаут в 1/65536 сек (0 = без ограничения).

Class MdbxErrorCodes

Namespace: [MDBX](#)

Assembly: mdbx-dotnet.dll

Коды ошибок libmdbx.

```
public static class MdbxErrorCodes
```

Inheritance

[object](#)  ← MdbxErrorCodes

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Fields

MDBX_KEYEXIST

Элемент уже существует.

```
public const int MDBX_KEYEXIST = -30799
```

Field Value

[int](#) 

MDBX_NOTFOUND

Элемент не найден.

```
public const int MDBX_NOTFOUND = -30798
```

Field Value

Class MdbxException

Namespace: [MDBX](#)

Assembly: mdbx-dotnet.dll

Исключение, возникающее при ошибках libmdbx.

```
public sealed class MdbxException : Exception, ISerializable
```

Inheritance

[object](#)  ← [Exception](#)  ← MdbxException

Implements

[ISerializable](#) 

Inherited Members

[Exception.GetBaseException\(\)](#)  , [Exception.GetType\(\)](#)  , [Exception.ToString\(\)](#)  , [Exception.Data](#)  , [Exception.HelpLink](#)  , [Exception.HResult](#)  , [Exception.InnerException](#)  , [Exception.Message](#)  , [Exception.Source](#)  , [Exception.StackTrace](#)  , [Exception.TargetSite](#)  , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.ReferenceEquals\(object, object\)](#) 

Constructors

MdbxException(int)

Инициализирует новое исключение с кодом ошибки.

```
public MdbxException(int errorCode)
```

Parameters

errorCode [int](#) 

Код ошибки libmdbx.

MdbxException(int, string)

Инициализирует новое исключение с кодом ошибки и сообщением.

```
public MdbxException(int errorCode, string message)
```

Parameters

errorCode [int](#)

Код ошибки libmdbx.

message [string](#)

Сообщение об ошибке.

Properties

ErrorCode

Код ошибки libmdbx.

```
public int ErrorCode { get; }
```

Property Value

[int](#)

Class MdbxQuery

Namespace: [MDBX](#)

Assembly: mdbx-dotnet.dll

Высокоуровневая отложенная (lazy) абстракция над курсором MDBX. Представляет собой [IEnumerable<T>](#), перебирающий все записи таблицы поверх нативного курсора. Каждый вызов [GetEnumerator\(\)](#) открывает новый курсор, а перебор выполняется лениво, что делает тип полностью совместимым с LINQ (Where, Select, First, ToList и т.д.).

```
public sealed class MdbxQuery : IEnumerable<MdbxEntry>, IEnumerable
```

Inheritance

[object](#) ← MdbxQuery

Implements

[IEnumerable](#) <[MdbxEntry](#)>, [IEnumerable](#)

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Extension Methods

[MdbxQueryExtensions.Entries<TKey, TValue>\(MdbxQuery, Func<byte\[\], TKey>, Func<byte\[\], TValue>\)](#),
[MdbxQueryExtensions.Keys\(MdbxQuery\)](#),
[MdbxQueryExtensions.Keys<TKey>\(MdbxQuery, Func<byte\[\], TKey>\)](#),
[MdbxQueryExtensions.Values\(MdbxQuery\)](#),
[MdbxQueryExtensions.Values<TValue>\(MdbxQuery, Func<byte\[\], TValue>\)](#).

Methods

GetEnumerator()

Returns an enumerator that iterates through the collection.

```
public IEnumerator<MdbxEntry> GetEnumerator()
```

Returns

[IEnumerator](#) <[MdbxEntry](#)>

An enumerator that can be used to iterate through the collection.

Class MdbxQueryExtensions

Namespace: [MDBX](#)

Assembly: mdbx-dotnet.dll

Методы расширения для [MdbxQuery](#), упрощающие проекцию записей и построение LINQ-запросов, где параметр предиката — это значение или ключ.

```
public static class MdbxQueryExtensions
```

Inheritance

[object](#) ← MdbxQueryExtensions

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Methods

Entries<TKey, TValue>(MdbxQuery, Func<byte[], TKey>, Func<byte[], TValue>)

Проецирует записи в пары ключ-значение заданных типов.

```
public static IEnumerable<KeyValuePair<TKey, TValue>> Entries<TKey, TValue>(this MdbxQuery query, Func<byte[], TKey> keySelector, Func<byte[], TValue> valueSelector)
```

Parameters

query [MdbxQuery](#)

Запрос на основе курсора.

keySelector [Func](#)<[byte](#)[], TKey>

Преобразование "сырых" байтов ключа в TKey.

valueSelector [Func](#)<[byte](#)[], TValue>

Преобразование "сырых" байтов значения в TValue.

Returns

[IEnumerable](#) [<KeyValuePair](#) [<TKey, TValue>](#) >

Последовательность пар ключ-значение.

Type Parameters

TKey

Целевой тип ключа.

TValue

Целевой тип значения.

Keys(MdbxQuery)

Проецирует записи в "сырые" ключи.

```
public static IEnumerable<byte[]> Keys(this MdbxQuery query)
```

Parameters

query [MdbxQuery](#)

Запрос на основе курсора.

Returns

[IEnumerable](#) [<byte](#) [\[\]](#) >

Последовательность ключей.

Keys<TKey>(MdbxQuery, Func<byte[], TKey>)

Проецирует ключи записей в заданный тип.

```
public static IEnumerable<TKey> Keys<TKey>(this MdbxQuery query, Func<byte[],  
TKey> selector)
```

Parameters

query [MdbxQuery](#).

Запрос на основе курсора.

selector [Func](#) <[byte](#) [], TKey>

Преобразование "сырых" байтов ключа в TKey.

Returns

[IEnumerable](#) <TKey>

Последовательность преобразованных ключей.

Type Parameters

TKey

Целевой тип ключа.

Values(MdbxQuery)

Проецирует записи в "сырые" значения (без ключей).

```
public static IEnumerable<byte[]> Values(this MdbxQuery query)
```

Parameters

query [MdbxQuery](#).

Запрос на основе курсора.

Returns

[IEnumerable](#) <[byte](#) []>

Последовательность значений.

Values<TValue>(MdbxQuery, Func<byte[], TValue>)

Проецирует значения записей в заданный тип. Позволяет писать запросы вида:

`query.Values(Encoding.UTF8.GetString).Where(x => x == "Some Value")`, где `x` — это уже преобразованное значение.

```
public static IEnumerable<TValue> Values<TValue>(this MdbxQuery query, Func<byte[],  
TValue> selector)
```

Parameters

`query` [MdbxQuery](#)

Запрос на основе курсора.

`selector` [Func](#) [byte](#)[], TValue>

Преобразование "сырых" байтов значения в TValue.

Returns

[IEnumerable](#) [byte](#)[], TValue>

Последовательность преобразованных значений.

Type Parameters

TValue

Целевой тип значения.

Class MdbxTransaction

Namespace: [MDBX](#)

Assembly: mdbx-dotnet.dll

Высокоуровневая обертка транзакции MDBX.

```
public sealed class MdbxTransaction : IDisposable
```

Inheritance

[object](#)  ← MdbxTransaction

Implements

[IDisposable](#) 

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Properties

NativeTxn

Получает нативный указатель на транзакцию.

```
public MDBX_txn* NativeTxn { get; }
```

Property Value

[MDBX_txn](#) 

Methods

Commit()

Фиксирует транзакцию.

```
public void Commit()
```

Dispose()

Освобождает ресурсы транзакции.

```
public void Dispose()
```

~MdbxTransaction()

Финализатор.

```
protected ~MdbxTransaction()
```

OpenCursor(MdbxDatabase)

Открывает курсор для таблицы в рамках этой транзакции.

```
public MdbxCursor OpenCursor(MdbxDatabase db)
```

Parameters

db [MdbxDatabase](#)

Таблица (DBI).

Returns

[MdbxCursor](#)

Новый курсор.

Rollback()

Откатывает транзакцию.

```
public void Rollback()
```

Namespace MDBX.Native

Classes

[NativeLibraryLoader](#)

Загрузчик нативных библиотек libmdbx.

Class NativeLibraryLoader

Namespace: [MDBX.Native](#)

Assembly: mdbx-dotnet.dll

Загрузчик нативных библиотек libmdbx.

```
public static class NativeLibraryLoader
```

Inheritance

[object](#)  ← NativeLibraryLoader

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Remarks

Обеспечивает кроссплатформенную загрузку библиотеки mdbx.dll (Windows) или libmdbx.so (Linux).

Fields

LibraryName

Имя нативной библиотеки для DllImport.

```
public const string LibraryName = "mdbx"
```

Field Value

[string](#) 

Methods

Initialize()

Инициализирует загрузчик нативных библиотек.

```
public static void Initialize()
```

Remarks

Устанавливает резолвер для DllImport, который будет искать библиотеку в стандартных местах для текущей платформы. Безопасен для многократного вызова — второй и последующие вызовы игнорируются.

Namespace MDBX.Native.Bindings.Crud

Classes

[NativeMdbx](#)

Частичный класс для нативных методов CRUD операций libmdbx.

Class NativeMdbx

Namespace: [MDBX.Native.Bindings.Crud](#)

Assembly: mdbx-dotnet.dll

Частичный класс для нативных методов CRUD операций libmdbx.

```
public static class NativeMdbx
```

Inheritance

[object](#)  ← NativeMdbx

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Methods

MdbxDel(MDBX_txn*, uint, MdbxVal*, MdbxVal*)

Удаляет элемент.

```
public static extern int MdbxDel(MDBX_txn* txn, uint dbi, MdbxVal* key, MdbxVal* data)
```

Parameters

txn [MDBX_txn](#)*

Транзакция.

dbi [uint](#) 

DBI.

key [MdbxVal](#)*

Ключ.

data [MdbxVal](#)*

Значение (опционально).

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxGet(MDBX_txn*, uint, MdbxVal*, MdbxVal*)

Получает значение по ключу.

```
public static extern int MdbxGet(MDBX_txn* txn, uint dbi, MdbxVal* key, MdbxVal* data)
```

Parameters

txn [MDBX_txn*](#)

Транзакция.

dbi [uint](#)

DBI.

key [MdbxVal*](#)

Ключ.

data [MdbxVal*](#)

Значение.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxGetEqualOrGreat(MDBX_txn*, uint, MdbxVal*, MdbxVal*)

Получает значение по ключу или ближайшему большему ключу.

```
public static extern int MdbxGetEqualOrGreat(MDBX_txn* txn, uint dbi, MdbxVal* key,
MdbxVal* data)
```

Parameters

txn [MDBX_txn](#)*

Транзакция.

dbi [uint](#)

DBI.

key [MdbxVal](#)*

Ключ (на вход: искомый ключ, на выходе: найденный ключ).

data [MdbxVal](#)*

Значение.

Returns

[int](#)

Код ошибки (0 при успехе, MDBX_NOTFOUND если не найдено).

MdbxGetEx(MDBX_txn*, uint, MdbxVal*, MdbxVal*, nuint*)

Получает значение по ключу с расширенными опциями.

```
public static extern int MdbxGetEx(MDBX_txn* txn, uint dbi, MdbxVal* key, MdbxVal* data,
nuint* valuesCount)
```

Parameters

txn [MDBX_txn](#)*

Транзакция.

dbi [uint](#)

DBI.

key [MdbxVal](#)*

Ключ.

data [MdbxVal](#)*

Значение.

valuesCount [nuint](#)^{*}

Количество значений.

Returns

[int](#)^{*}

Код ошибки (0 при успехе).

MdbxPut(MDBX_txn*, uint, MdbxVal*, MdbxVal*, MdbxPutFlags)

Записывает значение.

```
public static extern int MdbxPut(MDBX_txn* txn, uint dbi, MdbxVal* key, MdbxVal* data,
MdbxPutFlags flags)
```

Parameters

txn [MDBX_txn](#)*

Транзакция.

dbi [uint](#)^{*}

DBI.

key [MdbxVal](#)*

Ключ.

data [MdbxVal](#)*

Значение.

flags [MdbxPutFlags](#)

Флаги.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxReplace(MDBX_txn*, uint, MdbxVal*, MdbxVal*, MdbxVal*, MdbxPutFlags)

Заменяет значение.

```
public static extern int MdbxReplace(MDBX_txn* txn, uint dbi, MdbxVal* key, MdbxVal*
newData, MdbxVal* oldData, MdbxPutFlags flags)
```

Parameters

txn [MDBX_txn](#)*

Транзакция.

dbi [uint](#)

DBI.

key [MdbxVal](#)*

Ключ.

newData [MdbxVal](#)*

Новое значение.

oldData [MdbxVal](#)*

Старое значение.

flags [MdbxPutFlags](#)

Флаги.

Returns

[int](#)

Код ошибки (0 при успехе).

`MdbxReplaceEx(MDBX_txn*, uint, MdbxVal*, MdbxVal*, MdbxVal*, MdbxPutFlags, MdbxPreserveFunc?, void*)`

Заменяет значение с расширенными опциями.

```
public static extern int MdbxReplaceEx(MDBX_txn* txn, uint dbi, MdbxVal* key,
MdbxVal* newData, MdbxVal* oldData, MdbxPutFlags flags, MdbxPreserveFunc? preserver,
void* preserver_context)
```

Parameters

`txn` [MDBX_txn*](#)

Транзакция.

`dbi` [uint](#)

DBI.

`key` [MdbxVal*](#)

Ключ.

`newData` [MdbxVal*](#)

Новое значение.

`oldData` [MdbxVal*](#)

Старое значение.

`flags` [MdbxPutFlags](#)

Флаги.

`preserver` [MdbxPreserveFunc](#)

Функция сохранения данных.

`preserver_context` [void](#)^{*}

Контекст для функции сохранения.

Returns

[int](#)

Код ошибки (0 при успехе).

Namespace MDBX.Native.Bindings.Cursor

Classes

[NativeMdbx](#)

Частичный класс для нативных методов курсоров libmdbx.

Class NativeMdbx

Namespace: [MDBX.Native.Bindings.Cursor](#)

Assembly: mdbx-dotnet.dll

Частичный класс для нативных методов курсоров libmdbx.

```
public static class NativeMdbx
```

Inheritance

[object](#)  ← NativeMdbx

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Methods

MdbxCursorBind(MDBX_txn*, MDBX_cursor*, uint)

Привязывает курсор к транзакции и таблице.

```
public static extern int MdbxCursorBind(MDBX_txn* txn, MDBX_cursor* cursor, uint dbi)
```

Parameters

txn [MDBX_txn](#)*

Транзакция.

cursor [MDBX_cursor](#)*

Курсор.

dbi [uint](#) 

DBI.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxCursorBunchDelete(MDBX_cursor*, MdbxBunchAction, ulong*)

Пакетное удаление через курсор.

```
public static extern int MdbxCursorBunchDelete(MDBX_cursor* cursor, MdbxBunchAction action,
ulong* numberOfAffected)
```

Parameters

cursor [MDBX_cursor*](#)

Курсор.

action [MdbxBunchAction](#)

Действие.

numberOfAffected [ulong](#)*

Количество затронутых элементов.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxCursorClose(MDBX_cursor*)

Закрывает курсор.

```
public static extern void MdbxCursorClose(MDBX_cursor* cursor)
```

Parameters

cursor [MDBX_cursor*](#)

Курсор.

MdbxCursorClose2(MDBX_cursor*)

Закрывает курсор с возвратом кода ошибки.

```
public static extern int MdbxCursorClose2(MDBX_cursor* cursor)
```

Parameters

cursor [MDBX_cursor*](#)

Курсор.

Returns

[int](#)[↗]

Код ошибки (0 при успехе).

MdbxCursorCompare(MDBX_cursor*, MDBX_cursor*, bool)

Сравнивает два курсора.

```
public static extern int MdbxCursorCompare(MDBX_cursor* left, MDBX_cursor* right,  
bool ignoreMultival)
```

Parameters

left [MDBX_cursor*](#)

Левый курсор.

right [MDBX_cursor*](#)

Правый курсор.

ignoreMultival [bool](#)[↗]

Игнорировать множественные значения.

Returns

[int](#)

Результат сравнения.

MdbxCursorCopy(MDBX_cursor*, MDBX_cursor*)

Копирует курсор.

```
public static extern int MdbxCursorCopy(MDBX_cursor* src, MDBX_cursor* dest)
```

Parameters

src [MDBX_cursor*](#)

Исходный курсор.

dest [MDBX_cursor*](#)

Курсор назначения.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxCursorCount(MDBX_cursor*, nuint*)

Получает количество элементов.

```
public static extern int MdbxCursorCount(MDBX_cursor* cursor, nuint* count)
```

Parameters

cursor [MDBX_cursor*](#)

Курсор.

count [nuint](#)^{*}

Количество.

Returns

[int](#)^{*}

Код ошибки (0 при успехе).

MdbxCursorCountEx(MDBX_cursor*, nuint*, MdbxStat*, nuint)

Получает количество элементов с статистикой.

```
public static extern int MdbxCursorCountEx(MDBX_cursor* cursor, nuint* count, MdbxStat*
stat, nuint bytes)
```

Parameters

cursor [MDBX_cursor](#)^{*}

Курсор.

count [nuint](#)^{*}

Количество.

stat [MdbxStat](#)^{*}

Статистика.

bytes [nuint](#)^{*}

Размер структуры.

Returns

[int](#)^{*}

Код ошибки (0 при успехе).

MdbxCursorCreate(void*)

Создает курсор.

```
public static extern MDBX_cursor* MdbxCursorCreate(void* context)
```

Parameters

context [void](#)*

Пользовательский контекст.

Returns

[MDBX_cursor](#)*

Указатель на курсор.

MdbxCursorDbi(MDBX_cursor*)

Получает DBI курсора.

```
public static extern uint MdbxCursorDbi(MDBX_cursor* cursor)
```

Parameters

cursor [MDBX_cursor](#)*

Курсор.

Returns

[uint](#)

DBI.

MdbxCursorDel(MDBX_cursor*, MdbxPutFlags)

Удаляет элемент через курсор.

```
public static extern int MdbxCursorDel(MDBX_cursor* cursor, MdbxPutFlags flags)
```

Parameters

cursor [MDBX_cursor*](#)

Курсор.

flags [MdbxPutFlags](#)

Флаги.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxCursorDeleteRange(MDBX_cursor*, MDBX_cursor*, bool, bool)

Удаляет диапазон через курсор.

```
public static extern int MdbxCursorDeleteRange(MDBX_cursor* begin, MDBX_cursor* end, bool  
endIncluding, bool force)
```

Parameters

begin [MDBX_cursor*](#)

Начальный курсор.

end [MDBX_cursor*](#)

Конечный курсор.

endIncluding [bool](#)

Включая конечный.

force [bool](#)

Принудительно.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxCursorDistance(MDBX_cursor*, MDBX_cursor*, nint*, MdbxCursorOp)

Получает расстояние между курсорами.

```
public static extern int MdbxCursorDistance(MDBX_cursor* first, MDBX_cursor* last, nint* distance, MdbxCursorOp op)
```

Parameters

first [MDBX_cursor*](#)

Первый курсор.

last [MDBX_cursor*](#)

Последний курсор.

distance [nint](#)*

Расстояние.

op [MdbxCursorOp](#)

Операция.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxCursorDistribute(MDBX_cursor*, MDBX_cursor*, MDBX_cursor**, uint, uint)

Распределяет элементы между курсорами.

```
public static extern int MdbxCursorDistribute(MDBX_cursor* first, MDBX_cursor* last,
MDBX_cursor** array, uint count, uint deepness)
```

Parameters

first [MDBX_cursor*](#)

Первый курсор.

last [MDBX_cursor*](#)

Последний курсор.

array [MDBX_cursor**](#)

Массив курсоров.

count [uint](#)

Количество.

deepness [uint](#)

Глубина.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxCursorEof(MDBX_cursor*)

Проверяет, достигнут ли конец таблицы.

```
public static extern int MdbxCursorEof(MDBX_cursor* cursor)
```


Parameters

cursor [MDBX_cursor](#)*

Курсор.

Returns

[int](#)

1 если конец, 0 иначе.

MdbxCursorGet(MDBX_cursor*, MdbxVal*, MdbxVal*, MdbxCursorOp)

Получает элемент через курсор.

```
public static extern int MdbxCursorGet(MDBX_cursor* cursor, MdbxVal* key, MdbxVal* data,
MdbxCursorOp op)
```

Parameters

cursor [MDBX_cursor](#)*

Курсор.

key [MdbxVal](#)*

Ключ.

data [MdbxVal](#)*

Значение.

op [MdbxCursorOp](#)

Операция.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxCursorGetBatch(MDBX_cursor*, nuint*, MdbxVal*, nuint, MdbxCursorOp)

Получает пакет элементов через курсор.

```
public static extern int MdbxCursorGetBatch(MDBX_cursor* cursor, nuint* count, MdbxVal* pairs, nuint limit, MdbxCursorOp op)
```

Parameters

cursor [MDBX_cursor](#)*

Курсор.

count [nuint](#)*

Количество элементов.

pairs [MdbxVal](#)*

Массив пар ключ-значение.

limit [nuint](#)

Лимит.

op [MdbxCursorOp](#)

Операция.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxCursorGetUserctx(MDBX_cursor*)

Получает пользовательский контекст курсора.

```
public static extern void* MdbxCursorGetUserctx(MDBX_cursor* cursor)
```

Parameters

cursor [MDBX_cursor](#)*

Курсор.

Returns

[void](#)^{*}

Указатель на контекст.

MdbxCursorIgnord(MDBX_cursor*)

Игнорирует дубликаты.

```
public static extern int MdbxCursorIgnord(MDBX_cursor* cursor)
```

Parameters

cursor [MDBX_cursor](#)*

Курсор.

Returns

[int](#)^{*}

Код ошибки (0 при успехе).

MdbxCursorOnFirst(MDBX_cursor*)

Проверяет, находится ли курсор на первом элементе.

```
public static extern int MdbxCursorOnFirst(MDBX_cursor* cursor)
```

Parameters

cursor [MDBX_cursor](#)*

Курсор.

Returns

[int](#)

1 если на первом, 0 иначе.

MdbxCursorOnFirstDup(MDBX_cursor*)

Проверяет, находится ли курсор на первом дубликате.

```
public static extern int MdbxCursorOnFirstDup(MDBX_cursor* cursor)
```

Parameters

cursor [MDBX_cursor](#)*

Курсор.

Returns

[int](#)

1 если на первом дубликате, 0 иначе.

MdbxCursorOnLast(MDBX_cursor*)

Проверяет, находится ли курсор на последнем элементе.

```
public static extern int MdbxCursorOnLast(MDBX_cursor* cursor)
```

Parameters

cursor [MDBX_cursor](#)*

Курсор.

Returns

[int](#)

1 если на последнем, 0 иначе.

MdbxCursorOnLastDup(MDBX_cursor*)

Проверяет, находится ли курсор на последнем дубликate.

```
public static extern int MdbxCursorOnLastDup(MDBX_cursor* cursor)
```

Parameters

cursor [MDBX_cursor*](#)

Курсор.

Returns

[int](#)

1 если на последнем дубликate, 0 иначе.

MdbxCursorOpen(MDBX_txn*, uint, MDBX_cursor**)

Открывает курсор для таблицы.

```
public static extern int MdbxCursorOpen(MDBX_txn* txn, uint dbi, MDBX_cursor** cursor)
```

Parameters

txn [MDBX_txn*](#)

Транзакция.

dbi [uint](#)

DBI.

cursor [MDBX_cursor](#)**

Указатель для курсора.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxCursorPut(MDBX_cursor*, MdbxVal*, MdbxVal*, MdbxPutFlags)

Записывает элемент через курсор.

```
public static extern int MdbxCursorPut(MDBX_cursor* cursor, MdbxVal* key, MdbxVal* data,
MdbxPutFlags flags)
```

Parameters

cursor [MDBX_cursor](#)*

Курсор.

key [MdbxVal](#)*

Ключ.

data [MdbxVal](#)*

Значение.

flags [MdbxPutFlags](#)

Флаги.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxCursorRenew(MDBX_txn*, MDBX_cursor*)

Обновляет курсор для новой транзакции.

```
public static extern int MdbxCursorRenew(MDBX_txn* txn, MDBX_cursor* cursor)
```

Parameters

txn [MDBX_txn*](#)

Транзакция.

cursor [MDBX_cursor*](#)

Курсор.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxCursorReset(MDBX_cursor*)

Сбрасывает курсор.

```
public static extern int MdbxCursorReset(MDBX_cursor* cursor)
```

Parameters

cursor [MDBX_cursor*](#)

Курсор.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxCursorScan(MDBX_cursor*, delegate* unmanaged[Cdecl]
<void*, MdbxVal*, MdbxVal*, void*, int>, void*, MdbxCursorOp,
MdbxVal*)

Сканирует элементы через курсор.

```
public static extern int MdbxCursorScan(MDBX_cursor* cursor, delegate* unmanaged[Cdecl]  
<void*, MdbxVal*, MdbxVal*, void*, int> predicate, void* context, MdbxCursorOp op,  
MdbxVal* arg)
```

Parameters

cursor [MDBX_cursor*](#)

Курсор.

predicate delegate* unmanaged[Cdecl] <[void](#)*, [MdbxVal](#)*, [MdbxVal](#)*, [void](#)*, [int](#)>

Функция предиката.

context [void](#)*

Контекст.

op [MdbxCursorOp](#)

Операция.

arg [MdbxVal](#)*

Аргумент.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxCursorScanFrom(MDBX_cursor*, delegate*
unmanaged[Cdecl] <void*, MdbxVal*, MdbxVal*, void*, int>,
void*, MdbxCursorOp, MdbxVal*, MdbxVal*)

Сканирует элементы через курсор начиная с позиции.

```
public static extern int MdbxCursorScanFrom(MDBX_cursor* cursor, delegate* unmanaged[Cdecl]  
<void*, MdbxVal*, MdbxVal*, void*, int> predicate, void* context, MdbxCursorOp op, MdbxVal*  
arg, MdbxVal* startKey)
```

Parameters

cursor [MDBX_cursor](#)*

Курсор.

predicate delegate* unmanaged[Cdecl]<[void](#)*, [MdbxVal](#)*, [MdbxVal](#)*, [void](#)*, [int](#)>

Функция предиката.

context [void](#)*

Контекст.

op [MdbxCursorOp](#)

Операция.

arg [MdbxVal](#)*

Аргумент.

startKey [MdbxVal](#)*

Начальный ключ.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxCursorScroll(MDBX_cursor*, nint, uint)

Прокручивает курсор.

```
public static extern int MdbxCursorScroll(MDBX_cursor* cursor, nint amount, uint deepness)
```

Parameters

cursor [MDBX_cursor](#)*

Курсор.

amount [nint](#)

Количество шагов.

deepness [uint](#)

Глубина.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxCursorSetUserctx(MDBX_cursor*, void*)

Устанавливает пользовательский контекст курсора.

```
public static extern int MdbxCursorSetUserctx(MDBX_cursor* cursor, void* ctx)
```

Parameters

cursor [MDBX_cursor](#)*

Курсор.

ctx [void](#)*

Контекст.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxCursorTxn(MDBX_cursor*)

Получает транзакцию курсора.

```
public static extern MDBX_txn* MdbxCursorTxn(MDBX_cursor* cursor)
```

Parameters

cursor [MDBX_cursor*](#)

Курсор.

Returns

[MDBX_txn*](#)

Указатель на транзакцию.

MdbxCursorUnbind(MDBX_cursor*)

Отвязывает курсор от таблицы.

```
public static extern int MdbxCursorUnbind(MDBX_cursor* cursor)
```

Parameters

cursor [MDBX_cursor*](#)

Курсор.

Returns

[int](#)[↗]

Код ошибки (0 при успехе).

Namespace MDBX.Native.Bindings.Dbi

Classes

[NativeMdbx](#)

Частичный класс для нативных методов таблиц (DBI) libmdbx.

Class NativeMdbx

Namespace: [MDBX.Native.Bindings.Dbi](#)

Assembly: mdbx-dotnet.dll

Частичный класс для нативных методов таблиц (DBI) libmdbx.

```
public static class NativeMdbx
```

Inheritance

[object](#)  ← NativeMdbx

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Methods

MdbxDbiClose(MDBX_env*, uint)

Закрывает таблицу.

```
public static extern int MdbxDbiClose(MDBX_env* env, uint dbi)
```

Parameters

env [MDBX_env*](#)

Окружение.

dbi [uint](#) 

DBI.

Returns

[int](#) 

Код ошибки (0 при успехе).

MdbxDbiDupsortDepthmask(MDBX_txn*, uint, uint*)

Получает маску глубины дубликатов.

```
public static extern int MdbxDbiDupsortDepthmask(MDBX_txn* txn, uint dbi, uint* mask)
```

Parameters

txn [MDBX_txn*](#)

Транзакция.

dbi [uint](#)

DBI.

mask [uint](#)*

Указатель для маски.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxDbiFlagsEx(MDBX_txn*, uint, uint*, uint*)

Получает флаги и состояние таблицы.

```
public static extern int MdbxDbiFlagsEx(MDBX_txn* txn, uint dbi, uint* flags, uint* state)
```

Parameters

txn [MDBX_txn*](#)

Транзакция.

dbi [uint](#)

DBI.

flags [uint](#)*

Указатель для флагов.

state [uint](#)*

Указатель для состояния.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxDbiOpen(MDBX_txn*, byte*, MdbxDbFlags, uint*)

Открывает таблицу по имени.

```
public static extern int MdbxDbiOpen(MDBX_txn* txn, byte* name, MdbxDbFlags flags,
uint* dbi)
```

Parameters

txn [MDBX_txn](#)*

Транзакция.

name [byte](#)*

Имя таблицы.

flags [MdbxDbFlags](#)

Флаги таблицы.

dbi [uint](#)*

Указатель для DBI.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxDbiOpen2(MDBX_txn*, MdbxVal*, MdbxDBFlags, uint*)

Открывает таблицу по значению.

```
public static extern int MdbxDbiOpen2(MDBX_txn* txn, MdbxVal* name, MdbxDBFlags flags,
uint* dbi)
```

Parameters

txn [MDBX_txn*](#)

Транзакция.

name [MdbxVal*](#)

Имя таблицы.

flags [MdbxDBFlags](#)

Флаги таблицы.

dbi [uint](#)[↗]*

Указатель для DBI.

Returns

[int](#)[↗]

Код ошибки (0 при успехе).

MdbxDbiOpenEx(MDBX_txn*, byte*, MdbxDBFlags, uint*, delegate* unmanaged[Cdecl]<MdbxVal*, MdbxVal*, int>, delegate* unmanaged[Cdecl]<MdbxVal*, MdbxVal*, int>)

Открывает таблицу по имени с пользовательскими функциями сравнения (устаревшая версия).


```
public static extern int MdbxDbiOpenEx(MDBX_txn* txn, byte* name, MdbxDbFlags flags, uint*
dbi, delegate* unmanaged[Cdecl]<MdbxVal*, MdbxVal*, int> keycmp, delegate* unmanaged[Cdecl]
<MdbxVal*, MdbxVal*, int> datacmp)
```

Parameters

txn [MDBX_txn](#)*

Транзакция.

name [byte](#)*

Имя таблицы (null-terminated).

flags [MdbxDbFlags](#)

Флаги таблицы.

dbi [uint](#)*

Указатель для DBI.

keycmp delegate* unmanaged[Cdecl]<[MdbxVal](#)*, [MdbxVal](#)*, [int](#)>

Функция сравнения ключей (nullptr = использовать по умолчанию).

datacmp delegate* unmanaged[Cdecl]<[MdbxVal](#)*, [MdbxVal](#)*, [int](#)>

Функция сравнения значений (nullptr = использовать по умолчанию).

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxDbiOpenEx2(MDBX_txn*, MdbxVal*, MdbxDbFlags, uint*,
delegate* unmanaged[Cdecl]<MdbxVal*, MdbxVal*, int>,
delegate* unmanaged[Cdecl]<MdbxVal*, MdbxVal*, int>)

Открывает таблицу по значению с пользовательскими функциями сравнения (устаревшая версия).

```
public static extern int MdbxDbiOpenEx2(MDBX_txn* txn, MdbxVal* name, MdbxDbFlags flags,
uint* dbi, delegate* unmanaged[Cdecl]<MdbxVal*, MdbxVal*, int> keycmp, delegate*
unmanaged[Cdecl]<MdbxVal*, MdbxVal*, int> datacmp)
```

Parameters

txn [MDBX_txn](#)*

Транзакция.

name [MdbxVal](#)*

Имя таблицы (MDBX_val).

flags [MdbxDbFlags](#)

Флаги таблицы.

dbi [uint](#)[↗]*

Указатель для DBI.

keycmp delegate* unmanaged[Cdecl]<[MdbxVal](#)*, [MdbxVal](#)*, [int](#)[↗]>

Функция сравнения ключей (nullptr = использовать по умолчанию).

datacmp delegate* unmanaged[Cdecl]<[MdbxVal](#)*, [MdbxVal](#)*, [int](#)[↗]>

Функция сравнения значений (nullptr = использовать по умолчанию).

Returns

[int](#)[↗]

Код ошибки (0 при успехе).

MdbxDbiRename(MDBX_txn*, uint, byte*)

Переименовывает таблицу.

```
public static extern int MdbxDbiRename(MDBX_txn* txn, uint dbi, byte* name)
```

Parameters

txn [MDBX_txn](#)*

Транзакция.

dbi [uint](#)

DBI.

name [byte](#)*

Новое имя.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxDbiRename2(MDBX_txn*, uint, MdbxVal*)

Переименовывает таблицу по значению.

```
public static extern int MdbxDbiRename2(MDBX_txn* txn, uint dbi, MdbxVal* name)
```

Parameters

txn [MDBX_txn](#)*

Транзакция.

dbi [uint](#)

DBI.

name [MdbxVal](#)*

Новое имя.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxDbiSequence(MDBX_txn*, uint, ulong*, ulong)

Получает/устанавливает последовательность таблицы.

```
public static extern int MdbxDbiSequence(MDBX_txn* txn, uint dbi, ulong* result,
ulong increment)
```

Parameters

txn [MDBX_txn*](#)

Транзакция.

dbi [uint](#)

DBI.

result [ulong](#)*

Указатель для результата.

increment [ulong](#)

Инкремент.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxDbiStat(MDBX_txn*, uint, MdbxStat*, nuint)

Получает статистику таблицы.

```
public static extern int MdbxDbiStat(MDBX_txn* txn, uint dbi, MdbxStat* stat, nuint bytes)
```

Parameters

txn [MDBX_txn](#)*

Транзакция.

dbi [uint](#)

DBI.

stat [MdbxStat](#)*

Структура для статистики.

bytes [nuint](#)

Размер структуры.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxDrop(MDBX_txn*, uint, bool)

Удаляет таблицу.

```
public static extern int MdbxDrop(MDBX_txn* txn, uint dbi, bool del)
```

Parameters

txn [MDBX_txn](#)*

Транзакция.

dbi [uint](#)

DBI.

del [bool](#)

Удалить данные.

Returns

Код ошибки (0 при успехе).

Namespace MDBX.Native.Bindings.Debug

Classes

[NativeMdbx](#)

Частичный класс для нативных методов отладки libmdbx.

Class NativeMdbx

Namespace: [MDBX.Native.Bindings.Debug](#)

Assembly: mdbx-dotnet.dll

Частичный класс для нативных методов отладки libmdbx.

```
public static class NativeMdbx
```

Inheritance

[object](#)  ← NativeMdbx

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Methods

MdbxCanaryGet(MDBX_txn*, MdbxCanary*)

Получает структуру-маяк (canary) окружения.

```
public static extern int MdbxCanaryGet(MDBX_txn* txn, MdbxCanary* canary)
```

Parameters

txn [MDBX_txn](#)*

Транзакция.

canary [MdbxCanary](#)*

Структура-маяк для заполнения.

Returns

[int](#) 

Код ошибки (0 при успехе).

MdbxCanaryPut(MDBX_txn*, MdbxCanary*)

Устанавливает структуру-маяк (canary) для окружения.

```
public static extern int MdbxCanaryPut(MDBX_txn* txn, MdbxCanary* canary)
```

Parameters

txn [MDBX_txn*](#)

Транзакция.

canary [MdbxCanary*](#)

Структура-маяк.

Returns

[int](#)[↗]

Код ошибки (0 при успехе).

MdbxDumpVal(MdbxVal*, byte*, nuint)

Форматирует значение MDBX_val в строку.

```
public static extern byte* MdbxDumpVal(MdbxVal* key, byte* buf, nuint bufsize)
```

Parameters

key [MdbxVal*](#)

Ключ для форматирования.

buf [byte](#)[↗]*

Буфер для результата.

bufsize [nuint](#)[↗]

Размер буфера.

Returns

[byte](#)[↗]*

Указатель на строку.

MdbxSetupDebug(MdbxLogLevel, MdbxDebugFlags, delegate* unmanaged[Cdecl]<MdbxLogLevel, byte*, int, byte*, byte*, void>)

Настраивает отладочное логирование libmdbx.

```
public static extern int MdbxSetupDebug(MdbxLogLevel logLevel, MdbxDebugFlags debugFlags,
delegate* unmanaged[Cdecl]<MdbxLogLevel, byte*, int, byte*, byte*, void> logger)
```

Parameters

logLevel [MdbxLogLevel](#)

Уровень логирования.

debugFlags [MdbxDebugFlags](#)

Флаги отладки.

logger delegate* unmanaged[Cdecl]<[MdbxLogLevel](#), [byte](#)[↗]*, [int](#)[↗], [byte](#)[↗]*, [byte](#)[↗]*, [void](#)[↗]>

Функция логирования.

Returns

[int](#)[↗]

Код ошибки (0 при успехе).

MdbxSetupDebugNofmt(MdbxLogLevel, MdbxDebugFlags, delegate* unmanaged[Cdecl]<MdbxLogLevel, byte*, int, byte*, void>, byte*, nuint)

Настраивает отладочное логирование без форматирования.

```
public static extern int MdbxSetupDebugNofmt(MdbxLogLevel logLevel, MdbxDebugFlags
debugFlags, delegate* unmanaged[Cdecl]<MdbxLogLevel, byte*, int, byte*, void> logger, byte*
loggerBuffer, nuint loggerBufferSize)
```

Parameters

logLevel [MdbxLogLevel](#)

Уровень логирования.

debugFlags [MdbxDebugFlags](#)

Флаги отладки.

logger delegate* unmanaged[Cdecl]<[MdbxLogLevel](#), [byte](#)[↗]*, [int](#)[↗], [byte](#)[↗]*, [void](#)[↗]>

Функция логирования без форматирования.

loggerBuffer [byte](#)[↗]*

Буфер для форматирования сообщений.

loggerBufferSize [nuint](#)[↗]

Размер буфера.

Returns

[int](#)[↗]

Код ошибки (0 при успехе).

Namespace MDBX.Native.Bindings.Env

Classes

[NativeMdbx](#)

Частичный класс для нативных методов окружения libmdbx.

Class NativeMdbx

Namespace: [MDBX.Native.Bindings.Env](#)

Assembly: mdbx-dotnet.dll

Частичный класс для нативных методов окружения libmdbx.

```
public static class NativeMdbx
```

Inheritance

[object](#)  ← NativeMdbx

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Methods

MdbxEnvClose(MDBX_env*)

Закрывает окружение MDBX.

```
public static extern int MdbxEnvClose(MDBX_env* env)
```

Parameters

env [MDBX_env*](#)

Окружение.

Returns

[int](#) 

Код ошибки (0 при успехе).

MdbxEnvCloseEx(MDBX_env*, bool)

Закрывает окружение MDBX с дополнительными опциями.

```
public static extern int MdbxEnvCloseEx(MDBX_env* env, bool dontSync)
```

Parameters

env [MDBX_env*](#)

Окружение.

dontSync [bool](#)

Не синхронизировать при закрытии.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxEnvCopy(MDBX_env*, byte*, MdbxCopyFlags)

Копирует окружение MDBX.

```
public static extern int MdbxEnvCopy(MDBX_env* env, byte* dest, MdbxCopyFlags flags)
```

Parameters

env [MDBX_env*](#)

Окружение.

dest [byte*](#)

Путь назначения.

flags [MdbxCopyFlags](#)

Флаги копирования.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxEnvCopy2Fd(MDBX_env*, nint, MdbxCopyFlags)

Копирует окружение MDBX в файловый дескриптор.

```
public static extern int MdbxEnvCopy2Fd(MDBX_env* env, nint fd, MdbxCopyFlags flags)
```

Parameters

env [MDBX_env*](#)

Окружение.

fd [nint](#)

Файловый дескриптор (mdbx_filehandle_t, кроссплатформенный).

flags [MdbxCopyFlags](#)

Флаги копирования.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxEnvCopyW(MDBX_env*, char*, MdbxCopyFlags)

Копирует окружение MDBX (Unicode версия).

```
public static extern int MdbxEnvCopyW(MDBX_env* env, char* dest, MdbxCopyFlags flags)
```

Parameters

env [MDBX_env*](#)

Окружение.

`dest` [char](#)[↗]*

Путь назначения.

`flags` [MdbxCopyFlags](#)

Флаги копирования.

Returns

[int](#)[↗]

Код ошибки (0 при успехе).

MdbxEnvCreate(MDBX_env**)

Создает окружение MDBX.

```
public static extern int MdbxEnvCreate(MDBX_env** penv)
```

Parameters

`penv` [MDBX_env](#)[↗]**

Указатель для хранения окружения.

Returns

[int](#)[↗]

Код ошибки (0 при успехе).

MdbxEnvDefrag(MDBX_env*, nuint, nuint, nuint, nuint, nint, nint, delegate* unmanaged[Cdecl]<void*, MDBX_defrag_result*, int>, void*, MDBX_defrag_result*)

Фрагментирует окружение (defrag).


```
public static extern int MdbxEnvDefrag(MDBX_env* env, nuint defragAtleast, nuint
timeAtleastDot16, nuint defragEnough, nuint timeLimitDot16, nint acceptableBacklash, nint
preferredBatch, delegate* unmanaged[Cdecl]<void*, MDBX_defrag_result*, int>
progressCallback, void* ctx, MDBX_defrag_result* result)
```

Parameters

env [MDBX_env*](#)

Окружение.

defragAtleast [nuint](#)

Минимальное количество страниц для дефрагментации.

timeAtleastDot16 [nuint](#)

Минимальное время (1/65536 сек).

defragEnough [nuint](#)

Достаточное количество страниц.

timeLimitDot16 [nuint](#)

Лимит времени (1/65536 сек).

acceptableBacklash [nint](#)

Допустимый откат.

preferredBatch [nint](#)

Предпочтительный размер пакета.

progressCallback [delegate* unmanaged\[Cdecl\]<\[void\]\(#\)*, \[MDBX_defrag_result\]\(#\)*, \[int\]\(#\)>](#)

Функция обратного вызова для уведомления о прогрессе.

ctx [void*](#)

Контекст для колбэка.

result [MDBX_defrag_result*](#)

Структура результата дефрагментации.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxEnvDelete(byte*, MdbxEnvDeleteMode)

Удаляет окружение MDBX (ANSI версия).

```
public static extern int MdbxEnvDelete(byte* pathname, MdbxEnvDeleteMode mode)
```

Parameters

pathname [byte](#)*

Путь к окружению.

mode [MdbxEnvDeleteMode](#)

Режим удаления.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxEnvDeleteW(char*, MdbxEnvDeleteMode)

Удаляет окружение MDBX (Unicode версия).

```
public static extern int MdbxEnvDeleteW(char* pathname, MdbxEnvDeleteMode mode)
```

Parameters

pathname [char](#)*

Путь к окружению.

mode [MdbxEnvDeleteMode](#)

Режим удаления.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxEnvOpen(MDBX_env*, byte*, MdbxEnvFlags, ushort)

Открывает окружение MDBX (ANSI версия).

```
public static extern int MdbxEnvOpen(MDBX_env* env, byte* pathname, MdbxEnvFlags flags,
ushort mode)
```

Parameters

env [MDBX_env](#)*

Окружение.

pathname [byte](#)*

Путь к файлу.

flags [MdbxEnvFlags](#)

Флаги окружения.

mode [ushort](#)

Режим доступа.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxEnvOpenForRecovery(MDBX_env*, byte*, uint, bool)

Открывает окружение для восстановления (ANSI версия).

```
public static extern int MdbxEnvOpenForRecovery(MDBX_env* env, byte* pathname, uint targetMeta, bool writeable)
```

Parameters

env [MDBX_env*](#)

Окружение.

pathname [byte*](#)

Путь к файлу.

targetMeta [uint](#)

Целевая мета-страница.

writeable [bool](#)

Доступно для записи.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxEnvOpenForRecoveryW(MDBX_env*, char*, uint, bool)

Открывает окружение для восстановления (Unicode версия).

```
public static extern int MdbxEnvOpenForRecoveryW(MDBX_env* env, char* pathname, uint targetMeta, bool writeable)
```

Parameters

env [MDBX_env*](#)

Окружение.

`pathname` [char*](#)

Путь к файлу.

`targetMeta` [uint](#)

Целевая мета-страница.

`writeable` [bool](#)

Доступно для записи.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxEnvOpenW(MDBX_env*, char*, MdbxEnvFlags, ushort)

Открывает окружение MDBX (Unicode версия).

```
public static extern int MdbxEnvOpenW(MDBX_env* env, char* pathname, MdbxEnvFlags flags,
ushort mode)
```

Parameters

`env` [MDBX_env*](#)

Окружение.

`pathname` [char*](#)

Путь к файлу.

`flags` [MdbxEnvFlags](#)

Флаги окружения.

`mode` [ushort](#)

Режим доступа.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxEnvResurrectAfterFork(MDBX_env*)

Восстанавливает окружение после fork.

```
public static extern int MdbxEnvResurrectAfterFork(MDBX_env* env)
```

Parameters

env [MDBX_env*](#)

Окружение.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxEnvTurnForRecovery(MDBX_env*, uint)

Переключает окружение для восстановления.

```
public static extern int MdbxEnvTurnForRecovery(MDBX_env* env, uint targetMeta)
```

Parameters

env [MDBX_env*](#)

Окружение.

targetMeta [uint](#)

Целевая мета-страница.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxEnvWarmup(MDBX_env*, MDBX_txn*, MdbxWarmupFlags, uint)

Прогревает окружение (предзагружает страницы в память).

```
public static extern int MdbxEnvWarmup(MDBX_env* env, MDBX_txn* txn, MdbxWarmupFlags flags,
uint timeoutSeconds16Dot16)
```

Parameters

env [MDBX_env*](#)

Окружение.

txn [MDBX_txn*](#)

Транзакция.

flags [MdbxWarmupFlags](#)

Флаги прогрева.

timeoutSeconds16Dot16 [uint](#)

Таймаут в 1/65536 сек (0 = без ограничения).

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxPreopenSnapinfo(byte*, MDBX_envinfo*, nuint)

Открывает снимок информации окружения (ANSI версия).

```
public static extern int MdbxPreopenSnapinfo(byte* pathname, MDBX_envinfo* info,
nuint bytes)
```

Parameters

pathname [byte](#)*

Путь к файлу.

info [MDBX_envinfo](#)*

Структура информации.

bytes [nuint](#)

Размер структуры.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxPreopenSnapinfoW(char*, MDBX_envinfo*, nuint)

Открывает снимок информации окружения (Unicode версия).

```
public static extern int MdbxPreopenSnapinfoW(char* pathname, MDBX_envinfo* info,
nuint bytes)
```

Parameters

pathname [char](#)*

Путь к файлу.

info [MDBX_envinfo](#)*

Структура информации.

bytes [nuint](#)

Размер структуры.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxTxnCopy2Fd(MDBX_txn*, nint, MdbxCopyFlags)

Копирует транзакцию в файловый дескриптор.

```
public static extern int MdbxTxnCopy2Fd(MDBX_txn* txn, nint fd, MdbxCopyFlags flags)
```

Parameters

txn [MDBX_txn*](#)

Транзакция.

fd [nint](#)

Файловый дескриптор (mdbx_filehandle_t, кроссплатформенный).

flags [MdbxCopyFlags](#)

Флаги копирования.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxTxnCopy2Pathname(MDBX_txn*, byte*, MdbxCopyFlags)

Копирует транзакцию в путь назначения (ANSI версия).

```
public static extern int MdbxTxnCopy2Pathname(MDBX_txn* txn, byte* dest,  
MdbxCopyFlags flags)
```

Parameters

txn [MDBX_txn](#)*

Транзакция.

dest [byte](#)*

Путь назначения.

flags [MdbxCopyFlags](#)

Флаги копирования.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxTxnCopy2PathnameW(MDBX_txn*, char*, MdbxCopyFlags)

Копирует транзакцию в путь назначения (Unicode версия, Windows только).

```
public static extern int MdbxTxnCopy2PathnameW(MDBX_txn* txn, char* dest,  
MdbxCopyFlags flags)
```

Parameters

txn [MDBX_txn](#)*

Транзакция.

dest [char](#)*

Путь назначения.

flags [MdbxCopyFlags](#)

Флаги копирования.

Returns

Код ошибки (0 при успехе).

Namespace MDBX.Native.Bindings.Error Handling

Classes

[NativeMdbx](#)

Частичный класс для нативных методов обработки ошибок libmdbx.

Class NativeMdbx

Namespace: [MDBX.Native.Bindings.ErrorHandling](#)

Assembly: mdbx-dotnet.dll

Частичный класс для нативных методов обработки ошибок libmdbx.

```
public static class NativeMdbx
```

Inheritance

[object](#)  ← NativeMdbx

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Methods

MdbxAssertFail(byte*, byte*, uint)

Функция аварийного завершения при ошибке (assert).

```
public static extern void MdbxAssertFail(byte* msg, byte* func, uint line)
```

Parameters

msg [byte](#) *

Сообщение об ошибке.

func [byte](#) *

Имя функции, в которой произошла ошибка.

line [uint](#) 

Номер строки.

MdbxLiberr2Str(int)

Возвращает строковое описание кода ошибки libmdbx (версия только для ошибок libmdbx).

```
public static extern byte* MdbxLiberr2Str(int errnum)
```

Parameters

errnum [int](#)

Код ошибки.

Returns

[byte](#)*

Указатель на статическую строку с описанием ошибки.

MdbxSetPanic(delegate* unmanaged[Cdecl]<byte*, byte*, uint, void*, void>)

Устанавливает функцию обработки паники (критических ошибок).

```
public static extern void MdbxSetPanic(delegate* unmanaged[Cdecl]<byte*, byte*, uint, void*, void> func)
```

Parameters

func [delegate](#)* [unmanaged](#)[[Cdecl](#)]<[byte](#)*, [byte](#)*, [uint](#), [void](#)*, [void](#)>

Указатель на функцию обработки паники.

MdbxStrerror(int)

Возвращает строковое описание кода ошибки libmdbx.

```
public static extern byte* MdbxStrerror(int errnum)
```

Parameters

`errno` [int](#)

Код ошибки.

Returns

[byte](#)*

Указатель на статическую строку с описанием ошибки.

MdbxStrerrorAnsi2Oem(int)

Возвращает строковое описание кода ошибки в кодировке ANSI/OEM (Windows).

```
public static extern byte* MdbxStrerrorAnsi2Oem(int errno)
```

Parameters

`errno` [int](#)

Код ошибки.

Returns

[byte](#)*

Указатель на статическую строку с описанием ошибки.

MdbxStrerrorR(int, byte*, nuint)

Копирует строковое описание кода ошибки в предоставленный буфер.

```
public static extern byte* MdbxStrerrorR(int errno, byte* buf, nuint buflen)
```

Parameters

`errno` [int](#)

Код ошибки.

`buf` [byte](#)*

Буфер для копирования строки.

`buflen` [nuint](#)

Размер буфера в байтах.

Returns

[byte](#)*

Указатель на буфер с строкой описания ошибки.

MdbxStrerrorRAnsi2Oem(int, byte*, nuint)

Копирует строковое описание кода ошибки в предоставленный буфер (ANSI/OEM версия).

```
public static extern byte* MdbxStrerrorRAnsi2Oem(int errnum, byte* buf, nuint buflen)
```

Parameters

`errnum` [int](#)

Код ошибки.

`buf` [byte](#)*

Буфер для копирования строки.

`buflen` [nuint](#)

Размер буфера в байтах.

Returns

[byte](#)*

Указатель на буфер с строкой описания ошибки.

Namespace MDBX.Native.Bindings.Extra

Classes

[NativeMdbx](#)

Частичный класс для нативных методов дополнительных операций libmdbx.

Class NativeMdbx

Namespace: [MDBX.Native.Bindings.Extra](#)

Assembly: mdbx-dotnet.dll

Частичный класс для нативных методов дополнительных операций libmdbx.

```
public static class NativeMdbx
```

Inheritance

[object](#)  ← NativeMdbx

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Methods

MdbxCacheGet(MDBX_txn*, uint, MdbxVal*, MdbxVal*, MdbxCacheEntry*)

Получает элемент из таблицы с использованием кэша (многопоточная версия).

```
public static extern MdbxCacheResult MdbxCacheGet(MDBX_txn* txn, uint dbi, MdbxVal* key,  
MdbxVal* data, MdbxCacheEntry* entry)
```

Parameters

txn [MDBX_txn](#)*

Транзакция.

dbi [uint](#) 

DBI.

key [MdbxVal](#)*

Ключ.

data [MdbxVal](#)*

Значение.

entry [MdbxCacheEntry](#)*

Кэш-запись.

Returns

[MdbxCacheResult](#)

Результат кэш-операции (MDBX_cache_result_t).

MdbxCacheGetSingleThreaded(MDBX_txn*, uint, MdbxVal*, MdbxVal*, MdbxCacheEntry*)

Получает элемент из таблицы с использованием кэша (однопоточная версия).

```
public static extern MdbxCacheResult MdbxCacheGetSingleThreaded(MDBX_txn* txn, uint dbi,
MdbxVal* key, MdbxVal* data, MdbxCacheEntry* entry)
```

Parameters

txn [MDBX_txn](#)*

Транзакция.

dbi [uint](#)

DBI.

key [MdbxVal](#)*

Ключ.

data [MdbxVal](#)*

Значение.

entry [MdbxCacheEntry](#)*

Кэш-запись.

Returns

[MdbxCacheResult](#)

Результат кэш-операции (MDBX_cache_result_t).

MdbxCmp(MDBX_txn*, uint, MdbxVal*, MdbxVal*)

Сравнивает ключи.

```
public static extern int MdbxCmp(MDBX_txn* txn, uint dbi, MdbxVal* a, MdbxVal* b)
```

Parameters

txn [MDBX_txn](#)*

Транзакция.

dbi [uint](#)

DBI.

a [MdbxVal](#)*

Первый ключ.

b [MdbxVal](#)*

Второй ключ.

Returns

[int](#)

Результат сравнения.

MdbxDbiSequence(MDBX_txn*, uint, ulong*, ulong)

Получает/устанавливает последовательность таблицы.

```
public static extern int MdbxDbiSequence(MDBX_txn* txn, uint dbi, ulong* result,
ulong increment)
```

Parameters

txn [MDBX_txn*](#)

Транзакция.

dbi [uint](#)

DBI.

result [ulong](#)*

Результат.

increment [ulong](#)

Инкремент.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxDcmp(MDBX_txn*, uint, MdbxVal*, MdbxVal*)

Сравнивает значения.

```
public static extern int MdbxDcmp(MDBX_txn* txn, uint dbi, MdbxVal* a, MdbxVal* b)
```

Parameters

txn [MDBX_txn*](#)

Транзакция.

dbi [uint](#)

DBI.

a [MdbxVal](#)*

Первое значение.

b [MdbxVal](#)*

Второе значение.

Returns

[int](#)[↗]

Результат сравнения.

MdbxDoubleFromKey(MdbxVal)

Извлекает double из ключа.

```
public static extern double MdbxDoubleFromKey(MdbxVal key)
```

Parameters

key [MdbxVal](#)

Ключ.

Returns

[double](#)[↗]

Double значение.

MdbxDumpVal(MdbxVal*, byte*, nuint)

Форматирует значение в строку.

```
public static extern byte* MdbxDumpVal(MdbxVal* key, byte* buf, nuint bufsize)
```

Parameters

key [MdbxVal](#)*

Ключ.

buf [byte](#)*

Буфер.

bufsize [nuint](#)

Размер буфера.

Returns

[byte](#)*

Указатель на строку.

MdbxEnumerateTables(MDBX_txn*, delegate*
unmanaged[Cdecl]<void*, MDBX_txn*, MdbxVal*, uint, uint,
int>, void*)

Перечисляет таблицы.

```
public static extern int MdbxEnumerateTables(MDBX_txn* txn, delegate* unmanaged[Cdecl]  
<void*, MDBX_txn*, MdbxVal*, uint, uint, int> func, void* ctx)
```

Parameters

txn [MDBX_txn](#)*

Транзакция.

func delegate* unmanaged[Cdecl]<[void](#)*, [MDBX_txn](#)*, [MdbxVal](#)*, [uint](#), [uint](#), [int](#)>

Функция обратного вызова.

ctx [void](#)*

Контекст.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxEnvChk(MDBX_env*, MdbxChkCallbacks*, MdbxChkContext*, MdbxChkFlags, MdbxChkSeverity, uint)

Проверяет целостность базы данных.

```
public static extern int MdbxEnvChk(MDBX_env* env, MdbxChkCallbacks* cb, MdbxChkContext* ctx, MdbxChkFlags flags, MdbxChkSeverity verbosity, uint timeoutSeconds16dot16)
```

Parameters

env [MDBX_env](#)*

Окружение.

cb [MdbxChkCallbacks](#)*

Функции обратного вызова для проверки.

ctx [MdbxChkContext](#)*

Контекст проверки.

flags [MdbxChkFlags](#)

Флаги проверки.

verbosity [MdbxChkSeverity](#)

Уровень детализации.

timeoutSeconds16dot16 [uint](#)

Таймаут в 1/65536 секунд (0 = без ограничения).

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxEnvChkEncountProblem(MdbxChkContext*)

Учитывает проблемы, обнаруженные приложением во время проверки.

```
public static extern int MdbxEnvChkEncountProblem(MdbxChkContext* ctx)
```

Parameters

ctx [MdbxChkContext*](#)

Контекст проверки.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxEstimateDistance(MDBX_cursor*, MDBX_cursor*, nint*)

Оценивает расстояние между курсорами.

```
public static extern int MdbxEstimateDistance(MDBX_cursor* first, MDBX_cursor* last,  
nint* distanceItems)
```

Parameters

first [MDBX_cursor*](#)

Первый курсор.

last [MDBX_cursor*](#)

Последний курсор.

distanceItems [nint](#)*

Количество элементов.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxEstimateMove(MDBX_cursor*, MdbxVal*, MdbxVal*, MdbxCursorOp, nint*)

Оценивает движение курсора.

```
public static extern int MdbxEstimateMove(MDBX_cursor* cursor, MdbxVal* key, MdbxVal* data,
MdbxCursorOp moveOp, nint* distanceItems)
```

Parameters

cursor [MDBX_cursor](#)*

Курсор.

key [MdbxVal](#)*

Ключ.

data [MdbxVal](#)*

Значение.

moveOp [MdbxCursorOp](#)

Операция движения.

distanceItems [nint](#)*

Количество элементов.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxEstimateRange(MDBX_txn*, uint, MdbxVal*, MdbxVal*, nint*)

Оценивает диапазон.

```
public static extern int MdbxEstimateRange(MDBX_txn* txn, uint dbi, MdbxVal* beginKey,
MdbxVal* endKey, nint* distanceItems)
```

Parameters

txn [MDBX_txn*](#)

Транзакция.

dbi [uint](#)

DBI.

beginKey [MdbxVal*](#)

Начальный ключ.

endKey [MdbxVal*](#)

Конечный ключ.

distanceItems [nint](#)*

Количество элементов.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxFloatFromKey(MdbxVal)

Извлекает float из ключа.

```
public static extern float MdbxFloatFromKey(MdbxVal key)
```

Parameters

key [MdbxVal](#)

Ключ.

Returns

[float](#)

Float значение.

MdbxGcInfo(MDBX_txn*, MdbxGcInfo*, nuint, delegate* unmanaged[Cdecl]<void*, MDBX_txn*, ulong, nuint, nuint, bool, int>, void*)

Получает информацию о сборке мусора (GC) и использовании страниц.

```
public static extern int MdbxGcInfo(MDBX_txn* txn, MdbxGcInfo* info, nuint bytes, delegate* unmanaged[Cdecl]<void*, MDBX_txn*, ulong, nuint, nuint, bool, int> iterFunc, void* iterCtx)
```

Parameters

txn [MDBX_txn](#)*

Транзакция.

info [MdbxGcInfo](#)*

Структура для информации о GC.

bytes [nuint](#)

Размер структуры info.

iterFunc delegate* unmanaged[Cdecl]<[void](#)*, [MDBX_txn](#)*, [ulong](#), [nuint](#), [nuint](#), [bool](#), [int](#)>

Функция итерации GC (nullptr = не вызывать).

iterCtx [void](#)*

Контекст для функции итерации.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxGetDatacmp(MdbxDbFlags)

Получает функцию сравнения значений.

```
public static extern delegate* unmanaged[Cdecl]<MdbxVal*, MdbxVal*, int>  
MdbxGetDatacmp(MdbxDbFlags flags)
```

Parameters

flags [MdbxDbFlags](#)

Флаги таблицы.

Returns

delegate* unmanaged[Cdecl]<[MdbxVal*](#), [MdbxVal*](#), [int](#)>

Указатель на функцию сравнения.

MdbxGetKeycmp(MdbxDbFlags)

Получает функцию сравнения ключей.

```
public static extern delegate* unmanaged[Cdecl]<MdbxVal*, MdbxVal*, int>  
MdbxGetKeycmp(MdbxDbFlags flags)
```

Parameters

flags [MdbxDbFlags](#)

Флаги таблицы.

Returns

`delegate* unmanaged[Cdecl]<MdbxVal*, MdbxVal*, int>`

Указатель на функцию сравнения.

MdbxInt32FromKey(MdbxVal)

Извлекает int32 из ключа.

```
public static extern int MdbxInt32FromKey(MdbxVal key)
```

Parameters

key [MdbxVal](#)

Ключ.

Returns

[int](#)

Int32 значение.

MdbxInt64FromKey(MdbxVal)

Извлекает int64 из ключа.

```
public static extern long MdbxInt64FromKey(MdbxVal key)
```

Parameters

key [MdbxVal](#)

Ключ.

Returns

[long](#)

Int64 значение.

MdbxIsDirty(MDBX_txn*, void*)

Проверяет, является ли адрес "грязной" страницы.

```
public static extern int MdbxIsDirty(MDBX_txn* txn, void* ptr)
```

Parameters

txn [MDBX_txn*](#)

Транзакция.

ptr [void*](#)

Указатель для проверки.

Returns

[int](#)

MDBX_RESULT_TRUE (адрес на грязной странице), MDBX_RESULT_FALSE (адрес НЕ на грязной странице), или код ошибки.

MdbxJsonIntegerFromKey(MdbxVal)

Извлекает JSON integer из ключа.

```
public static extern long MdbxJsonIntegerFromKey(MdbxVal key)
```

Parameters

key [MdbxVal](#)

Ключ.

Returns

[long](#)

JSON integer.

MdbxKeyFromDouble(double)

Преобразует double в ключ.

```
public static extern ulong MdbxKeyFromDouble(double ieee754_64bit)
```

Parameters

ieee754_64bit [double](#)

Double значение.

Returns

[ulong](#)

Ключ.

MdbxKeyFromFloat(float)

Преобразует float в ключ.

```
public static extern uint MdbxKeyFromFloat(float ieee754_32bit)
```

Parameters

ieee754_32bit [float](#)

Float значение.

Returns

[uint](#)

Ключ.

MdbxKeyFromJsonInteger(long)

Преобразует JSON integer в ключ.


```
public static extern ulong MdbxKeyFromJsonInteger(long jsonInteger)
```

Parameters

`jsonInteger` [long](#)

JSON integer.

Returns

[ulong](#)

Ключ.

MdbxKeyFromPtrdouble(double*)

Преобразует указатель на double в ключ.

```
public static extern ulong MdbxKeyFromPtrdouble(double* ieee754_64bit)
```

Parameters

`ieee754_64bit` [double](#)*

Указатель на double.

Returns

[ulong](#)

Ключ.

MdbxKeyFromPtrfloat(float*)

Преобразует указатель на float в ключ.

```
public static extern uint MdbxKeyFromPtrfloat(float* ieee754_32bit)
```

Parameters

`ieee754_32bit` [float](#)^{*}

Указатель на float.

Returns

[uint](#)^{*}

Ключ.

MdbxReaderCheck(MDBX_env*, int*)

Проверяет читателей.

```
public static extern int MdbxReaderCheck(MDBX_env* env, int* dead)
```

Parameters

`env` [MDBX_env](#)^{*}

Окружение.

`dead` [int](#)^{*}

Количество мертвых читателей.

Returns

[int](#)^{*}

Код ошибки (0 при успехе).

MdbxReaderList(MDBX_env*, delegate* unmanaged[Cdecl] <void*, int, int, uint, uint, ulong, ulong, MdbxVal*, int>, void*)

Получает список читателей.

```
public static extern int MdbxReaderList(MDBX_env* env, delegate* unmanaged[Cdecl]<void*,
int, int, uint, uint, ulong, ulong, MdbxVal*, int> func, void* ctx)
```

Parameters

env [MDBX_env](#)*

Окружение.

func delegate* unmanaged[Cdecl]<[void](#)*, [int](#), [int](#), [uint](#), [uint](#), [ulong](#), [ulong](#), [MdbxVal](#)*, [int](#)>

Функция обратного вызова.

ctx [void](#)*

Контекст.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxSetPanic(delegate* unmanaged[Cdecl]<byte*, byte*, uint, void*, void>)

Устанавливает функцию паники.

```
public static extern void MdbxSetPanic(delegate* unmanaged[Cdecl]<byte*, byte*, uint, void*,
void> func)
```

Parameters

func delegate* unmanaged[Cdecl]<[byte](#)*, [byte](#)*, [uint](#), [void](#)*, [void](#)>

Функция паники.

MdbxThreadRegister(MDBX_env*)

Регистрирует поток.

```
public static extern int MdbxThreadRegister(MDBX_env* env)
```

Parameters

`env` [MDBX_env*](#)

Окружение.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxThreadUnregister(MDBX_env*)

Deregisters thread.

```
public static extern int MdbxThreadUnregister(MDBX_env* env)
```

Parameters

`env` [MDBX_env*](#)

Environment.

Returns

[int](#)

Error code (0 on success).

MdbxTxnLock(MDBX_env*, bool)

Блокирует окружение для транзакции.

```
public static extern int MdbxTxnLock(MDBX_env* env, bool dontWait)
```

Parameters

`env` [MDBX_env*](#)

Окружение.

`dontWait` [bool](#)

Не ждать.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxTxnUnlock(MDBX_env*)

Разблокирует окружение.

```
public static extern int MdbxTxnUnlock(MDBX_env* env)
```

Parameters

`env` [MDBX_env*](#)

Окружение.

Returns

[int](#)

Код ошибки (0 при успехе).

Namespace MDBX.Native.Bindings.Settings

Classes

[NativeMdbx](#)

Частичный класс для нативных методов настроек libmdbx.

Class NativeMdbx

Namespace: [MDBX.Native.Bindings.Settings](#)

Assembly: mdbx-dotnet.dll

Частичный класс для нативных методов настроек libmdbx.

```
public static class NativeMdbx
```

Inheritance

[object](#)  ← NativeMdbx

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Methods

MdbxDefaultPagesize()

Получает размер страницы по умолчанию.

```
public static extern nuint MdbxDefaultPagesize()
```

Returns

[nuint](#) 

Размер страницы в байтах.

MdbxEnvGetFd(MDBX_env*, nint*)

Получает дескриптор файла окружения.

```
public static extern int MdbxEnvGetFd(MDBX_env* env, nint* fd)
```

Parameters

env [MDBX_env](#)*

Окружение.

fd [nint](#)*

Указатель для дескриптора.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxEnvGetFlags(MDBX_env*, uint*)

Получает флаги окружения.

```
public static extern int MdbxEnvGetFlags(MDBX_env* env, uint* flags)
```

Parameters

env [MDBX_env](#)*

Окружение.

flags [uint](#)*

Указатель для флагов.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxEnvGetHsr(MDBX_env*)

Получает колбэк Handle-Slow-Readers.


```
public static extern delegate* unmanaged[Cdecl]<MDBX_env*, MDBX_txn*, uint, uint, ulong,
uint, nuint, int> MdbxEnvGetHsr(MDBX_env* env)
```

Parameters

env [MDBX_env*](#)

Окружение.

Returns

delegate* unmanaged[Cdecl]<[MDBX_env*](#), [MDBX_txn*](#), [uint](#), [uint](#), [ulong](#), [uint](#), [nuint](#), [int](#)>

Указатель на функцию колбэка.

MdbxEnvGetMaxkeysize(MDBX_env*)

Получает максимальный размер ключа (устаревшая версия без флагов).

```
public static extern int MdbxEnvGetMaxkeysize(MDBX_env* env)
```

Parameters

env [MDBX_env*](#)

Окружение.

Returns

[int](#)

Максимальный размер ключа.

MdbxEnvGetMaxkeysizeEx(MDBX_env*, MdbxDbFlags)

Получает максимальный размер ключа.

```
public static extern int MdbxEnvGetMaxkeysizeEx(MDBX_env* env, MdbxDbFlags flags)
```

Parameters

env [MDBX_env*](#)

Окружение.

flags [MdbxDbFlags](#)

Флаги таблицы.

Returns

[int](#)

Максимальный размер ключа.

MdbxEnvGetMaxvalsizeEx(MDBX_env*, MdbxDbFlags)

Получает максимальный размер значения.

```
public static extern int MdbxEnvGetMaxvalsizeEx(MDBX_env* env, MdbxDbFlags flags)
```

Parameters

env [MDBX_env*](#)

Окружение.

flags [MdbxDbFlags](#)

Флаги таблицы.

Returns

[int](#)

Максимальный размер значения.

MdbxEnvGetOption(MDBX_env*, MdbxOption, ulong*)

Получает параметр окружения.

```
public static extern int MdbxEnvGetOption(MDBX_env* env, MdbxOption option, ulong* pValue)
```

Parameters

env [MDBX_env](#)*

Окружение.

option [MdbxOption](#)

Параметр.

pValue [ulong](#)*[↗]

Указатель для значения.

Returns

[int](#)[↗]

Код ошибки (0 при успехе).

MdbxEnvGetPairsize4pageMax(MDBX_env*, MdbxDBFlags)

Получает максимальный размер пары ключ-значение для страницы.

```
public static extern int MdbxEnvGetPairsize4pageMax(MDBX_env* env, MdbxDBFlags flags)
```

Parameters

env [MDBX_env](#)*

Окружение.

flags [MdbxDBFlags](#)

Флаги таблицы.

Returns

[int](#)[↗]

Максимальный размер пары.

MdbxEnvGetPath(MDBX_env*, byte**)

Получает путь к окружению.

```
public static extern int MdbxEnvGetPath(MDBX_env* env, byte** dest)
```

Parameters

env [MDBX_env*](#)

Окружение.

dest [byte](#)**

Буфер для пути.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxEnvGetPathW(MDBX_env*, char**)

Получает путь к окружению (wide version).

```
public static extern int MdbxEnvGetPathW(MDBX_env* env, char** dest)
```

Parameters

env [MDBX_env*](#)

Окружение.

dest [char](#)**

Буфер для пути.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxEnvGetUserctx(MDBX_env*)

Получает пользовательский контекст окружения.

```
public static extern void* MdbxEnvGetUserctx(MDBX_env* env)
```

Parameters

env [MDBX_env](#)*

Окружение.

Returns

[void](#)*

Указатель на пользовательский контекст.

MdbxEnvGetValsize4pageMax(MDBX_env*, MdbxDbFlags)

Получает максимальный размер значения для страницы.

```
public static extern int MdbxEnvGetValsize4pageMax(MDBX_env* env, MdbxDbFlags flags)
```

Parameters

env [MDBX_env](#)*

Окружение.

flags [MdbxDbFlags](#)

Флаги таблицы.

Returns

[int](#)

Максимальный размер значения.

MdbxEnvSetFlags(MDBX_env*, MdbxEnvFlags, bool)

Устанавливает флаги окружения.

```
public static extern int MdbxEnvSetFlags(MDBX_env* env, MdbxEnvFlags flags, bool onoff)
```

Parameters

env [MDBX_env*](#)

Окружение.

flags [MdbxEnvFlags](#)

Флаги.

onoff [bool](#)

Включить или выключить.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxEnvSetGeometry(MDBX_env*, nint, nint, nint, nint, nint, nint)

Устанавливает геометрию окружения (размеры файлов).

```
public static extern int MdbxEnvSetGeometry(MDBX_env* env, nint sizeLower, nint sizeNow, nint sizeUpper, nint growthStep, nint shrinkThreshold, nint pagesize)
```

Parameters

`env` [MDBX_env*](#)

Окружение.

`sizeLower` [nint](#)

Нижний предел размера.

`sizeNow` [nint](#)

Текущий размер.

`sizeUpper` [nint](#)

Верхний предел размера.

`growthStep` [nint](#)

Шаг роста.

`shrinkThreshold` [nint](#)

Порог сжатия.

`pagesize` [nint](#)

Размер страницы.

Returns

[int](#)

Код ошибки (0 при успехе).

`MdbxEnvSetHsr(MDBX_env*, delegate* unmanaged[Cdecl]
<MDBX_env*, MDBX_txn*, uint, uint, ulong, uint, nuint, int>)`

Устанавливает колбэк Handle-Slow-Readers.

```
public static extern int MdbxEnvSetHsr(MDBX_env* env, delegate* unmanaged[Cdecl]<MDBX_env*,  
MDBX_txn*, uint, uint, ulong, uint, nuint, int> hsrCallback)
```

Parameters

env [MDBX_env](#)*

Окружение.

hsrCallback `delegate* unmanaged[Cdecl]<MDBX\_env*, MDBX\_txn*, uint, uint, ulong, uint, nuint, int>`

Функция колбэка.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxEnvSetOption(MDBX_env*, MdbxOption, ulong)

Устанавливает параметр окружения.

```
public static extern int MdbxEnvSetOption(MDBX_env* env, MdbxOption option, ulong value)
```

Parameters

env [MDBX_env](#)*

Окружение.

option [MdbxOption](#)

Параметр.

value [ulong](#)

Значение.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxEnvSetUserctx(MDBX_env*, void*)

Устанавливает пользовательский контекст окружения.

```
public static extern int MdbxEnvSetUserctx(MDBX_env* env, void* ctx)
```

Parameters

env [MDBX_env*](#)

Окружение.

ctx [void*](#)

Пользовательский контекст.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxGetSysraminfo(nint*, nint*, nint*)

Получает информацию о системной RAM.

```
public static extern int MdbxGetSysraminfo(nint* pageSize, nint* totalPages,  
nint* availPages)
```

Parameters

pageSize [nint*](#)

Размер страницы.

totalPages [nint*](#)

Общее количество страниц.

availPages [nint*](#)

Доступное количество страниц.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxIsReadaheadReasonable(nuint, nint)

Проверяет, разумно ли использовать readahead для заданного объема.

```
public static extern int MdbxIsReadaheadReasonable(nuint volume, nint redundancy)
```

Parameters

volume [nuint](#)

Объем данных.

redundancy [nint](#)

Избыточность.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxLimitsDbsizeMax(nint)

Получает максимальный размер базы данных.

```
public static extern nint MdbxLimitsDbsizeMax(nint pagesize)
```

Parameters

pagesize [nint](#)

Размер страницы.

Returns

[nint](#)

Максимальный размер.

MdbxLimitsDbsizeMin(nint)

Получает минимальный размер базы данных.

```
public static extern nint MdbxLimitsDbsizeMin(nint pagesize)
```

Parameters

pagesize [nint](#)

Размер страницы.

Returns

[nint](#)

Минимальный размер.

MdbxLimitsKeysizeMax(nint, MdbxDbFlags)

Получает максимальный размер ключа.

```
public static extern nint MdbxLimitsKeysizeMax(nint pagesize, MdbxDbFlags flags)
```

Parameters

pagesize [nint](#)

Размер страницы.

flags [MdbxDbFlags](#)

Флаги таблицы.

Returns

[nint](#)

Максимальный размер ключа.

MdbxLimitsKeysizeMin(MdbxDbFlags)

Получает минимальный размер ключа.

```
public static extern nint MdbxLimitsKeysizeMin(MdbxDbFlags flags)
```

Parameters

flags [MdbxDbFlags](#)

Флаги таблицы.

Returns

[nint](#)

Минимальный размер ключа.

MdbxLimitsPairsize4pageMax(nint, MdbxDbFlags)

Получает максимальный размер пары ключ-значение для страницы.

```
public static extern nint MdbxLimitsPairsize4pageMax(nint pagesize, MdbxDbFlags flags)
```

Parameters

pagesize [nint](#)

Размер страницы.

flags [MdbxDbFlags](#)

Флаги таблицы.

Returns

[nint](#)

Максимальный размер пары.

MdbxLimitsTxnsizeMax(nint)

Получает максимальный размер транзакции.

```
public static extern nint MdbxLimitsTxnsizeMax(nint pagesize)
```

Parameters

pagesize [nint](#)

Размер страницы.

Returns

[nint](#)

Максимальный размер транзакции.

MdbxLimitsValse4pageMax(nint, MdbxDbFlags)

Получает максимальный размер значения для страницы.

```
public static extern nint MdbxLimitsValse4pageMax(nint pagesize, MdbxDbFlags flags)
```

Parameters

pagesize [nint](#)

Размер страницы.

flags [MdbxDbFlags](#)

Флаги таблицы.

Returns

[nint](#)

Максимальный размер значения.

MdbxLimitsValsizeMax(nint, MdbxDbFlags)

Получает максимальный размер значения.

```
public static extern nint MdbxLimitsValsizeMax(nint pagesize, MdbxDbFlags flags)
```

Parameters

pagesize [nint](#)

Размер страницы.

flags [MdbxDbFlags](#)

Флаги таблицы.

Returns

[nint](#)

Максимальный размер значения.

MdbxLimitsValsizeMin(MdbxDbFlags)

Получает минимальный размер значения.

```
public static extern nint MdbxLimitsValsizeMin(MdbxDbFlags flags)
```

Parameters

flags [MdbxDbFlags](#)

Флаги таблицы.

Returns

[nint](#)[↗]

Минимальный размер значения.

MdbxRatio2Digits(ulong, ulong, int, byte*, nuint)

Преобразует соотношение в строку с заданной точностью.

```
public static extern byte* MdbxRatio2Digits(ulong numerator, ulong denominator, int precision, byte* buffer, nuint bufferSize)
```

Parameters

numerator [ulong](#)[↗]

Числитель.

denominator [ulong](#)[↗]

Знаменатель.

precision [int](#)[↗]

Точность.

buffer [byte](#)[↗]*

Буфер для результата.

bufferSize [nuint](#)[↗]

Размер буфера.

Returns

[byte](#)[↗]*

Указатель на строку.

MdbxRatio2Percents(ulong, ulong, byte*, nuint)

Преобразует соотношение в проценты.

```
public static extern byte* MdbxRatio2Percents(ulong value, ulong whole, byte* buffer,
nuint bufferSize)
```

Parameters

value [ulong](#)

Значение.

whole [ulong](#)

Целое.

buffer [byte](#)*

Буфер для результата.

bufferSize [nuint](#)

Размер буфера.

Returns

[byte](#)*

Указатель на строку.

Namespace MDBX.Native.Bindings.Stat

Classes

[NativeMdbx](#)

Частичный класс для нативных методов статистики libmdbx.

Class NativeMdbx

Namespace: [MDBX.Native.Bindings.Stat](#)

Assembly: mdbx-dotnet.dll

Частичный класс для нативных методов статистики libmdbx.

```
public static class NativeMdbx
```

Inheritance

[object](#)  ← NativeMdbx

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Methods

MdbxEnvInfoEx(MDBX_env*, MDBX_txn*, MDBX_envinfo*, nuint)

Получает информацию об окружении.

```
public static extern int MdbxEnvInfoEx(MDBX_env* env, MDBX_txn* txn, MDBX_envinfo* info,  
nuint bytes)
```

Parameters

env [MDBX_env](#)*

Окружение.

txn [MDBX_txn](#)*

Транзакция (может быть null).

info [MDBX_envinfo](#)*

Структура для информации.

bytes [nuint](#) 

Размер структуры.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxEnvStatEx(MDBX_env*, MDBX_txn*, MdbxStat*, nuint)

Получает статистику окружения.

```
public static extern int MdbxEnvStatEx(MDBX_env* env, MDBX_txn* txn, MdbxStat* stat,
nuint bytes)
```

Parameters

env [MDBX_env](#)*

Окружение.

txn [MDBX_txn](#)*

Транзакция (может быть null).

stat [MdbxStat](#)*

Структура для статистики.

bytes [nuint](#)

Размер структуры.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxEnvSyncEx(MDBX_env*, bool, bool)

Синхронизирует окружение.

```
public static extern int MdbxEnvSyncEx(MDBX_env* env, bool force, bool nonblock)
```

Parameters

`env` [MDBX_env](#)*

Окружение.

`force` [bool](#)↗

Принудительная синхронизация.

`nonblock` [bool](#)↗

Неблокирующая.

Returns

[int](#)↗

Код ошибки (0 при успехе).

Namespace MDBX.Native.Bindings.Txn

Classes

[NativeMdbx](#)

Частичный класс для нативных методов транзакций libmdbx.

Class NativeMdbx

Namespace: [MDBX.Native.Bindings.Txn](#)

Assembly: mdbx-dotnet.dll

Частичный класс для нативных методов транзакций libmdbx.

```
public static class NativeMdbx
```

Inheritance

[object](#)  ← NativeMdbx

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Methods

MdbxTxnAbortEx(MDBX_txn*, MdbxCommitLatency*)

Отменяет транзакцию.

```
public static extern int MdbxTxnAbortEx(MDBX_txn* txn, MdbxCommitLatency* latency)
```

Parameters

txn [MDBX_txn](#)*

Транзакция.

latency [MdbxCommitLatency](#)*

Информация о задержках.

Returns

[int](#) 

Код ошибки (0 при успехе).

MdbxTxnAmend(MDBX_txn*, MDBX_txn**, MdbxTxnFlags, void*)

Изменяет read-транзакцию в write-транзакцию.

```
public static extern int MdbxTxnAmend(MDBX_txn* readTxn, MDBX_txn** ptrWriteTxn,
MdbxTxnFlags flags, void* context)
```

Parameters

readTxn [MDBX_txn*](#)

Read-транзакция.

ptrWriteTxn [MDBX_txn**](#)

Указатель для write-транзакции.

flags [MdbxTxnFlags](#)

Флаги.

context [void](#)[↗]*

Пользовательский контекст.

Returns

[int](#)[↗]

Код ошибки (0 при успехе).

MdbxTxnBeginEx(MDBX_env*, MDBX_txn*, MdbxTxnFlags, MDBX_txn**, nuint)

Начинает транзакцию.

```
public static extern int MdbxTxnBeginEx(MDBX_env* env, MDBX_txn* parent, MdbxTxnFlags flags,
MDBX_txn** txn, nuint size)
```

Parameters

env [MDBX_env*](#)

Окружение.

parent [MDBX_txn*](#)

Родительская транзакция.

flags [MdbxTxnFlags](#)

Флаги транзакции.

txn [MDBX_txn**](#)

Указатель для транзакции.

size [nuint](#)

Размер.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxTxnBreak(MDBX_txn*)

Ломает транзакцию (прерывает операцию).

```
public static extern int MdbxTxnBreak(MDBX_txn* txn)
```

Parameters

txn [MDBX_txn*](#)

Транзакция.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxTxnCheckpoint(MDBX_txn*, MdbxTxnFlags, MdbxCommitLatency*)

Выполняет checkpoint транзакции.

```
public static extern int MdbxTxnCheckpoint(MDBX_txn* txn, MdbxTxnFlags weakeningDurability,
MdbxCommitLatency* latency)
```

Parameters

txn [MDBX_txn*](#)

Транзакция.

weakeningDurability [MdbxTxnFlags](#)

Флаги ослабления durability.

latency [MdbxCommitLatency*](#)

Информация о задержках.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxTxnClone(MDBX_txn*, MDBX_txn**, void*)

Клонирует транзакцию.

```
public static extern int MdbxTxnClone(MDBX_txn* origin, MDBX_txn** inOutClone,
void* context)
```

Parameters

origin [MDBX_txn*](#)

Исходная транзакция.

`inOutClone` [MDBX_txn](#)**

Указатель для клона.

`context` [void](#)*

Пользовательский контекст.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxTxnCommitEmbarkRead(MDBX_txn**, MdbxCommitLatency*)

Фиксирует транзакцию для embark read.

```
public static extern int MdbxTxnCommitEmbarkRead(MDBX_txn** ptxn,  
MdbxCommitLatency* latency)
```

Parameters

`ptxn` [MDBX_txn](#)**

Указатель на транзакцию.

`latency` [MdbxCommitLatency](#)*

Информация о задержках.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxTxnCommitEx(MDBX_txn*, MdbxCommitLatency*)

Фиксирует транзакцию.

```
public static extern int MdbxTxnCommitEx(MDBX_txn* txn, MdbxCommitLatency* latency)
```

Parameters

txn [MDBX_txn](#)*

Транзакция.

latency [MdbxCommitLatency](#)*

Информация о задержках.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxTxnEnv(MDBX_txn*)

Получает окружение транзакции.

```
public static extern MDBX_env* MdbxTxnEnv(MDBX_txn* txn)
```

Parameters

txn [MDBX_txn](#)*

Транзакция.

Returns

[MDBX_env](#)*

Указатель на окружение.

MdbxTxnFlags(MDBX_txn*)

Получает флаги транзакции.

```
public static extern MdbxTxnFlags MdbxTxnFlags(MDBX_txn* txn)
```

Parameters

txn [MDBX_txn](#)*

Транзакция.

Returns

[MdbxTxnFlags](#)

Флаги транзакции.

MdbxTxnGetUserctx(MDBX_txn*)

Получает пользовательский контекст транзакции.

```
public static extern void* MdbxTxnGetUserctx(MDBX_txn* txn)
```

Parameters

txn [MDBX_txn](#)*

Транзакция.

Returns

[void](#)^{*}

Указатель на пользовательский контекст.

MdbxTxnId(MDBX_txn*)

Получает ID транзакции.

```
public static extern ulong MdbxTxnId(MDBX_txn* txn)
```

Parameters

txn [MDBX_txn*](#)

Транзакция.

Returns

[ulong](#)[↗]

ID транзакции.

MdbxTxnInfo(MDBX_txn*, MdbxTxnInfo*, bool)

Получает информацию о транзакции.

```
public static extern int MdbxTxnInfo(MDBX_txn* txn, MdbxTxnInfo* info, bool scanRlt)
```

Parameters

txn [MDBX_txn*](#)

Транзакция.

info [MdbxTxnInfo*](#)

Структура для информации.

scanRlt [bool](#)[↗]

Сканировать читателей.

Returns

[int](#)[↗]

Код ошибки (0 при успехе).

MdbxTxnPark(MDBX_txn*, bool)

Паркует транзакцию.

```
public static extern int MdbxTxnPark(MDBX_txn* txn, bool autounpark)
```

Parameters

txn [MDBX_txn](#)*

Транзакция.

autounpark [bool](#)

Автоматически unpark при доступе.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxTxnRefresh(MDBX_txn*)

Обновляет (refresh) транзакцию.

```
public static extern int MdbxTxnRefresh(MDBX_txn* txn)
```

Parameters

txn [MDBX_txn](#)*

Транзакция.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxTxnReleaseAllCursorsEx(MDBX_txn*, bool, nuint*)

Освобождает все курсоры транзакции.

```
public static extern int MdbxTxnReleaseAllCursorsEx(MDBX_txn* txn, bool unbind,
nuint* count)
```

Parameters

txn [MDBX_txn](#)*

Транзакция.

unbind [bool](#)

Отвязать курсоры.

count [nuint](#)*

Количество освобожденных курсоров.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxTxnRenew(MDBX_txn*)

Обновляет (renew) транзакцию.

```
public static extern int MdbxTxnRenew(MDBX_txn* txn)
```

Parameters

txn [MDBX_txn](#)*

Транзакция.

Returns

[int](#)

Код ошибки (0 при успехе).

MdbxTxnReset(MDBX_txn*)

Сбрасывает транзакцию (освобождает ресурсы, но не откатывает).

```
public static extern int MdbxTxnReset(MDBX_txn* txn)
```

Parameters

txn [MDBX_txn*](#)

Транзакция.

Returns

[int](#)[↗]

Код ошибки (0 при успехе).

MdbxTxnRollback(MDBX_txn*)

Откатывает транзакцию.

```
public static extern int MdbxTxnRollback(MDBX_txn* txn)
```

Parameters

txn [MDBX_txn*](#)

Транзакция.

Returns

[int](#)[↗]

Код ошибки (0 при успехе).

MdbxTxnSetUserctx(MDBX_txn*, void*)

Устанавливает пользовательский контекст транзакции.


```
public static extern int MdbxTxnSetUserctx(MDBX_txn* txn, void* ctx)
```

Parameters

txn [MDBX_txn](#)*

Транзакция.

ctx [void](#)*^{*}

Пользовательский контекст.

Returns

[int](#)^{*}

Код ошибки (0 при успехе).

MdbxTxnStraggler(MDBX_txn*, int*)

Получает информацию о "straggler" транзакции.

```
public static extern int MdbxTxnStraggler(MDBX_txn* txn, int* percent)
```

Parameters

txn [MDBX_txn](#)*

Транзакция.

percent [int](#)^{*}

Процент.

Returns

[int](#)^{*}

Код ошибки (0 при успехе).

MdbxTxnUnpark(MDBX_txn*, bool)

Отменяет парковку транзакции.

```
public static extern int MdbxTxnUnpark(MDBX_txn* txn, bool restartIfOusted)
```

Parameters

txn [MDBX_txn*](#)

Транзакция.

restartIfOusted [bool](#) 

Перезапустить если вытеснен.

Returns

[int](#) 

Код ошибки (0 при успехе).

Namespace MDBX.Native.Types

Classes

[MdbxChkGc](#)

Псевдоним для GarbageCollectorDB: ((void*)((ptrdiff_t)-1)).

[MdbxChkMain](#)

Псевдоним для MainDB: ((void*)((ptrdiff_t)0)).

[MdbxChkMeta](#)

Псевдоним для MetaPages: ((void*)((ptrdiff_t)-2)).

Structs

[MDBX_cursor](#)

Непрозрачная структура курсора MDBX.

[MDBX_defrag_result](#)

Структура с результатами и прогрессом дефрагментации.

[MDBX_env](#)

Непрозрачная структура окружения MDBX.

[MDBX_envinfo](#)

Информация об окружении MDBX.

[MDBX_envinfo.MDBX_bootid](#)

Идентификатор загрузки.

[MDBX_envinfo.MDBX_bootid_pair](#)

Пара идентификаторов.

[MDBX_envinfo.MDBX_dxbid](#)

Идентификатор файла БД.

[MDBX_envinfo.MDBX_geo](#)

Геометрия файлов.

[MDBX_envinfo.MDBX_pgop_stat](#)

Статистика операций со страницами.

[MDBX_txn](#)

Непрозрачная структура транзакции MDBX.

[MdbxCacheEntry](#)

Легкая структура кэш-записи, используемая функциями `mdbx_cache_get()` и `mdbx_cache_get_SingleThreaded()`.

[MdbxCacheResult](#)

Пара кода ошибки и статуса кэша, возвращаемая функциями `mdbx_cache_get()` и `mdbx_cache_get_SingleThreaded()`.

[MdbxCanary](#)

Структура-маяк (canary) для отслеживания изменений окружения.

[MdbxChkCallbacks](#)

Набор функций обратного вызова, используемых для проверки целостности базы данных.

[MdbxChkContext](#)

Контекст проверки целостности базы данных.

[MdbxChkHistogram](#)

Гистограмма для проверки целостности базы данных.

[MdbxChkHistogram.MdbxChkHistogramRange](#)

Диапазон гистограммы.

[MdbxChkHistogramData](#)

Гистограмма для проверки целостности базы данных (управляемая версия для использования вне unsafe контекста). Осуществляет сопоставление с нативной структурой `MDBX_chk_histogram`. Использует фиксированный буфер для массива диапазонов, что позволяет использовать указатели на структуры, содержащие эту гистограмму.

[MdbxChkIssue](#)

Проблема, обнаруженная во время проверки целостности базы данных.

[MdbxChkLine](#)

Виртуальная строка отчета, генерируемого во время проверки целостности базы данных.

[MdbxChkResultData](#)

Данные результата проверки.

[MdbxChkScope](#)

Иерархический контекст во время проверки целостности базы данных.

[MdbxChkTable](#)

Информация о таблице ключ-значение во время проверки целостности базы данных.

[MdbxChkTableHistograms](#)

Гистограммы таблицы.

[MdbxChkTablePages](#)

Статистика страниц таблицы.

[MdbxChkUserTableCookie](#)

Неизменяемый тип для связывания дополнительных данных, связанных с определенной таблицей ключ-значение во время проверки целостности базы данных.

[MdbxCommitLatency](#)

Информация о задержках при фиксации транзакции.

[MdbxCommitLatency.MdbxGcProf](#)

Статистика профилирования GC.

[MdbxCommitLatency.MdbxPnlMerge](#)

Метрики слияния списков страниц.

[MdbxGcInfo](#)

Информация о сборке мусора (GC) и использовании страниц.

[MdbxGcInfo.MdbxGcReclaimable](#)

Информация о возвращаемых страницах.

[MdbxStat](#)

Статистика окружения MDBX.

[MdbxTxnInfo](#)

Информация о транзакции MDBX.

[MdbxVal](#)

Структура для передачи ключей и данных в таблицах MDBX.

Enums

[MdbxBunchAction](#)

Действие для пакетного удаления курсором.

[MdbxCacheStatus](#)

Статус кэша, возвращаемый функциями `mdbx_cache_get()` и `mdbx_cache_get_SingleThreaded()`.

[MdbxChkFlags](#)

Флаги проверки целостности базы данных.

[MdbxChkSeverity](#)

Уровни логирования и детализации информации, передаваемые через функции обратного вызова во время проверки целостности базы данных.

[MdbxChkStage](#)

Этапы проверки, сообщаемые через функции обратного вызова во время проверки целостности базы данных.

[MdbxCopyFlags](#)

Флаги копирования окружения.

[MdbxCursorOp](#)

Операции курсора.

[MdbxDbFlags](#)

Флаги базы данных (таблицы).

[MdbxDebugFlags](#)

Флаги отладки.

[MdbxDefragStoppingReasons](#)

Причины остановки дефрагментации базы данных.

[MdbxEnvDeleteMode](#)

Режим удаления окружения.

[MdbxEnvFlags](#)

Флаги окружения.

[MdbxLogLevel](#)

Уровни логирования.

[MdbxOption](#)

Параметры окружения.

[MdbxPutFlags](#)

Флаги операции вставки/обновления.

[MdbxTxnFlags](#)

Флаги транзакции.

[MdbxWarmupFlags](#)

Флаги прогрева окружения.

Delegates

[MdbxCmpFunc](#)

Функция сравнения ключей.

[MdbxDebugFunc](#)

Функция отладки (логгирования).

[MdbxDebugFuncNofmt](#)

Функция отладки без формата.

[MdbxDefragProgressFunc](#)

Функция уведомления о прогрессе дефрагментации.

[MdbxGclterFunc](#)

Функция итерации GC.

[MdbxHsrFunc](#)

Функция обработки медленных читателей (HSR - Handle Slow Readers).

[MdbxPanicFunc](#)

Функция паники (обработчик критических ошибок).

[MdbxPredicateFunc](#)

Функция предиката для сканирования курсора.

[MdbxPreserveFunc](#)

Функция сохранения данных для `mdbx_replace_ex`.

[MdbxReaderListFunc](#)

Функция перечисления читателей.

[MdbxTableEnumFunc](#)

Функция перечисления таблиц.

Struct MDBX_cursor

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Непрозрачная структура курсора MDBX.

```
public struct MDBX_cursor
```

Inherited Members

[ValueType.Equals\(object\)](#) , [ValueType.GetHashCode\(\)](#) , [ValueType.ToString\(\)](#) ,
[object.Equals\(object, object\)](#) , [object.GetType\(\)](#) , [object.ReferenceEquals\(object, object\)](#)

Struct MDBX_defrag_result

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Структура с результатами и прогрессом дефрагментации.

```
public struct MDBX_defrag_result
```

Inherited Members

[ValueType.Equals\(object\)](#) , [ValueType.GetHashCode\(\)](#) , [ValueType.ToString\(\)](#) ,
[object.Equals\(object, object\)](#) , [object.GetType\(\)](#) , [object.ReferenceEquals\(object, object\)](#)

Fields

cycles

Количество выполненных циклов дефрагментации.

```
public uint cycles
```

Field Value

[uint](#)

obstructed_pgno

Номер страницы, на которой остановилась дефрагментация.

```
public nuint obstructed_pgno
```

Field Value

[nuint](#)

obstructed_span

Длина спана large/overflow-страницы, на котором остановилась дефрагментация.

```
public nuint obstructed_span
```

Field Value

[nuint](#)

obstructed_txnid

ID транзакции самого раннего MVCC-снимка, блокирующего дефрагментацию.

```
public ulong obstructed_txnid
```

Field Value

[ulong](#)

obstructor_pid

ID процесса читателя, блокирующего дефрагментацию.

```
public uint obstructor_pid
```

Field Value

[uint](#)

obstructor_tid

ID потока читателя, блокирующего дефрагментацию.

```
public uint obstructor_tid
```

Field Value

[uint](#)

pages_left

Оценочное количество страниц, которые потенциально могут быть дефрагментированы.

```
public uint pages_left
```

Field Value

[nuint](#)

pages_moved

Общее количество перемещенных страниц.

```
public uint pages_moved
```

Field Value

[nuint](#)

pages_retained

Количество страниц, удерживаемых другими процессами через MVCC-снимки.

```
public uint pages_retained
```

Field Value

[nuint](#)

pages_scheduled

Количество страниц, запланированных к перемещению на следующем этапе.

```
public nuint pages_scheduled
```

Field Value

[nuint](#)

pages_shrunked

Количество страниц, на которое уменьшился размер базы данных.

```
public nint pages_shrunked
```

Field Value

[nint](#)

pages_whole

Общее количество страниц в базе данных.

```
public nuint pages_whole
```

Field Value

[nuint](#)

rough_estimation_cycle_progress_permille

Примерный прогресс текущего цикла дефрагментации в промилле (1000 = 100%).

```
public uint rough_estimation_cycle_progress_permille
```

Field Value

[uint](#)

spent_time_dot16

Затраченное время с начала дефрагментации в 1/65536 секунды.

```
public uint spent_time_dot16
```

Field Value

[uint](#)

stopping_reasons

Причины остановки дефрагментации (маска флагов).

```
public uint stopping_reasons
```

Field Value

[uint](#)

Struct MDBX_env

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Непрозрачная структура окружения MDBX.

```
public struct MDBX_env
```

Inherited Members

[ValueType.Equals\(object\)](#) , [ValueType.GetHashCode\(\)](#) , [ValueType.ToString\(\)](#) ,
[object.Equals\(object, object\)](#) , [object.GetType\(\)](#) , [object.ReferenceEquals\(object, object\)](#)

Struct MDBX_envinfo

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Информация об окружении MDBX.

```
public struct MDBX_envinfo
```

Inherited Members

[ValueType.Equals\(object\)](#), [ValueType.GetHashCode\(\)](#), [ValueType.ToString\(\)](#),
[object.Equals\(object, object\)](#), [object.GetType\(\)](#), [object.ReferenceEquals\(object, object\)](#)

Fields

MiAutosyncPeriodSeconds16Dot16

Период автосинхронизации (1/65536 сек).

```
public uint MiAutosyncPeriodSeconds16Dot16
```

Field Value

[uint](#)

MiAutosyncThreshold

Порог автосинхронизации.

```
public ulong MiAutosyncThreshold
```

Field Value

[ulong](#)

MiBootid

Идентификатор загрузки.

```
public MDBX_envinfo.MDBX_bootid MiBootid
```

Field Value

[MDBX_envinfo.MDBX_bootid](#)

MiDxbFallocated

Выделенный размер файла БД.

```
public ulong MiDxbFallocated
```

Field Value

[ulong](#)

MiDxbFsize

Текущий размер файла БД.

```
public ulong MiDxbFsize
```

Field Value

[ulong](#)

MiDxbPagesize

Размер страницы БД.

```
public uint MiDxbPagesize
```


Field Value

[uint](#)

MiDxbid

Идентификатор файла БД.

```
public MDBX_envinfo.MDBX_dxbid MiDxbid
```

Field Value

[MDBX_envinfo.MDBX_dxbid](#)

MiGeo

Геометрия файлов.

```
public MDBX_envinfo.MDBX_geo MiGeo
```

Field Value

[MDBX_envinfo.MDBX_geo](#)

MiLastPgno

Номер последней использованной страницы.

```
public ulong MiLastPgno
```

Field Value

[ulong](#)

MiLatterReaderTxnid

ID последней reader транзакции.

```
public ulong MiLatterReaderTxnid
```

Field Value

[ulong](#)↗

MiMapsize

Размер memory map.

```
public ulong MiMapsize
```

Field Value

[ulong](#)↗

MiMaxreaders

Максимальное количество reader слотов.

```
public uint MiMaxreaders
```

Field Value

[uint](#)↗

MiMetaSign

Сигнатуры мета-информации.

```
public ulong* MiMetaSign
```

Field Value

[ulong](#)^{*}

MiMetaTxnid

Мета-информация.

```
public ulong* MiMetaTxnid
```

Field Value

[ulong](#)^{*}

MiMode

Текущий режим окружения.

```
public uint MiMode
```

Field Value

[uint](#)

MiNumreaders

Количество используемых reader слотов.

```
public uint MiNumreaders
```

Field Value

[uint](#)

MiPgopStat

Статистика операций со страницами.

```
public MDBX_envinfo.MDBX_pgop_stat MiPgopStat
```

Field Value

[MDBX_envinfo.MDBX_pgop_stat](#)

MiRecentTxnid

ID последней зафиксированной транзакции.

```
public ulong MiRecentTxnid
```

Field Value

[ulong](#)↗

MiSelfLatterReaderTxnid

ID последней reader транзакции текущего процесса.

```
public ulong MiSelfLatterReaderTxnid
```

Field Value

[ulong](#)↗

MiSinceReaderCheckSeconds16Dot16

Время с последней проверки читателей (1/65536 сек).

```
public uint MiSinceReaderCheckSeconds16Dot16
```

Field Value

[uint](#)↗

MiSinceSyncSeconds16Dot16

Время с последней нестабильной фиксации (1/65536 сек).

```
public uint MiSinceSyncSeconds16Dot16
```

Field Value

[uint](#)

MiSysIoblk

Размер блока IO.

```
public uint MiSysIoblk
```

Field Value

[uint](#)

MiSysPagesize

Системный размер страницы.

```
public uint MiSysPagesize
```

Field Value

[uint](#)

MiSysUpcb1k

Размер блока Unified Page Cache.

```
public uint MiSysUpcb1k
```

Field Value

[uint](#) 

MiUnsyncVolume

Несинхронизированный объем.

```
public ulong MiUnsyncVolume
```

Field Value

[ulong](#) 

Struct MDBX_envinfo.MDBX_bootid

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Идентификатор загрузки.

```
public struct MDBX_envinfo.MDBX_bootid
```

Inherited Members

[ValueType.Equals\(object\)](#)[☞](#) , [ValueType.GetHashCode\(\)](#)[☞](#) , [ValueType.ToString\(\)](#)[☞](#) , [object.Equals\(object, object\)](#)[☞](#) , [object.GetType\(\)](#)[☞](#) , [object.ReferenceEquals\(object, object\)](#)[☞](#)

Fields

Current

Текущий идентификатор.

```
public MDBX_envinfo.MDBX_bootid_pair Current
```

Field Value

[MDBX_envinfo.MDBX_bootid_pair](#)

Meta

Мета идентификаторы.

```
public ulong* Meta
```

Field Value

[ulong](#)[☞](#)*

Struct MDBX_envinfo.MDBX_bootid_pair

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Пара идентификаторов.

```
public struct MDBX_envinfo.MDBX_bootid_pair
```

Inherited Members

[ValueType.Equals\(object\)](#)[☞](#) , [ValueType.GetHashCode\(\)](#)[☞](#) , [ValueType.ToString\(\)](#)[☞](#) , [object.Equals\(object, object\)](#)[☞](#) , [object.GetType\(\)](#)[☞](#) , [object.ReferenceEquals\(object, object\)](#)[☞](#)

Fields

X

Первый идентификатор.

```
public ulong X
```

Field Value

[ulong](#)[☞](#)

Y

Второй идентификатор.

```
public ulong Y
```

Field Value

[ulong](#)[☞](#)

Struct MDBX_envinfo.MDBX_dxbid

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Идентификатор файла БД.

```
public struct MDBX_envinfo.MDBX_dxbid
```

Inherited Members

[ValueType.Equals\(object\)](#)[↗](#) , [ValueType.GetHashCode\(\)](#)[↗](#) , [ValueType.ToString\(\)](#)[↗](#) ,
[object.Equals\(object, object\)](#)[↗](#) , [object.GetType\(\)](#)[↗](#) , [object.ReferenceEquals\(object, object\)](#)[↗](#)

Fields

X

Первый идентификатор.

```
public ulong X
```

Field Value

[ulong](#)[↗](#)

Y

Второй идентификатор.

```
public ulong Y
```

Field Value

[ulong](#)[↗](#)

Struct MDBX_envinfo.MDBX_geo

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Геометрия файлов.

```
public struct MDBX_envinfo.MDBX_geo
```

Inherited Members

[ValueType.Equals\(object\)](#)[↗](#) , [ValueType.GetHashCode\(\)](#)[↗](#) , [ValueType.ToString\(\)](#)[↗](#) ,
[object.Equals\(object, object\)](#)[↗](#) , [object.GetType\(\)](#)[↗](#) , [object.ReferenceEquals\(object, object\)](#)[↗](#)

Fields

Current

Текущий размер файла.

```
public ulong Current
```

Field Value

[ulong](#)[↗](#)

Grow

Шаг роста.

```
public ulong Grow
```

Field Value

[ulong](#)[↗](#)

Lower

Нижний предел размера файла.

```
public ulong Lower
```

Field Value

[ulong](#) 

Shrink

Порог сжатия.

```
public ulong Shrink
```

Field Value

[ulong](#) 

Upper

Верхний предел размера файла.

```
public ulong Upper
```

Field Value

[ulong](#) 

Struct MDBX_envinfo.MDBX_pgop_stat

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Статистика операций со страницами.

```
public struct MDBX_envinfo.MDBX_pgop_stat
```

Inherited Members

[ValueType.Equals\(object\)](#)[☞](#) , [ValueType.GetHashCode\(\)](#)[☞](#) , [ValueType.ToString\(\)](#)[☞](#) , [object.Equals\(object, object\)](#)[☞](#) , [object.GetType\(\)](#)[☞](#) , [object.ReferenceEquals\(object, object\)](#)[☞](#)

Fields

Clone

Количество клонированных страниц.

```
public ulong Clone
```

Field Value

[ulong](#)[☞](#)

Cow

Количество скопированных страниц.

```
public ulong Cow
```

Field Value

[ulong](#)[☞](#)

Fsync

Количество операций fsync.

`public ulong Fsync`

Field Value

[ulong](#) 

Merge

Количество слияний страниц.

`public ulong Merge`

Field Value

[ulong](#) 

Mincore

Количество вызовов mincore.

`public ulong Mincore`

Field Value

[ulong](#) 

Msync

Количество операций msync.

`public ulong Msync`

Field Value

[ulong](#) 

Newly

Количество новых страниц.

```
public ulong Newly
```

Field Value

[ulong](#) 

Prefault

Количество prefault операций.

```
public ulong Prefault
```

Field Value

[ulong](#) 

Spill

Количество spill операций.

```
public ulong Spill
```

Field Value

[ulong](#) 

Split

Количество разделений страниц.

```
public ulong Split
```

Field Value

[ulong](#) 

Unspill

Количество unspill операций.

```
public ulong Unspill
```

Field Value

[ulong](#) 

Wops

Количество операций записи.

```
public ulong Wops
```

Field Value

[ulong](#) 

Struct MDBX_txn

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Непрозрачная структура транзакции MDBX.

```
public struct MDBX_txn
```

Inherited Members

[ValueType.Equals\(object\)](#) , [ValueType.GetHashCode\(\)](#) , [ValueType.ToString\(\)](#) ,
[object.Equals\(object, object\)](#) , [object.GetType\(\)](#) , [object.ReferenceEquals\(object, object\)](#)

Enum MdbxBunchAction

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Действие для пакетного удаления курсором.

```
public enum MdbxBunchAction
```

Fields

MDBX_DELETE_AFTER_EXCLUDING = 9

Удалить все после текущего (исключая текущее).

MDBX_DELETE_AFTER_INCLUDING = 8

Удалить все после текущего (включая текущее).

MDBX_DELETE_BEFORE_EXCLUDING = 6

Удалить все до текущего (исключая текущее).

MDBX_DELETE_BEFORE_INCLUDING = 7

Удалить все до текущего (включая текущее).

MDBX_DELETE_CURRENT_MULTIVAL_AFTER_EXCLUDING = 4

Удалить все мультимножества после текущего (исключая текущее).

MDBX_DELETE_CURRENT_MULTIVAL_AFTER_INCLUDING = 3

Удалить все мультимножества после текущего (включая текущее).

MDBX_DELETE_CURRENT_MULTIVAL_ALL = 5

Удалить все мультимножества для текущего ключа.

MDBX_DELETE_CURRENT_MULTIVAL_BEFORE_EXCLUDING = 1

Удалить все мультимножества до текущего (исключая текущее).

MDBX_DELETE_CURRENT_MULTIVAL_BEFORE_INCLUDING = 2

Удалить все мультимножества до текущего (включая текущее).

`MDBX_DELETE_CURRENT_VALUE = 0`

Удалить текущее значение.

`MDBX_DELETE_WHOLE = 10`

Удалить все.

Struct MdbxCacheEntry

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Легкая структура кэш-записи, используемая функциями `mdbx_cache_get()` и `mdbx_cache_get_SingleThreaded()`.

```
public struct MdbxCacheEntry
```

Inherited Members

[ValueType.Equals\(object\)](#)[↗] , [ValueType.GetHashCode\(\)](#)[↗] , [ValueType.ToString\(\)](#)[↗] ,
[object.Equals\(object, object\)](#)[↗] , [object.GetType\(\)](#)[↗] , [object.ReferenceEquals\(object, object\)](#)[↗]

Remarks

Подход к кэшированию заключается в сохранении смещения значения данных, полученного из базы данных, с экстремально быстрой проверкой его актуальности на основе идентификатора транзакции внутри внутренних B-деревьевых структур. Кэш-запись может быть размещена в разделяемой памяти и использоваться несколькими процессами. Каждая кэш-запись должна быть инициализирована функцией `mdbx_cache_init()` перед первым использованием.

Fields

LastConfirmedTxnid

Идентификатор недавней транзакции/MVCC-снимка, в котором кэш-запись была проверена и подтверждена.

```
public ulong LastConfirmedTxnid
```

Field Value

[ulong](#)[↗]

Length

Длина кэшируемого значения данных для соответствующего ключа.

```
public uint Length
```

Field Value

[uint](#)

Offset

Смещение кэшируемого значения данных для соответствующего ключа. Нулевое значение означает MDBX_NOTFOUND.

```
public nuint Offset
```

Field Value

[nuint](#)

TrunkTxnid

Идентификатор транзакции/MVCC-снимка страницы или другой внутренней структуры БД, содержащей кэшируемые данные или отражающей их состояние.

```
public ulong TrunkTxnid
```

Field Value

[ulong](#)

Struct MdbxCacheResult

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Пара кода ошибки и статуса кэша, возвращаемая функциями `mdbx_cache_get()` и `mdbx_cache_get_SingleThreaded()`.

```
public struct MdbxCacheResult
```

Inherited Members

[ValueType.Equals\(object\)](#)[↗](#) , [ValueType.GetHashCode\(\)](#)[↗](#) , [ValueType.ToString\(\)](#)[↗](#) , [object.Equals\(object, object\)](#)[↗](#) , [object.GetType\(\)](#)[↗](#) , [object.ReferenceEquals\(object, object\)](#)[↗](#)

Remarks

Соответствует структуре `MDBX_cache_result_t` из заголовочного файла `mdbx.h`. Содержит код ошибки получения данных (аналогично `mdbx_get()`) и статус кэш-операции.

Fields

Errcode

Код ошибки получения данных, аналогичный возвращаемому функцией `mdbx_get()`.

```
public int Errcode
```

Field Value

[int](#)[↗](#)

Status

Статус кэш-операции как значение перечисления `MDBX_cache_status_t`.

```
public MdbxCacheStatus Status
```

Field Value

[MdbxCacheStatus](#)

Enum MdbxCacheStatus

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Статус кэша, возвращаемый функциями `mdbx_cache_get()` и `mdbx_cache_get_SingleThreaded()`.

```
public enum MdbxCacheStatus
```

Fields

MDBX_CACHE_BEHIND = -2

Результат получен в обход кэша, так как транзакция слишком старая. Кэш-запись отражает более новую версию данных, недоступную в текущем MVCC-снимке. Кэш-запись не изменена. Результат корректен до конца транзакции.

MDBX_CACHE_CONFIRMED = 3

Результат корректен и получен из кэш-записи, которая была немного обновлена для отражения актуальности данных для недавних зафиксированных транзакций. Кэш-запись слегка обновлена. Для read-транзакции результат корректен до конца транзакции. Для write-транзакций результат корректен до явного изменения значения или завершения транзакции.

MDBX_CACHE_DIRTY = 1

Результат корректен, но не может быть закэширован, так как изменения ещё не зафиксированы. Кэш-запись не изменена, так как изменения данных ещё не зафиксированы и могут быть отменены. Результат корректен до следующего изменения внутри текущей write-транзакции.

MDBX_CACHE_ERROR = -3

Произошла ошибка, не связанная с отсутствием ключа в кэше. Кэш-запись не изменена.

MDBX_CACHE_HIT = 2

Результат корректен и получен из кэш-записи, которая не была изменена. Кэш-запись не изменена, так как данные не изменились с момента последней проверки. Для read-транзакции результат корректен до конца транзакции. Для write-транзакций результат корректен до явного изменения значения или завершения транзакции.

MDBX_CACHE_RACE = 0

Результат получен в обход кэша из-за обнаружения гонки при обновлении кэш-записи другим потоком. Кэш-запись не изменена. Для read-транзакции результат корректен до конца транзакции. Для write-транзакций результат корректен до явного изменения значения или завершения транзакции.

MDBX_CACHE_REFRESHED = 4

Результат корректен и соответствует свежим данным, прочитанным из базы данных, которые также были помещены в кэш-запись. Кэш-запись полностью обновлена для отражения актуальных данных. Для read-транзакции результат корректен до конца транзакции. Для write-транзакций результат корректен до явного изменения значения или завершения транзакции.

MDBX_CACHE_UNABLE = -1

Результат корректен, но не может быть закэширован из-за ABA-подобной проблемы в истории данных. Кэш-запись не изменена. Результат корректен до конца транзакции.

Remarks

Значения соответствуют перечислению `MDBX_cache_status_t` из заголовочного файла `mdbx.h`. Отрицательные значения означают ошибки или специальные случаи, положительные — успех.

Struct MdbxCanary

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Структура-маяк (canary) для отслеживания изменений окружения.

```
public struct MdbxCanary
```

Inherited Members

[ValueType.Equals\(object\)](#) , [ValueType.GetHashCode\(\)](#) , [ValueType.ToString\(\)](#) ,
[object.Equals\(object, object\)](#) , [object.GetType\(\)](#) , [object.ReferenceEquals\(object, object\)](#)

Fields

V

Четвертое значение маркера (ID транзакции).

```
public ulong V
```

Field Value

[ulong](#)

X

Первое значение маркера.

```
public ulong X
```

Field Value

[ulong](#)

Y

Второе значение маркера.

```
public ulong Y
```

Field Value

[ulong](#) 

Z

Третье значение маркера.

```
public ulong Z
```

Field Value

[ulong](#) 

Struct MdbxChkCallbacks

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Набор функций обратного вызова, используемых для проверки целостности базы данных.

```
public struct MdbxChkCallbacks
```

Inherited Members

[ValueType.Equals\(object\)](#) , [ValueType.GetHashCode\(\)](#) , [ValueType.ToString\(\)](#) ,
[object.Equals\(object, object\)](#) , [object.GetType\(\)](#) , [object.ReferenceEquals\(object, object\)](#)

Remarks

Соответствует структуре `MDBX_chk_callbacks_t` из заголовочного файла `mdbx.h`. Все функции обратного вызова опциональны, неиспользуемые должны быть `nullptr`. Содержит 16 полей-указателей на функции. 3 функции используют `va_list`, поэтому для них используется `void*`.

Fields

CheckBreak

Функция проверки прерывания.

```
public delegate* unmanaged[Cdecl]<MdbxChkContext*, bool> CheckBreak
```

Field Value

`delegate* unmanaged[Cdecl]<MdbxChkContext*, bool>`

Issue

Функция обработки проблемы (использует `va_list`, поэтому `void*`).

```
public void* Issue
```

Field Value

[void](#)^{*}

PrintBegin

Начало печати.

```
public delegate* unmanaged[Cdecl]<MdbxChkContext*, MdbxChkSeverity, MdbxChkLine*> PrintBegin
```

Field Value

delegate* unmanaged[Cdecl]<[MdbxChkContext](#)*, [MdbxChkSeverity](#), [MdbxChkLine](#)*>

PrintChars

Печать СИМВОЛОВ.

```
public delegate* unmanaged[Cdecl]<MdbxChkLine*, byte*, nuint, void> PrintChars
```

Field Value

delegate* unmanaged[Cdecl]<[MdbxChkLine](#)*, [byte](#)^{*}, [nuint](#)^{*}, [void](#)^{*}>

PrintDone

Завершение печати.

```
public delegate* unmanaged[Cdecl]<MdbxChkLine*, void> PrintDone
```

Field Value

delegate* unmanaged[Cdecl]<[MdbxChkLine](#)*, [void](#)^{*}>

PrintFlush

Сброс печати.

```
public delegate* unmanaged[Cdecl]<MdbxChkLine*, void> PrintFlush
```

Field Value

delegate* unmanaged[Cdecl]<[MdbxChkLine](#)*, [void](#)>

PrintFormat

Печать форматированной строки (использует va_list, поэтому void*).

```
public void* PrintFormat
```

Field Value

[void](#)*

PrintSize

Печать размера.

```
public delegate* unmanaged[Cdecl]<MdbxChkLine*, byte*, ulong, byte*, void> PrintSize
```

Field Value

delegate* unmanaged[Cdecl]<[MdbxChkLine](#)*, [byte](#)*, [ulong](#), [byte](#)*, [void](#)>

ScopeConclude

Функция завершения области.

```
public delegate* unmanaged[Cdecl]<MdbxChkContext*, MdbxChkScope*, MdbxChkScope*,  
int> ScopeConclude
```

Field Value

delegate* unmanaged[Cdecl]<[MdbxChkContext*](#), [MdbxChkScope*](#), [MdbxChkScope*](#), [int](#)>

ScopePop

Функция выхода из области.

```
public delegate* unmanaged[Cdecl]<MdbxChkContext*, MdbxChkScope*, MdbxChkScope*,  
void> ScopePop
```

Field Value

delegate* unmanaged[Cdecl]<[MdbxChkContext*](#), [MdbxChkScope*](#), [MdbxChkScope*](#), [void](#)>

ScopePush

Функция входа в область (использует va_list, поэтому void*).

```
public void* ScopePush
```

Field Value

[void](#)*

StageBegin

Начало этапа.

```
public delegate* unmanaged[Cdecl]<MdbxChkContext*, MdbxChkStage> StageBegin
```

Field Value

delegate* unmanaged[Cdecl]<[MdbxChkContext*](#), [MdbxChkStage](#)>

StageEnd

Конец этапа.

```
public delegate* unmanaged[Cdecl]<MdbxChkContext*, MdbxChkStage, int> StageEnd
```

Field Value

delegate* unmanaged[Cdecl]<[MdbxChkContext*](#), [MdbxChkStage](#), [int](#)>

TableConclude

Завершение таблицы.

```
public delegate* unmanaged[Cdecl]<MdbxChkContext*, MdbxChkTable*, MDBX_cursor*,  
int> TableConclude
```

Field Value

delegate* unmanaged[Cdecl]<[MdbxChkContext*](#), [MdbxChkTable*](#), [MDBX_cursor*](#), [int](#)>

TableDispose

Освобождение таблицы.

```
public delegate* unmanaged[Cdecl]<MdbxChkContext*, MdbxChkTable*, void> TableDispose
```

Field Value

delegate* unmanaged[Cdecl]<[MdbxChkContext*](#), [MdbxChkTable*](#), [void](#)>

TableFilter

Фильтр таблицы.

```
public delegate* unmanaged[Cdecl]<MdbxChkContext*, MdbxVal*, uint,
```

MdbxChkUserTableCookie*> TableFilter

Field Value

delegate* unmanaged[Cdecl]<[MdbxChkContext*](#), [MdbxVal*](#), [uint](#)[↗], [MdbxChkUserTableCookie*](#)>

TableHandleKv

Обработка пары ключ-значение.

```
public delegate* unmanaged[Cdecl]<MdbxChkContext*, MdbxChkTable*, nuint, MdbxVal*, MdbxVal*,  
int> TableHandleKv
```

Field Value

delegate* unmanaged[Cdecl]<[MdbxChkContext*](#), [MdbxChkTable*](#), [nuint](#)[↗], [MdbxVal*](#), [MdbxVal*](#), [int](#)[↗]>

Struct MdbxChkContext

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Контекст проверки целостности базы данных.

```
public struct MdbxChkContext
```

Inherited Members

[ValueType.Equals\(object\)](#)[☞](#) , [ValueType.GetHashCode\(\)](#)[☞](#) , [ValueType.ToString\(\)](#)[☞](#) , [object.Equals\(object, object\)](#)[☞](#) , [object.GetType\(\)](#)[☞](#) , [object.ReferenceEquals\(object, object\)](#)[☞](#)

Remarks

Соответствует структуре MDBX_chk_context_t из заголовочного файла mdbx.h.

Fields

Env

Окружение.

```
public MDBX_env* Env
```

Field Value

[MDBX_env](#)*

Internal

Внутренние данные (непрозрачный указатель).

```
public void* Internal
```

Field Value

[void](#)^{*}

Result

Результат проверки.

```
public MdbxChkResultData Result
```

Field Value

[MdbxChkResultData](#)

Scope

Текущая область.

```
public MdbxChkScope* Scope
```

Field Value

[MdbxChkScope](#)^{*}

ScopeNesting

Глубина вложенности областей.

```
public byte ScopeNesting
```

Field Value

[byte](#)^{*}

Txn

Транзакция.

public MDBX_txn* Txn

Field Value

[MDBX_txn](#)*

Enum MdbxChkFlags

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Флаги проверки целостности базы данных.

[Flags]

```
public enum MdbxChkFlags
```

Fields

MDBX_CHK_DEFAULTS = 0

Режим проверки по умолчанию, включая read-only режим.

MDBX_CHK_IGNORE_ORDER = 8

Игнорировать порядок ключей и значений. Необходимо при проверке баз данных с нестандартными функциями сравнения.

MDBX_CHK_READWRITE = 1

Проверка в read-write режиме с блокировкой и приостановкой записных транзакций.

MDBX_CHK_SKIP_BTREE_TRAVERSAL = 2

Пропустить обход дерева страниц.

MDBX_CHK_SKIP_KV_TRAVERSAL = 4

Пропустить итерацию и просмотр записей ключ-значение.

Remarks

Значения соответствуют перечислению `MDBX_chk_flags_t` из заголовочного файла `mdbx.h`. Могут быть объединены с помощью битовых операций.

Class MdbxChkGc

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Псевдоним для GarbageCollectorDB: ((void*)((ptrdiff_t)-1)).

```
public static class MdbxChkGc
```

Inheritance

[object](#)  ← MdbxChkGc

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  ,
[object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Struct MdbxChkHistogram

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Гистограмма для проверки целостности базы данных.

```
public struct MdbxChkHistogram
```

Inherited Members

[ValueType.Equals\(object\)](#)[↗] , [ValueType.GetHashCode\(\)](#)[↗] , [ValueType.ToString\(\)](#)[↗] ,
[object.Equals\(object, object\)](#)[↗] , [object.GetType\(\)](#)[↗] , [object.ReferenceEquals\(object, object\)](#)[↗]

Fields

Amount

Общее количество элементов.

```
public uint Amount
```

Field Value

[uint](#)[↗]

Count

Количество интервалов.

```
public uint Count
```

Field Value

[uint](#)[↗]

Le1Amount

Количество элементов ≤ 1 .

```
public uint Le1Amount
```

Field Value

[uint](#)

Le1Count

Количество интервалов ≤ 1 .

```
public uint Le1Count
```

Field Value

[uint](#)

Ranges

Диапазоны гистограммы (9 штук).

```
public MdbxChkHistogram.MdbxChkHistogramRange[] Ranges
```

Field Value

[MdbxChkHistogramRange\[\]](#)

Struct MdbxChkHistogram.MdbxChkHistogramRange

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Диапазон гистограммы.

```
public struct MdbxChkHistogram.MdbxChkHistogramRange
```

Inherited Members

[ValueType.Equals\(object\)](#)[↗] , [ValueType.GetHashCode\(\)](#)[↗] , [ValueType.ToString\(\)](#)[↗] ,
[object.Equals\(object, object\)](#)[↗] , [object.GetType\(\)](#)[↗] , [object.ReferenceEquals\(object, object\)](#)[↗]

Fields

Amount

Количество элементов в диапазоне.

```
public uint Amount
```

Field Value

[uint](#)[↗]

Begin

Начало диапазона.

```
public uint Begin
```

Field Value

[uint](#)[↗]

Count

Количество интервалов в диапазоне.

```
public uint Count
```

Field Value

[uint](#)

End

Конец диапазона.

```
public uint End
```

Field Value

[uint](#)

Struct MdbxChkHistogramData

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Гистограмма для проверки целостности базы данных (управляемая версия для использования вне unsafe контекста). Осуществляет сопоставление с нативной структурой MDBX_chk_histogram. Использует фиксированный буфер для массива диапазонов, что позволяет использовать указатели на структуры, содержащие эту гистограмму.

```
public struct MdbxChkHistogramData
```

Inherited Members

[ValueType.Equals\(object\)](#)[↗](#) , [ValueType.GetHashCode\(\)](#)[↗](#) , [ValueType.ToString\(\)](#)[↗](#) , [object.Equals\(object, object\)](#)[↗](#) , [object.GetType\(\)](#)[↗](#) , [object.ReferenceEquals\(object, object\)](#)[↗](#)

Fields

Amount

Общее количество элементов.

```
public nuint Amount
```

Field Value

[nuint](#)[↗](#)

Count

Количество интервалов.

```
public nuint Count
```

Field Value

[nuint](#)

Le1Amount

Количество элементов ≤ 1 .

```
public nuint Le1Amount
```

Field Value

[nuint](#)

Le1Count

Количество интервалов ≤ 1 .

```
public nuint Le1Count
```

Field Value

[nuint](#)

RangeSize

Размер одного диапазона гистограммы в байтах ($4 * \text{sizeof}(\text{nuint}) = 32$ на x64).

```
public const int RangeSize = 32
```

Field Value

[int](#)

Ranges

Диапазоны гистограммы (9 штук), как фиксированный буфер байт.

public byte* Ranges

Field Value

[byte](#)^{*}

Struct MdbxChkIssue

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Проблема, обнаруженная во время проверки целостности базы данных.

```
public struct MdbxChkIssue
```

Inherited Members

[ValueType.Equals\(object\)](#) , [ValueType.GetHashCode\(\)](#) , [ValueType.ToString\(\)](#) ,
[object.Equals\(object, object\)](#) , [object.GetType\(\)](#) , [object.ReferenceEquals\(object, object\)](#)

Remarks

Соответствует структуре MDBX_chk_issue_t из заголовочного файла mdbx.h.

Fields

Caption

Заголовок проблемы.

```
public byte* Caption
```

Field Value

[byte](#)*

Count

Количество вхождений проблемы.

```
public nuint Count
```

Field Value

[nuint](#)

Next

Указатель на следующую проблему в списке.

```
public MdbxChkIssue* Next
```

Field Value

[MdbxChkIssue](#)*

Struct MdbxChkLine

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Виртуальная строка отчета, генерируемого во время проверки целостности базы данных.

```
public struct MdbxChkLine
```

Inherited Members

[ValueType.Equals\(object\)](#)[☞](#) , [ValueType.GetHashCode\(\)](#)[☞](#) , [ValueType.ToString\(\)](#)[☞](#) , [object.Equals\(object, object\)](#)[☞](#) , [object.GetType\(\)](#)[☞](#) , [object.ReferenceEquals\(object, object\)](#)[☞](#)

Remarks

Соответствует структуре MDBX_chk_line_t из заголовочного файла mdbx.h.

Fields

Begin

Указатель на начало строки вывода.

```
public byte* Begin
```

Field Value

[byte](#)[☞](#)*

Context

Указатель на контекст проверки.

```
public MdbxChkContext* Context
```

Field Value

[MdbxChkContext*](#)

Empty

Флаг пустой строки.

```
public byte Empty
```

Field Value

[byte](#)

End

Указатель на конец строки вывода.

```
public byte* End
```

Field Value

[byte](#)*

Out

Указатель на текущую позицию вывода.

```
public byte* Out
```

Field Value

[byte](#)*

ScopeDepth

Глубина вложенности области.

public byte ScopeDepth

Field Value

[byte](#)

Severity

Уровень серьезности вывода.

public byte Severity

Field Value

[byte](#)

Class MdbxChkMain

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Псевдоним для MainDB: ((void*)((ptrdiff_t)0)).

```
public static class MdbxChkMain
```

Inheritance

[object](#)  ← MdbxChkMain

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  ,
[object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Class MdbxChkMeta

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Псевдоним для MetaPages: ((void*)((ptrdiff_t)-2)).

```
public static class MdbxChkMeta
```

Inheritance

[object](#)  ← MdbxChkMeta

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  ,
[object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Struct MdbxChkResultData

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Данные результата проверки.

```
public struct MdbxChkResultData
```

Inherited Members

[ValueType.Equals\(object\)](#)[☞](#) , [ValueType.GetHashCode\(\)](#)[☞](#) , [ValueType.ToString\(\)](#)[☞](#) , [object.Equals\(object, object\)](#)[☞](#) , [object.GetType\(\)](#)[☞](#) , [object.ReferenceEquals\(object, object\)](#)[☞](#)

Fields

AllocPages

Обработанные страницы, возвращаемые страницы, страницы GC, выделенные страницы, сопоставленные страницы.

```
public uint AllocPages
```

Field Value

[uint](#)[☞](#)

BackedPages

Обработанные страницы, возвращаемые страницы, страницы GC, выделенные страницы, сопоставленные страницы.

```
public uint BackedPages
```

Field Value

[uint](#)[☞](#)

GcPages

Обработанные страницы, возвращаемые страницы, страницы GC, выделенные страницы, сопоставленные страницы.

```
public uint GcPages
```

Field Value

[uint](#)

GcTreeProblems

Проблемы: метаданные, дерево, GC, KV, общие.

```
public uint GcTreeProblems
```

Field Value

[uint](#)

HistogramPageAge

Гистограмма возраста страниц.

```
public MdbxChkHistogramData HistogramPageAge
```

Field Value

[MdbxChkHistogramData](#)

HistogramPgnoPayload

Гистограмма ргно удерживаемого читателями.

```
public MdbxChkHistogramData HistogramPgnoPayload
```

Field Value

[MdbxChkHistogramData](#)

HistogramPgnoRetained

Гистограмма ргно, используемого всей полезной нагрузкой.

```
public MdbxChkHistogramData HistogramPgnoRetained
```

Field Value

[MdbxChkHistogramData](#)

KvTreeProblems

Проблемы: метаданные, дерево, GC, KV, общие.

```
public nuint KvTreeProblems
```

Field Value

[nuint](#)

ProblemsGc

Проблемы: метаданные, дерево, GC, KV, общие.

```
public nuint ProblemsGc
```

Field Value

[nuint](#)

ProblemsKv

Проблемы: метаданные, дерево, GC, KV, общие.

`public uint ProblemsKv`

Field Value

[uint](#)

ProblemsMeta

Проблемы: метаданные, дерево, GC, KV, общие.

`public uint ProblemsMeta`

Field Value

[uint](#)

ProcessedPages

Обработанные страницы, возвращаемые страницы, страницы GC, выделенные страницы, сопоставленные страницы.

`public uint ProcessedPages`

Field Value

[uint](#)

RecentTxnid

ID транзакций.

`public ulong RecentTxnid`

Field Value

[ulong](#)

ReclaimablePages

Обработанные страницы, возвращаемые страницы, страницы GC, выделенные страницы, сопоставленные страницы.

```
public nuint ReclaimablePages
```

Field Value

[nuint](#)

SteadyTxnid

ID транзакций.

```
public ulong SteadyTxnid
```

Field Value

[ulong](#)

TableProcessed

Общее и обработанное количество таблиц.

```
public nuint TableProcessed
```

Field Value

[nuint](#)

TableTotal

Общее и обработанное количество таблиц.

```
public nuint TableTotal
```

Field Value

[nuint](#)

Tables

Указатель на массив указателей на структуры MDBX_chk_table_t.

```
public MdbxChkTable* Tables
```

Field Value

[MdbxChkTable](#)*

TotalPayloadBytes

Общий объем полезной нагрузки в байтах.

```
public nuint TotalPayloadBytes
```

Field Value

[nuint](#)

TotalProblems

Проблемы: метаданные, дерево, GC, KV, общие.

```
public nuint TotalProblems
```

Field Value

[nuint](#)

TotalUnusedBytes

Общее количество потерянных байт и страниц.

```
public nuint TotalUnusedBytes
```

Field Value

[nuint](#)

TreeProblems

Проблемы: метаданные, дерево, GC, KV, общие.

```
public nuint TreeProblems
```

Field Value

[nuint](#)

UnusedPages

Общее количество потерянных байт и страниц.

```
public nuint UnusedPages
```

Field Value

[nuint](#)

Struct MdbxChkScope

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Иерархический контекст во время проверки целостности базы данных.

```
public struct MdbxChkScope
```

Inherited Members

[ValueType.Equals\(object\)](#)[↗](#) , [ValueType.GetHashCode\(\)](#)[↗](#) , [ValueType.ToString\(\)](#)[↗](#) ,
[object.Equals\(object, object\)](#)[↗](#) , [object.GetType\(\)](#)[↗](#) , [object.ReferenceEquals\(object, object\)](#)[↗](#)

Remarks

Соответствует структуре MDBX_chk_scope_t из заголовочного файла mdbx.h.

Fields

Internal

Внутренние данные (непрозрачный указатель).

```
public void* Internal
```

Field Value

[void](#)[↗](#)*

Issues

Список проблем, обнаруженных в данной области.

```
public MdbxChkIssue* Issues
```

Field Value

[MdbxChkIssue*](#)

Object

Объект, связанный с этой областью.

```
public void* Object
```

Field Value

[void](#)[↗]*

Stage

Этап проверки.

```
public MdbxChkStage Stage
```

Field Value

[MdbxChkStage](#)

SubtotalIssues

Подсчет проблем в этой области.

```
public nuint SubtotalIssues
```

Field Value

[nuint](#)[↗]

UsrO

Пользовательские данные (union: ptr/number).

```
public nint Usr0
```

Field Value

[nint](#)

UsrV

Пользовательские данные (union: ptr/number).

```
public nint UsrV
```

Field Value

[nint](#)

UsrZ

Пользовательские данные (union: ptr/number).

```
public nint UsrZ
```

Field Value

[nint](#)

Verbosity

Уровень детализации (verbosity).

```
public MdbxChkSeverity Verbosity
```

Field Value

[MdbxChkSeverity](#)

Enum MdbxChkSeverity

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Уровни логирования и детализации информации, передаваемые через функции обратного вызова во время проверки целостности базы данных.

```
public enum MdbxChkSeverity : uint
```

Fields

`MDBX_chk_details = 137`

Детали.

`MDBX_chk_error = 17`

Ошибка.

`MDBX_chk_extra = 154`

Дополнительная детализация.

`MDBX_chk_fatal = 0`

Фатальная ошибка.

`MDBX_chk_info = 103`

Информационное сообщение.

`MDBX_chk_notice = 51`

Уведомление.

`MDBX_chk_processing = 86`

Обработка.

`MDBX_chk_resolution = 85`

Разрешение проблемы.

`MDBX_chk_result = 68`

Результат проверки.

`MDBX_chk_severity_kind_mask = 15`

Маска для извлечения категории severity.

`MDBX_chk_severity_prio_shift = 4`

Сдвиг приоритета для определения категории severity.

`MDBX_chk_verbose = 120`

Подробное логирование.

`MDBX_chk_warning = 34`

Предупреждение.

Remarks

Значения соответствуют перечислению `MDBX_chk_severity_t` из заголовочного файла `mdbx.h`.

Enum MdbxChkStage

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Этапы проверки, сообщаемые через функции обратного вызова во время проверки целостности базы данных.

```
public enum MdbxChkStage
```

Fields

`MDBX_chk_conclude = 9`

Завершение.

`MDBX_chk_finalize = 11`

Финализация.

`MDBX_chk_gc = 5`

Проверка сборщика мусора.

`MDBX_chk_init = 1`

Инициализация.

`MDBX_chk_lock = 2`

Блокировка.

`MDBX_chk_maindb = 7`

Проверка MainDB.

`MDBX_chk_meta = 3`

Проверка метаданных.

`MDBX_chk_none = 0`

Нет этапа.

`MDBX_chk_space = 6`

Проверка пространства.

`MDBX_chk_tables = 8`

Проверка таблиц.

`MDBX_chk_tree = 4`

Обход дерева.

`MDBX_chk_unlock = 10`

Снятие блокировки.

Remarks

Значения соответствуют перечислению `MDBX_chk_stage_t` из заголовочного файла `mdbx.h`.

Struct MdbxChkTable

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Информация о таблице ключ-значение во время проверки целостности базы данных.

```
public struct MdbxChkTable
```

Inherited Members

[ValueType.Equals\(object\)](#) , [ValueType.GetHashCode\(\)](#) , [ValueType.ToString\(\)](#) ,
[object.Equals\(object, object\)](#) , [object.GetType\(\)](#) , [object.ReferenceEquals\(object, object\)](#)

Remarks

Соответствует структуре MDBX_chk_table_t из заголовочного файла mdbx.h.

Fields

Cookie

cookie для связывания дополнительных данных.

```
public MdbxChkUserTableCookie* Cookie
```

Field Value

[MdbxChkUserTableCookie*](#)

Flags

Флаги таблицы.

```
public uint Flags
```

Field Value

[uint](#)

Histogram

Гистограммы.

```
public MdbxChkTableHistograms Histogram
```

Field Value

[MdbxChkTableHistograms](#)

Id

Идентификатор таблицы.

```
public int Id
```

Field Value

[int](#)

LostBytes

Потерянные байты.

```
public nuint LostBytes
```

Field Value

[nuint](#)

Name

Имя таблицы.

`public MdbxVal Name`

Field Value

[MdbxVal](#)

Pages

Статистика страниц.

`public MdbxChkTablePages Pages`

Field Value

[MdbxChkTablePages](#)

PayloadBytes

Объем полезной нагрузки в байтах.

`public nuint PayloadBytes`

Field Value

[nuint](#)

Struct MdbxChkTableHistograms

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Гистограммы таблицы.

```
public struct MdbxChkTableHistograms
```

Inherited Members

[ValueType.Equals\(object\)](#)[↗](#) , [ValueType.GetHashCode\(\)](#)[↗](#) , [ValueType.ToString\(\)](#)[↗](#) ,
[object.Equals\(object, object\)](#)[↗](#) , [object.GetType\(\)](#)[↗](#) , [object.ReferenceEquals\(object, object\)](#)[↗](#)

Fields

Height

Гистограмма высоты дерева.

```
public MdbxChkHistogramData Height
```

Field Value

[MdbxChkHistogramData](#)

KeyLen

Гистограмма длин ключей.

```
public MdbxChkHistogramData KeyLen
```

Field Value

[MdbxChkHistogramData](#)

LargeOrNestedDensity

Гистограмма заполнения вложенных деревьев ветвящих и листовых страниц в процентах.

```
public MdbxChkHistogramData LargeOrNestedDensity
```

Field Value

[MdbxChkHistogramData](#)

LargePages

Гистограмма длины больших/переполненных страниц.

```
public MdbxChkHistogramData LargePages
```

Field Value

[MdbxChkHistogramData](#)

Multival

Гистограмма количества многозначных записей.

```
public MdbxChkHistogramData Multival
```

Field Value

[MdbxChkHistogramData](#)

NestedHeightOrGcSpanLength

Гистограмма высоты вложенных деревьев или длины span для GC.

```
public MdbxChkHistogramData NestedHeightOrGcSpanLength
```

Field Value

[MdbxChkHistogramData](#)

PageAge

Гистограмма возраста страниц.

```
public MdbxChkHistogramData PageAge
```

Field Value

[MdbxChkHistogramData](#)

Pgno

Гистограмма используемых pgno.

```
public MdbxChkHistogramData Pgno
```

Field Value

[MdbxChkHistogramData](#)

TreeDensity

Гистограмма заполнения ветвящих и листовых страниц в процентах.

```
public MdbxChkHistogramData TreeDensity
```

Field Value

[MdbxChkHistogramData](#)

ValLen

Гистограмма длин значений.

```
public MdbxChkHistogramData ValLen
```

Field Value

[MdbxChkHistogramData](#)

Struct MdbxChkTablePages

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Статистика страниц таблицы.

```
public struct MdbxChkTablePages
```

Inherited Members

[ValueType.Equals\(object\)](#)[↗] , [ValueType.GetHashCode\(\)](#)[↗] , [ValueType.ToString\(\)](#)[↗] ,
[object.Equals\(object, object\)](#)[↗] , [object.GetType\(\)](#)[↗] , [object.ReferenceEquals\(object, object\)](#)[↗]

Fields

All

Общее количество страниц.

```
public uint All
```

Field Value

[uint](#)[↗]

Branch

Количество ветвящих страниц.

```
public uint Branch
```

Field Value

[uint](#)[↗]

Broken

Количество сломанных страниц.

```
public uint Broken
```

Field Value

[uint](#)

Empty

Количество пустых страниц.

```
public uint Empty
```

Field Value

[uint](#)

Leaf

Количество листовых страниц.

```
public uint Leaf
```

Field Value

[uint](#)

NestedBranch

Количество вложенных ветвящих страниц.

```
public uint NestedBranch
```

Field Value

[nuint](#)

NestedLeaf

Количество вложенных листовых страниц.

```
public nuint NestedLeaf
```

Field Value

[nuint](#)

NestedSubleaf

Количество вложенных сублистовых страниц.

```
public nuint NestedSubleaf
```

Field Value

[nuint](#)

Struct MdbxChkUserTableCookie

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Неизменяемый тип для связывания дополнительных данных, связанных с определенной таблицей ключ-значение во время проверки целостности базы данных.

```
public struct MdbxChkUserTableCookie
```

Inherited Members

[ValueType.Equals\(object\)](#)[↗] , [ValueType.GetHashCode\(\)](#)[↗] , [ValueType.ToString\(\)](#)[↗] ,
[object.Equals\(object, object\)](#)[↗] , [object.GetType\(\)](#)[↗] , [object.ReferenceEquals\(object, object\)](#)[↗]

Remarks

Соответствует типу MDBX_chk_user_table_cookie_t из заголовочного файла mdbx.h.

Delegate MdbxCmpFunc

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Функция сравнения ключей.

```
public delegate int MdbxCmpFunc(MdbxVal* a, MdbxVal* b)
```

Parameters

a [MdbxVal](#)*

Первый ключ.

b [MdbxVal](#)*

Второй ключ.

Returns

[int](#)[↗]

Отрицательное значение если $a < b$, 0 если $a == b$, положительное если $a > b$.

Struct MdbxCommitLatency

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Информация о задержках при фиксации транзакции.

```
public struct MdbxCommitLatency
```

Inherited Members

[ValueType.Equals\(object\)](#) , [ValueType.GetHashCode\(\)](#) , [ValueType.ToString\(\)](#) ,
[object.Equals\(object, object\)](#) , [object.GetType\(\)](#) , [object.ReferenceEquals\(object, object\)](#)

Fields

Audit

Длительность внутреннего аудита, если он включен.

```
public uint Audit
```

Field Value

[uint](#)

Ending

Длительность завершения транзакции (освобождение ресурсов).

```
public uint Ending
```

Field Value

[uint](#)

GcCputime

CPU-время в пользовательском режиме, затраченное на обновление GC.

```
public uint GcCputime
```

Field Value

[uint](#)

GcProf

Статистика профилирования GC.

```
public MdbxCommitLatency.MdbxGcProf GcProf
```

Field Value

[MdbxCommitLatency.MdbxGcProf](#)

GcWallclock

Длительность обновления GC по стене часов.

```
public uint GcWallclock
```

Field Value

[uint](#)

Preparation

Длительность подготовки (фиксация дочерних транзакций, обновление записей таблиц и уничтожение курсоров).

```
public uint Preparation
```

Field Value

[uint](#)

Sync

Длительность синхронизации записанных данных на диск/хранилище.

```
public uint Sync
```

Field Value

[uint](#)

Whole

Общая длительность фиксации транзакции.

```
public uint Whole
```

Field Value

[uint](#)

Write

Длительность записи грязных/измененных страниц данных в файловую систему.

```
public uint Write
```

Field Value

[uint](#)

Struct MdbxCommitLatency.MdbxGcProf

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Статистика профилирования GC.

```
public struct MdbxCommitLatency.MdbxGcProf
```

Inherited Members

[ValueType.Equals\(object\)](#) , [ValueType.GetHashCode\(\)](#) , [ValueType.ToString\(\)](#) ,
[object.Equals\(object, object\)](#) , [object.GetType\(\)](#) , [object.ReferenceEquals\(object, object\)](#)

Fields

Coalescences

Количество итераций слияния элементов GC.

```
public uint Coalescences
```

Field Value

[uint](#)

Flushes

Количество принудительных фиксаций на диск для избежания роста базы данных при работе вне режима MDBX_UTTERLY_NOSYNC.

```
public uint Flushes
```

Field Value

[uint](#)

Kicks

Количество обращений к механизму Handle-Slow-Readers.

```
public uint Kicks
```

Field Value

[uint](#)

MaxReaderLag

Максимальная наблюдаемая разница между последним и старейшим прочитанными MVCC-снимками.

```
public uint MaxReaderLag
```

Field Value

[uint](#)

MaxRetainedPages

Максимальное отмеченное количество страниц, удерживаемых от повторного использования.

```
public uint MaxRetainedPages
```

Field Value

[uint](#)

PnlMergeSelf

Метрики объема работы и стоимости слияния списков страниц (merge self).

```
public MdbxCommitLatency.MdbxPnlMerge PnlMergeSelf
```

Field Value

[MdbxCommitLatency.MdbxPnlMerge](#)

PnlMergeWork

Метрики объема работы и стоимости слияния списков страниц (merge work).

```
public MdbxCommitLatency.MdbxPnlMerge PnlMergeWork
```

Field Value

[MdbxCommitLatency.MdbxPnlMerge](#)

SelfCounter

Количество выполнений медленного пути GC для обслуживания и обновления самого GC.

```
public uint SelfCounter
```

Field Value

[uint](#)

SelfMajflt

Количество страничных ошибок внутри GC при выделении и подготовке страниц для самого GC.

```
public uint SelfMajflt
```

Field Value

[uint](#)

SelfRsteps

Количество итераций поиска внутри GC при выделении страниц для обслуживания самого GC.

```
public uint SelfRsteps
```

Field Value

[uint](#)

SelfRtimeMonotonic

Время "по стене часов", затраченное на чтение и поиск внутри GC для обслуживания самого GC.

```
public uint SelfRtimeMonotonic
```

Field Value

[uint](#)

SelfXpages

Количество запросов на выделение последовательностей страниц для самого GC.

```
public uint SelfXpages
```

Field Value

[uint](#)

SelfXtimeCpu

CPU-время в пользовательском режиме, затраченное на подготовку страниц из GC для обслуживания самого GC.

```
public uint SelfXtimeCpu
```

Field Value

[uint](#)

Wipes

Количество предыдущих надежных/стабильных зафиксированных точек, удаленных при работе в режиме MDBX_UTTERLY_NOSYNC.

```
public uint Wipes
```

Field Value

[uint](#)

Wloops

Количество итераций обновления GC больше 1, если были повторы/перезапуски.

```
public uint Wloops
```

Field Value

[uint](#)

WorkCounter

Количество медленных/глубоких поисков GC для размещения пользовательских данных.

```
public uint WorkCounter
```

Field Value

[uint](#)

WorkMajflt

Количество страничных ошибок внутри GC при выделении и подготовке страниц для пользовательских данных.

```
public uint WorkMajflt
```

Field Value

[uint](#)

WorkRsteps

Количество итераций поиска внутри GC при выделении страниц для пользовательских данных.

```
public uint WorkRsteps
```

Field Value

[uint](#)

WorkRtimeMonotonic

Время "по стене часов", затраченное на чтение и поиск внутри GC для пользовательских данных.

```
public uint WorkRtimeMonotonic
```

Field Value

[uint](#)

WorkXpages

Количество запросов на выделение последовательностей страниц для пользовательских данных.

```
public uint WorkXpages
```

Field Value

[uint](#)

WorkXtimeCpu

CPU-время в пользовательском режиме, затраченное на подготовку страниц из GC для пользовательских данных.

```
public uint WorkXtimeCpu
```

Field Value

[uint](#)

Struct MdbxCommitLatency.MdbxPnlMerge

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Метрики слияния списков страниц.

```
public struct MdbxCommitLatency.MdbxPnlMerge
```

Inherited Members

[ValueType.Equals\(object\)](#) , [ValueType.GetHashCode\(\)](#) , [ValueType.ToString\(\)](#) ,
[object.Equals\(object, object\)](#) , [object.GetType\(\)](#) , [object.ReferenceEquals\(object, object\)](#)

Fields

Calls

Количество вызовов слияния.

```
public uint Calls
```

Field Value

[uint](#)

Time

Время слияния.

```
public uint Time
```

Field Value

[uint](#)

Volume

Объем слияния.

```
public ulong Volume
```

Field Value

[ulong](#) 

Enum MdbxCopyFlags

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Флаги копирования окружения.

[Flags]

```
public enum MdbxCopyFlags : uint
```

Fields

`MDBX_CP_COMPACT = 1`

Сжимать данные при копировании.

`MDBX_CP_DEFAULT = 0`

Копировать без сжатия.

Enum MdbxCursorOp

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Операции курсора.

```
public enum MdbxCursorOp
```

Fields

`MDBX_FIRST = 0`

Получить первый элемент.

`MDBX_FIRST_DUP = 1`

Получить первый дубликат.

`MDBX_GET_BOTH = 2`

Получить пару ключ/данные.

`MDBX_GET_BOTH_RANGE = 3`

Получить ключ и первые данные, большие или равные указанным.

`MDBX_GET_CURRENT = 4`

Получить текущий элемент.

`MDBX_GET_MULTIPLE = 5`

Получить несколько дубликатов.

`MDBX_LAST = 6`

Получить последний элемент.

`MDBX_LAST_DUP = 7`

Получить последний дубликат.

`MDBX_NEXT = 8`

Получить следующий элемент.

`MDBX_NEXT_DUP = 9`

Получить следующий дубликат.

`MDBX_NEXT_MULTIPLE = 10`

Получить несколько следующих дубликатов.

`MDBX_NEXT_NODUP = 11`

Получить первый элемент следующего ключа.

`MDBX_PREV = 12`

Получить предыдущий элемент.

`MDBX_PREV_DUP = 13`

Получить предыдущий дубликат.

`MDBX_PREV_NODUP = 14`

Получить последний элемент предыдущего ключа.

`MDBX_SET = 15`

Установить ключ.

`MDBX_SET_KEY = 16`

Установить ключ и получить данные.

Enum MdbxDbFlags

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Флаги базы данных (таблицы).

[Flags]

```
public enum MdbxDbFlags : uint
```

Fields

MDBX_CREATE = 262144

Создать базу данных, если она не существует.

MDBX_DB_ACCEDE = 1073741824

Открыть существующую таблицу, созданную с неизвестными флагами.

MDBX_DB_DEFAULTS = 0

Флаги по умолчанию.

MDBX_DUPFIXED = 16

С фиксированным размером дубликаты (для оптимизации сравнения).

MDBX_DUPSORT = 4

Разрешить дубликаты ключей.

MDBX_INTEGERDUP = 32

Цифровые значения (для оптимизации сравнения).

MDBX_INTEGERKEY = 8

Цифровые ключи (для оптимизации сравнения).

MDBX_REVERSEDUP = 64

Обратное сравнение строковых значений дубликатов.

`MDBX_REVERSEKEY = 2`

Использовать обратный ключ (ключи хранятся в обратном порядке).

Enum MdbxDebugFlags

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Флаги отладки.

[Flags]

```
public enum MdbxDebugFlags : uint
```

Fields

MDBX_DBG_ASSERT = 1

Включить дорогостоящую проверку отладочных утверждений.

MDBX_DBG_AUDIT = 2

Включить дополнительные дорогостоящие проверки и глубокую верификацию списков страниц.

MDBX_DBG_DONT_UPGRADE = 64

Отключить автоматическое обновление формата БД.

MDBX_DBG_DUMP = 8

Включить включение мета-страниц БД в файлы core dump.

MDBX_DBG_JITTER = 4

Включить небольшие случайные задержки в критических точках.

MDBX_DBG_LEGACY_MULTIOPEN = 16

Разрешить многократное открытие окружения.

MDBX_DBG_LEGACY_OVERLAP = 32

Разрешить перекрытие read и write транзакций для одного потока.

MDBX_DBG_NONE = 0

Без флагов отладки.

Delegate MdbxDebugFunc

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Функция отладки (логирования).

```
public delegate void MdbxDebugFunc(MdbxLogLevel logLevel, byte* function, int line, byte*
fmt, byte* args)
```

Parameters

logLevel [MdbxLogLevel](#)

Уровень логирования.

function [byte](#)^{*}

Имя функции.

line [int](#)^{*}

Номер строки.

fmt [byte](#)^{*}

Форматная строка.

args [byte](#)^{*}

Аргументы формата.

Delegate MdbxDebugFuncNofmt

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Функция отладки без формата.

```
public delegate void MdbxDebugFuncNofmt(MdbxLogLevel logLevel, byte* function, int line, byte* msg)
```

Parameters

logLevel [MdbxLogLevel](#)

Уровень логирования.

function [byte](#)^{*}

Имя функции.

line [int](#)^{*}

Номер строки.

msg [byte](#)^{*}

Сообщение.

Delegate MdbxDefragProgressFunc

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Функция уведомления о прогрессе дефрагментации.

```
public delegate int MdbxDefragProgressFunc(void* ctx, MDBX_defrag_result* progress)
```

Parameters

ctx [void](#)^{*}

Пользовательский контекст.

progress [MDBX_defrag_result](#)^{*}

Указатель на структуру с результатами дефрагментации.

Returns

[int](#)^{*}

0 для продолжения, ненулевое значение для остановки.

Enum MdbxDefragStoppingReasons

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Причины остановки дефрагментации базы данных.

[Flags]

```
public enum MdbxDefragStoppingReasons : uint
```

Fields

MDBX_DEFRAG_ABORTED = 64

Дефрагментация была прервана пользователем с немедленным завершением.

MDBX_DEFRAG_DISCONTINUED = 4

Дефрагментация была прервана пользователем.

MDBX_DEFRAG_ENOUGH_THRESHOLD = 16

Достигнута цель дефрагментации, заданная пользователем.

MDBX_DEFRAG_ERROR = 128

Произошла ошибка во время дефрагментации.

MDBX_DEFRAG_LAGGARD_READER = 8

Читатели используют старые MVCC-снимки, блокирующие дефрагментацию.

MDBX_DEFRAG_LARGE_CHUNK = 2

Необходимо предварительное перемещение для формирования последовательности свободных страниц.

MDBX_DEFRAG_NOOBSTACLES = 0

Нет препятствий для дефрагментации.

MDBX_DEFRAG_STEP_SIZE = 1

Достигнут лимит размера транзакции шага.

`MDBX_DEFRAG_TIME_LIMIT = 32`

Истек лимит времени дефрагментации.

Enum MdbxEnvDeleteMode

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Режим удаления окружения.

```
public enum MdbxEnvDeleteMode : uint
```

Fields

`MDBX_ENV_DELETE_ALL = 1`

Удалить все файлы окружения.

`MDBX_ENV_DELETE_DATA = 0`

Только удалить данные, но оставить файлы.

Enum MdbxEnvFlags

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Флаги окружения.

[Flags]

```
public enum MdbxEnvFlags : uint
```

Fields

MDBX_ACCEDE = 1073741824

Использовать окружение/БД, уже открытые другим процессом(ами).

MDBX_ENV_DEFAULTS = 0

Флаги по умолчанию.

MDBX_EXCLUSIVE = 4194304

Исключительный/монопольный режим открытия окружения.

MDBX_LIFORECLAIM = 67108864

Использовать LRU-стратегию возврата страниц читателями.

MDBX_NOMEMINIT = 16777216

Не инициализировать malloc'ed память перед записью в файл данных.

MDBX_NOMETASYNC = 262144

Не сбрасывать метаданные на диск при коммите.

MDBX_NORDAHEAD = 8388608

Отключить предварительное чтение.

MDBX_NOSTICKYTHREADS = 2097152

Отвязать транзакции от потоков, насколько это возможно.

MDBX_NOSUBDIR = 16384

Отсутствие каталога окружения.

MDBX_PAGEPERTURB = 134217728

Отладка: заполнять/возмущать освобождаемые страницы.

MDBX_RDONLY = 131072

Режим только для чтения.

MDBX_SAFE_NOSYNC = 65536

Безопасный режим без синхронизации (данные сбрасываются периодически).

MDBX_UTTERLY_NOSYNC = 1114112

Полностью отключить синхронизацию (опасно для данных).

MDBX_VALIDATION = 8192

Дополнительная валидация структуры БД и содержимого страниц.

MDBX_WRITEMAP = 524288

Отображать данные в память с правами записи.

Struct MdbxGcInfo

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Информация о сборке мусора (GC) и использовании страниц.

```
public struct MdbxGcInfo
```

Inherited Members

[ValueType.Equals\(object\)](#)[↗] , [ValueType.GetHashCode\(\)](#)[↗] , [ValueType.ToString\(\)](#)[↗] ,
[object.Equals\(object, object\)](#)[↗] , [object.GetType\(\)](#)[↗] , [object.ReferenceEquals\(object, object\)](#)[↗]

Fields

GcReclaimable

Информация о возвращаемых (reclaimable) страницах.

```
public MdbxGcInfo.MdbxGcReclaimable GcReclaimable
```

Field Value

[MdbxGcInfo.MdbxGcReclaimable](#)

MaxReaderLag

Максимальное отставание читателя.

```
public uint MaxReaderLag
```

Field Value

[uint](#)[↗]

MaxRetainedPages

Максимальное количество удерживаемых страниц.

```
public nuint MaxRetainedPages
```

Field Value

[nuint](#)

PagesAllocated

Количество страниц, в настоящее время выделенных.

```
public nuint PagesAllocated
```

Field Value

[nuint](#)

PagesBacked

Количество страниц, в настоящее время отображенных файлом базы данных.

```
public nuint PagesBacked
```

Field Value

[nuint](#)

PagesGc

Количество всех страниц внутри GC, включая страницы В-дерева структуры самого GC.

```
public nuint PagesGc
```

Field Value

[nuint](#)

PagesTotal

Общее количество страниц в базе данных (верхний предел, определенный геометрией).

```
public nuint PagesTotal
```

Field Value

[nuint](#)

Struct MdbxGcInfo.MdbxGcReclaimable

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Информация о возвращаемых страницах.

```
public struct MdbxGcInfo.MdbxGcReclaimable
```

Inherited Members

[ValueType.Equals\(object\)](#)[☞](#) , [ValueType.GetHashCode\(\)](#)[☞](#) , [ValueType.ToString\(\)](#)[☞](#) , [object.Equals\(object, object\)](#)[☞](#) , [object.GetType\(\)](#)[☞](#) , [object.ReferenceEquals\(object, object\)](#)[☞](#)

Fields

Pages

Количество возвращаемых страниц, включая отложенные резервы в текущей write-транзакции.

```
public uint Pages
```

Field Value

[uint](#)[☞](#)

PgnoDistribution

Распределение возвращаемых страниц по файлу базы данных.

```
public MdbxChkHistogramData PgnoDistribution
```

Field Value

[MdbxChkHistogramData](#)

SpanHistogram

Гистограмма длиныSpanов последовательных возвращаемых страниц.

```
public MdbxChkHistogramData SpanHistogram
```

Field Value

[MdbxChkHistogramData](#)

Delegate MdbxGclterFunc

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Функция итерации GC.

```
public delegate int MdbxGcIterFunc(void* ctx, MDBX_txn* txn, ulong spanTxnid, uint spanPgno, uint spanLength, bool spanIsReclaimable)
```

Parameters

ctx [void](#)*

Пользовательский контекст.

txn [MDBX_txn](#)*

Транзакция.

spanTxnid [ulong](#)

ID транзакции span'a.

spanPgno [uint](#)

Номер начальной страницы span'a.

spanLength [uint](#)

Длина span'a (количество страниц).

spanIsReclaimable [bool](#)

Является ли span возвращаемым (reclaimable).

Returns

[int](#)

0 для продолжения, ненулевое значение для остановки.

Delegate MdbxHsrFunc

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Функция обработки медленных читателей (HSR - Handle Slow Readers).

```
public delegate int MdbxHsrFunc(MDBX_env* env, MDBX_txn* txn, uint pid, uint tid, ulong laggard, uint gap, nuint space, int retry)
```

Parameters

env [MDBX_env](#)*

Указатель на окружение.

txn [MDBX_txn](#)*

Указатель на транзакцию.

pid [uint](#)

ID процесса-читателя.

tid [uint](#)

ID потока-читателя.

laggard [ulong](#)

ID отстающей транзакции читателя.

gap [uint](#)

Задержка в чтениях.

space [nuint](#)

Оставшееся пространство.

retry [int](#)

Флаг повторной попытки.

Returns

[int](#)

0 в случае успеха, иначе код ошибки.

Enum MdbxLogLevel

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Уровни логирования.

```
public enum MdbxLogLevel
```

Fields

`MDBX_LOG_DEBUG = 5`

Отладочные сообщения.

`MDBX_LOG_DONTCHANGE = -1`

Не изменять текущие настройки (только для mdbx_setup_debug).

`MDBX_LOG_ERROR = 1`

Ошибки.

`MDBX_LOG_EXTRA = 7`

Дополнительные отладочные сообщения (дамп списков pgno).

`MDBX_LOG_FATAL = 0`

Критические условия (аварийные отказы).

`MDBX_LOG_MAX = 7`

Максимальный уровень логирования.

`MDBX_LOG_NOTICE = 3`

Уведомления.

`MDBX_LOG_TRACE = 6`

Трассировочные отладочные сообщения.

`MDBX_LOG_VERBOSE = 4`

Подробные информационные сообщения.

`MDBX_LOG_WARN = 2`

Предупреждения.

Enum MdbxOption

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Параметры окружения.

```
public enum MdbxOption
```

Fields

`MDBX_OPT_DUPLIMIT_LIMIT = 6`

Лимит предварительно выделенных элементов памяти для грязных страниц.

`MDBX_OPT_LOOSE_LIMIT = 5`

Лимит кэша грязных страниц для повторного использования в текущей транзакции.

`MDBX_OPT_MAX_DB = 0`

Максимальное количество именованных таблиц для окружения.

`MDBX_OPT_MAX_READERS = 1`

Максимальное количество потоков/слотов читателей.

`MDBX_OPT_NUM = 77`

Количество параметров.

`MDBX_OPT_NUMDBS = 15`

Количество таблиц.

`MDBX_OPT_OP_CACHEBYTES = 22`

Размер кэша в байтах.

`MDBX_OPT_OP_CACHELINES = 20`

Размер кэш-линии.

`MDBX_OPT_OP_CACHESIZE = 21`

Размер кэша в мегабайтах.

`MDBX_OPT_OP_COMPAT = 11`

Совместимость со старым интерфейсом.

`MDBX_OPT_OP_DEBUG = 75`

Режим отладки.

`MDBX_OPT_OP_FSLIMIT = 13`

Ограничение размера файла файловой системой.

`MDBX_OPT_OP_GC = 25`

Политика GC.

`MDBX_OPT_OP_IOBLK = 19`

Включить блокировку IO.

`MDBX_OPT_OP_MEMPOLICY = 23`

Политика управления памятью.

`MDBX_OPT_OP_MEMTEMPLATE = 14`

Шаблон памяти.

`MDBX_OPT_OP_METASYNC = 9`

Включить метасинхронизацию.

`MDBX_OPT_OP_MINCORE = 7`

Включить/отключить mincore().

`MDBX_OPT_OP_NOLOCK = 10`

Отключить файловую блокировку.

`MDBX_OPT_OP_PREFAULT_WRITE = 8`

Включить prefaul запись.

`MDBX_OPT_OP_RP_REINVOKE = 16`

Политика повторного использования читательских слотов.

`MDBX_OPT_OP_SCHEDULER = 12`

Настройка планировщика.

`MDBX_OPT_OP_TEST = 76`

Режим тестирования.

`MDBX_OPT_OP_THREADS = 24`

Политика потоков.

`MDBX_OPT_OP_UID = 17`

Включить уникальные идентификаторы.

`MDBX_OPT_OP_UPCBLK = 18`

Включить Unified Page Cache блоки.

`MDBX_OPT_OP_WALBUF = 32`

Размер буфера WAL.

`MDBX_OPT_OP_WALBUFCOUNT = 34`

Количество буферов WAL.

`MDBX_OPT_OP_WALBUFPOLICY = 33`

Политика буферизации WAL.

`MDBX_OPT_OP_WALCOMMIT = 42`

Политика фиксации WAL.

`MDBX_OPT_OP_WALCOMPRESS = 38`

Сжатие WAL.

`MDBX_OPT_OP_WALFLUSH = 31`

Политика сброса WAL.

`MDBX_OPT_OP_WALFREE = 41`

Политика освобождения WAL.

`MDBX_OPT_OP_WALGROW = 30`

Политика роста WAL.

`MDBX_OPT_OP_WALITEMS = 28`

Количество элементов WAL.

`MDBX_OPT_OP_WALMODE = 35`

Режим WAL.

`MDBX_OPT_OP_WALPAGES = 29`

Количество страниц WAL.

`MDBX_OPT_OP_WALPAGESIZE = 39`

Размер страницы WAL.

`MDBX_OPT_OP_WALPOLICY = 27`

Политика WAL.

`MDBX_OPT_OP_WALRESET = 44`

Политика сброса WAL.

`MDBX_OPT_OP_WALREUSE = 36`

Политика повторного использования WAL.

`MDBX_OPT_OP_WALROTATE = 46`

Политика ротации WAL.

`MDBX_OPT_OP_WALSIZE = 26`

Размер WAL.

`MDBX_OPT_OP_WALSIZE_LIMIT = 37`

Ограничение размера WAL.

`MDBX_OPT_OP_WALSLOTS = 40`

Количество слотов WAL.

`MDBX_OPT_OP_WALSWITCH = 45`

Политика переключения WAL.

`MDBX_OPT_OP_WALSYNC = 43`

Политика синхронизации WAL.

`MDBX_OPT_RP_AUGMENT_LIMIT = 4`

Лимит наращивания списка освобожденных/рециклированных номеров страниц.

`MDBX_OPT_SYNC_BYTES = 2`

Порог принудительного сброса буферов данных на диск.

`MDBX_OPT_SYNC_PERIOD = 3`

Относительный период с момента последней нестабильной фиксации для принудительного сброса.

Delegate MdbxPanicFunc

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Функция паники (обработчик критических ошибок).

```
public delegate void MdbxPanicFunc(byte* msg, byte* function, uint line, void* obj)
```

Parameters

msg [byte](#)^{*}

Сообщение об ошибке.

function [byte](#)^{*}

Имя функции, в которой произошла ошибка.

line [uint](#)

Номер строки.

obj [void](#)^{*}

Указатель на объект (окружение, транзакция и т.д.).

Delegate MdbxPredicateFunc

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Функция предиката для сканирования курсора.

```
public delegate int MdbxPredicateFunc(void* context, MdbxVal* key, MdbxVal* value, void* arg)
```

Parameters

context [void](#)^{*}

Пользовательский контекст.

key [MdbxVal](#)^{*}

Ключ.

value [MdbxVal](#)^{*}

Значение.

arg [void](#)^{*}

Дополнительный аргумент.

Returns

[int](#)^{*}

0 для продолжения, ненулевое значение для остановки.

Delegate MdbxPreserveFunc

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Функция сохранения данных для mdbx_replace_ex.

```
public delegate int MdbxPreserveFunc(void* context, MdbxVal* target, void* src, uint bytes)
```

Parameters

context [void](#)*

Пользовательский контекст.

target [MdbxVal](#)*

Целевой буфер.

src [void](#)*

Исходные данные.

bytes [uint](#)

Количество байт.

Returns

[int](#)

0 при успехе, иначе код ошибки.

Enum MdbxPutFlags

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Флаги операции вставки/обновления.

[Flags]

```
public enum MdbxPutFlags : uint
```

Fields

MDBX_ALLDUPS = 128

Все дубликаты (заменить все значения для ключа).

MDBX_APPEND = 131072

Добавить элемент в начало (для неуникальных ключей).

MDBX_APPENDDUP = 262144

Добавить дубликат в начало.

MDBX_CURRENT = 64

Текущее значение (обновить существующий ключ).

MDBX_MULTIPLE = 524288

Сохранить существующее значение при неудачной вставке.

MDBX_NODUPDATA = 32

Не записывать, если пара ключ-значение уже существует.

MDBX_NOOVERWRITE = 16

Вставить только если ключа нет.

MDBX_RESERVE = 65536

Только зарезервировать место для данных, не копировать их.

`MDBX_UPSERT = 0`

Вставить или обновить (UPSERT) — поведение по умолчанию.

Delegate MdbxReaderListFunc

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Функция перечисления читателей.

```
public delegate int MdbxReaderListFunc(void* ctx, int num, int slot, uint pid, uint thread,
ulong txnid, ulong since, MdbxVal* range)
```

Parameters

ctx [void](#)*

Пользовательский контекст.

num [int](#)

Номер слота читателя.

slot [int](#)

Индекс слота.

pid [uint](#)

ID процесса.

thread [uint](#)

ID потока.

txnid [ulong](#)

ID транзакции.

since [ulong](#)

Время начала транзакции.

range [MdbxVal](#)*

Диапазон данных.

Returns

[int](#) 

0 для продолжения, ненулевое значение для остановки.

Struct MdbxStat

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Статистика окружения MDBX.

```
public struct MdbxStat
```

Inherited Members

[ValueType.Equals\(object\)](#)[↗] , [ValueType.GetHashCode\(\)](#)[↗] , [ValueType.ToString\(\)](#)[↗] ,
[object.Equals\(object, object\)](#)[↗] , [object.GetType\(\)](#)[↗] , [object.ReferenceEquals\(object, object\)](#)[↗]

Fields

MsBranchPages

Количество страниц ветвей (внутренние страницы B+ дерева).

```
public uint MsBranchPages
```

Field Value

[uint](#)[↗]

MsDepth

Глубина дерева (глубина B+ дерева).

```
public uint MsDepth
```

Field Value

[uint](#)[↗]

MsEntries

Количество элементов данных.

```
public uint MsEntries
```

Field Value

[uint](#)

MsLeafPages

Количество листовых страниц.

```
public uint MsLeafPages
```

Field Value

[uint](#)

MsOverflowPages

Количество больших/переполняющих страниц.

```
public uint MsOverflowPages
```

Field Value

[uint](#)

MsPsize

Размер страницы базы данных в байтах.

```
public uint MsPsize
```

Field Value

[nuint](#) 

Delegate MdbxTableEnumFunc

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Функция перечисления таблиц.

```
public delegate int MdbxTableEnumFunc(void* ctx, MDBX_txn* txn, MdbxVal* name, uint flags, uint id)
```

Parameters

ctx [void](#)[↗]*

Пользовательский контекст.

txn [MDBX_txn](#)*

Транзакция.

name [MdbxVal](#)*

Имя таблицы.

flags [uint](#)[↗]

Флаги таблицы.

id [uint](#)[↗]

ID таблицы.

Returns

[int](#)[↗]

0 для продолжения, ненулевое значение для остановки.

Enum MdbxTxnFlags

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Флаги транзакции.

[Flags]

```
public enum MdbxTxnFlags : uint
```

Fields

MDBX_TXN_DEFAULTS = 0

Флаги по умолчанию (read-write транзакция).

MDBX_TXN_NOMETASYNC = 262144

Отключить метасинхронизацию для этой транзакции.

MDBX_TXN_NOSYNC = 65536

Безопасный режим без синхронизации для этой транзакции.

MDBX_TXN_NOWEAKING = 0

Использовать режим окружения для durability.

MDBX_TXN_RDONLY = 131072

Только для чтения.

MDBX_TXN_RDONLY_PREPARE = 16908288

Подготовить read-only транзакцию без запуска.

MDBX_TXN_TRY = 268435456

Не блокировать при запуске write-транзакции.

Struct MdbxTxnInfo

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Информация о транзакции MDBX.

```
public struct MdbxTxnInfo
```

Inherited Members

[ValueType.Equals\(object\)](#)[↗](#) , [ValueType.GetHashCode\(\)](#)[↗](#) , [ValueType.ToString\(\)](#)[↗](#) , [object.Equals\(object, object\)](#)[↗](#) , [object.GetType\(\)](#)[↗](#) , [object.ReferenceEquals\(object, object\)](#)[↗](#)

Fields

TxnId

ID транзакции. Для READ-ONLY транзакции соответствует снимку MVCC, который читается.

```
public ulong TxnId
```

Field Value

[ulong](#)[↗](#)

TxnPget

Количество операций получения страниц в транзакции.

```
public ulong TxnPget
```

Field Value

[ulong](#)[↗](#)

TxnReaderLag

Для READ-ONLY транзакции: отставание от последнего MVCC-снимка. Для WRITE транзакции (при scan_rlt=true): отставание самого старого читателя от текущей транзакции.

```
public ulong TxnReaderLag
```

Field Value

[ulong](#) 

TxnSpaceDirty

Для READ-ONLY: пространство, которое станет доступно после завершения транзакции. Для WRITE: обобщенный размер грязных страниц, созданных в транзакции.

```
public ulong TxnSpaceDirty
```

Field Value

[ulong](#) 

TxnSpaceLeftover

Для READ-ONLY: пространство, доступное для writer(s). Для WRITE: пространство внутри транзакции до ошибки MDBX_TXN_FULL.

```
public ulong TxnSpaceLeftover
```

Field Value

[ulong](#) 

TxnSpaceLimitHard

Верхняя граница размера файла базы данных.

```
public ulong TxnSpaceLimitHard
```

Field Value

[ulong](#) 

TxnSpaceLimitSoft

Текущий размер файла базы данных.

```
public ulong TxnSpaceLimitSoft
```

Field Value

[ulong](#) 

TxnSpaceRetired

Для READ-ONLY: общий размер страниц, удаленных зафиксированными write-транзакциями. Для WRITE: обобщенный размер удаленных страниц из-за Copy-On-Write.

```
public ulong TxnSpaceRetired
```

Field Value

[ulong](#) 

TxnSpaceUsed

Используемое пространство данной транзакцией.

```
public ulong TxnSpaceUsed
```

Field Value

Struct MdbxVal

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Структура для передачи ключей и данных в таблицах MDBX.

```
public struct MdbxVal
```

Inherited Members

[ValueType.Equals\(object\)](#)[↗] , [ValueType.GetHashCode\(\)](#)[↗] , [ValueType.ToString\(\)](#)[↗] ,
[object.Equals\(object, object\)](#)[↗] , [object.GetType\(\)](#)[↗] , [object.ReferenceEquals\(object, object\)](#)[↗]

Remarks

Значения, возвращаемые из таблицы, действительны только до следующей операции обновления или конца транзакции. Не изменяйте и не освобождайте их, так как они обычно указывают прямо в базу данных.

Constructors

MdbxVal(byte[])

Инициализирует новую структуру MdbxVal из управляемого массива байт.

```
public MdbxVal(byte[] data)
```

Parameters

data [byte](#)[↗][]

Массив байт.

MdbxVal(void*, nuint)

Инициализирует новую структуру MdbxVal.

```
public MdbxVal(void* pointer, nuint length)
```

Parameters

pointer [void](#)^{*}

Указатель на данные.

length [nuint](#)

Длина данных в байтах.

Fields

IovBase

Указатель на данные.

```
public void* IovBase
```

Field Value

[void](#)^{*}

IovLen

Длина данных в байтах.

```
public nuint IovLen
```

Field Value

[nuint](#)

Methods

AsSpan()

Получает данные в видеSpan byte.

```
public Span<byte> AsSpan()
```

Returns

[Span](#) <[byte](#)>

Enum MdbxWarmupFlags

Namespace: [MDBX.Native.Types](#)

Assembly: mdbx-dotnet.dll

Флаги прогрева окружения.

[Flags]

```
public enum MdbxWarmupFlags : uint
```

Fields

`MDBX_WARMUP_DATA = 1`

Прогреть страницы данных.

`MDBX_WARMUP_DEFAULT = 0`

Флаги по умолчанию.

`MDBX_WARMUP_META = 2`

Прогреть страницы метаданных.